Go 工程师的 Python 速成

面向大模型训练、推理与 AI Infra 的学习与实践指南

第 2 版 · 全部示例在 Python 3.14 上实测 · 版本信息核查至 2026-10-22

语法与对象模型 / 工程化与并发 / NumPy、CUDA 与 Triton / PyTorch

分布式训练 / LoRA 微调 / vLLM 推理 / AI Infra 容量规划与可观测

PYTHON FOR GO ENGINEERS

2026 年 10 月

目录

前言 6

本书写给谁 6

本书结构 6

第 2 版改了什么 6

实测环境 7

怎么用这本书 7

第 1 章 Python 心智模型：从 Go 到 Python 8

1.1 本章目标 8

1.2 Go 工程师视角 8

1.3 设计哲学的三个分歧 8

1.4 高频语法速通 10

1.5 函数：默认参数、变长参数、闭包、生成器 11

1.6 动手实践一：把 Go 风格的抓取器改写成 Pythonic 11

1.7 动手实践二：生成器管道流式处理大日志 12

1.8 常见坑（全部可运行：`python pitfalls.py`） 13

1.9 延伸阅读 15

第 2 章 核心原理：对象模型、GIL 与内存管理 16

2.1 本章目标 16

2.2 Go 工程师视角 16

2.3 一切皆对象与名称绑定 16

2.4 内存管理：引用计数 + 循环 GC 17

2.5 GIL：它保护什么，为什么 AI 场景下常常无所谓 18

2.6 版本时间线与关键特性 20

2.7 属性查找、描述符与元类 21

2.8 常见坑 23

2.9 延伸阅读 23

第 3 章 工程化：类型、并发与工具链 24

3.1 本章目标 24

3.2 Go 工程师视角 24

3.3 静态类型：提示而非契约 25

3.4 dataclass 与 pydantic v2：配置即代码 27

3.5 asyncio：协作式调度的单线程世界 29

3.6 并发模型选择：实测说话 . . . 32
3.7 工具链：uv + Ruff + pytest + profiling . . . 33
3.8 常见坑 . . . 33
3.9 延伸阅读 . . . 34

第 4 章 数值计算与 GPU：NumPy、CUDA 模型与 Triton . . . 35

4.1 本章目标 . . . 35
4.2 Go 工程师视角 . . . 35
4.3 ndarray：一个加强版 slice header . . . 36
4.4 动手实践一：向量化、视图与访存局部性（CPU 可跑） . . . 36
4.5 CUDA 最小知识：不写 CUDA 也必须懂 . . . 38
4.6 Triton：用 Python 写 GPU kernel . . . 38
4.7 动手实践二：Triton vector add（需要 GPU） . . . 39
4.8 常见坑 . . . 40
4.9 延伸阅读 . . . 40

第 5 章 PyTorch 深入：从张量到训练循环 . . . 41

5.1 本章目标 . . . 41
5.2 Go 工程师视角 . . . 41
5.3 张量内存模型 . . . 41
5.4 autograd：动态计算图 . . . 43
5.5 `nn.Module`：`__setattr__` 与参数注册 . . . 44
5.6 DataLoader、混合精度与 `torch.compile` . . . 45
5.7 动手实践二：完整训练循环 + 四档对比 . . . 45
5.8 常见坑 . . . 46
5.9 延伸阅读 . . . 47

第 6 章 分布式训练：DDP、FSDP 与并行策略 . . . 48

6.1 本章目标 . . . 48
6.2 Go 工程师视角 . . . 48
6.3 NCCL 与集合通信 . . . 49
6.4 并行策略地图 . . . 49
6.5 DDP：分桶同步与通信计算重叠 . . . 50
6.6 动手实践一：torchrun + DDP 最小完整例子（CPU 可跑） . . . 50
6.7 ZeRO 1/2/3 与显存构成 . . . 51
6.8 框架格局（2026-10） . . . 52
6.9 动手实践二：显存与通信估算（CPU 可跑） . . . 53
6.10 常见坑 . . . 53

6.11 延伸阅读 54

第 7 章 大模型微调：HuggingFace 生态与 LoRA 55

7.1 本章目标 55
7.2 Go 工程师视角 55
7.3 生态地图（2026-10-22 核查） 56
7.4 LoRA：低秩分解 56
7.5 QLoRA：把基座压进 4 bit 57
7.6 动手实践：Qwen3-0.6B + LoRA SFT 全链路 57
7.7 常见坑 59
7.8 延伸阅读 60

第 8 章 推理引擎：vLLM 与 SGLang 61

8.1 本章目标 61
8.2 Go 工程师视角 61
8.3 PagedAttention：把 KV cache 当虚拟内存管理 62
8.4 Continuous batching：迭代级调度 62
8.5 vLLM V1 架构与版本线 63
8.6 三个参数决定服务的性格 63
8.7 SGLang、TGI 与其他引擎 63
8.8 性能指标：TTFT、TPOT 与正确的压测口径 64
8.9 动手实践一：vLLM 离线批量推理（需要 NVIDIA GPU） 64
8.10 动手实践二：`vllm serve` 起 OpenAI 兼容服务（需要 NVIDIA GPU） 65
8.11 动手实践三：流式压测脚本（CPU 可跑） 65
8.12 常见坑 67
8.13 延伸阅读 67

第 9 章 AI Infra 工程：容量规划、量化、调度与可观测 68

9.1 本章目标 68
9.2 Go 工程师视角 68
9.3 KV cache 显存公式 69
9.4 动手实践一：KV cache 容量规划脚本（CPU 可跑） 69
9.5 量化：用精度评测换来的显存 70
9.6 PD 分离：两种负载拆到两个池子 71
9.7 Ray 与 K8s：推理服务的编排层 71
9.8 可观测分层：和 Go 后端同一套 Prometheus 纪律 72
9.9 动手实践二：Prometheus 抓取与告警（指标名已按 V1 修正） 73
9.10 常见坑 73

9.11 延伸阅读 74

第 10 章 90 天路线图与实战项目 75

10.1 本章目标 75

10.2 设计原则 75

10.3 三个 30 天 75

10.4 源码阅读：torchtitan → vLLM → transformers 77

10.5 实战项目清单 77

10.6 Go 工程师的优势与短板 78

10.7 结语 79

附录 A 第 1 版勘误对照 80

A.1 代码无法运行或结果错误 80

A.2 事实与版本 81

A.3 表述与交叉引用 82

附录 B 版本核查表 83

B.1 Python 解释器 83

B.2 主要库（PyPI 最新稳定版） 83

B.3 关键安全公告 84

附录 C 随书代码索引 85

前言

本书写给谁

写给有多年 Go 后台经验的工程师：熟悉并发、分布式系统、Kubernetes 和高并发服务，现在要进入大模型训练、推理和 AI Infra 领域，需要能独立读、写、改相关的 Python 代码。

本书不从“变量是什么”讲起。每一章都先给一张 Go 与 Python 的对照表，再讲清机制，最后给出可以直接运行的代码和实测输出。

本书结构

部分	章节	解决的问题
语言地基	第 1 – 3 章	能写、能读懂、能进生产：语法、对象模型、GIL、类型、并发、工具链
计算地基	第 4 – 5 章	数组内存模型、GPU 编程模型、PyTorch 训练循环
训练	第 6 – 7 章	分布式训练与显存账、LoRA 微调全链路
推理与 Infra	第 8 – 9 章	推理引擎原理、压测口径、容量规划、量化、可观测
落地	第 10 章	90 天路线图与 8 个递进项目

第 2 版改了什么

第 1 版的内容框架和选题是好的，问题集中在三类，第 2 版逐一处理，完整清单见附录 A。

1. **代码跑不通。** 第 1 版有 6 段主线代码无法运行：配置模型在导入时就报错、压测脚本引用未定义的变量、DDP 示例缺了整个训练循环、流式压测脚本的计时起点和并发控制缺失、容量规划脚本引用未定义的变量、训练基准在 CPU 上会因无条件调用 CUDA 同步而崩溃，且计时循环里根本没有反向传播。第 2 版所有代码都放在随书仓库里，并在本机实际运行过，书中的输出就是运行结果。
2. **版本与事实过时或有误。** 例如 pytest 当前是 9.x 而不是 8.4；vLLM 的 KV 占用指标已改名为 `vllm:kv_cache_usage_perc`，`vllm:prefix_cache_hit_rate` 并不存在；Ray 漏洞 CVE-2025-62593 的修复版本是 2.52.0，并且需要手动开启 token 认证；transformers v5 早在 2026 年 1 月就正式发布，不是 4 月才出 RC。附录 B 列出了 2026-10-22 从 PyPI 和官方文档核查的版本表。
3. **“预期输出”不可信。** 第 1 版的若干预期结果没有实测依据，有的甚至方向相反。例如它把“读 1/64 元素的跨步视图”和“读全部元素”比较，预期“跨步更慢”，本机复现的结果正好相反（0.20 ms 对 1.88 ms）；第 2 版改为读取相同数量的元素，实测按列读比按行读慢 11.4 倍。凡是第 2 版能在本机复现的数字，都换成了实测值；需要 GPU 才能复现的，明确标注为“引用数据”或“参考量级”。

此外，第 2 版补了几组第 1 版只讲了结论、没有给出证据的实验：同一份纯 Python 代码在有 GIL 和 free-threaded 两种解释器上的多线程扩展性；线程池、进程池、子解释器池的实测对比；生成器管道与“一次性读进内存”的峰值内存对比；在没有 GPU 的机器上用一个模拟的 LLM 服务调通流式压测脚本。

实测环境

项	版本
机器	Apple M4 Pro（8 性能核 + 4 能效核），macOS，arm64
Python	3.14.7（主环境）、3.14.7 free-threaded、3.15.0rc1、3.13.12
关键库	numpy 2.5.4、torch 2.14.1、pydantic 2.14.1、transformers 5.20.2、peft 0.21.2、trl 1.16.0
GPU	无 NVIDIA GPU；第 5、7 章用 Apple GPU（MPS）运行

需要 NVIDIA GPU 的实践（Triton kernel、vLLM 服务、NCCL 多卡）在本书中给出完整代码，但没有在本机运行，相关数字均注明来源。

怎么用这本书

- 每章的代码在随书目录 `code/chNN/` 下，可以直接 `uv run python xxx.py`。
- 标注“CPU 可跑”的实践在任何电脑上都能跑；标注“需要 GPU”的，可以先读代码，等有云 GPU 时再跑。
- 文中出现的版本号、日期、性能数字，都注明了核查日期或测量环境。AI 基础设施的版本变化很快，读到时请以官方文档为准。

第 1 章 Python 心智模型：从 Go 到 Python

1.1 本章目标

- 说清 Python 与 Go 在设计上的三个根本分歧：鸭子类型与显式接口、异常与 error 返回值、名称绑定与值语义。
- 掌握高频语法：切片、推导式、解包、f-string、match-case、海象运算符 `:=`，能直接读写。
- 掌握函数层的核心机制：默认参数、`*args`/`**kwargs`、闭包、生成器。
- 把一个 Go 风格的并发程序改写成 Python 风格，用生成器管道实现恒定内存的大文件处理。
- 识别 Go 工程师最容易踩的七个坑，每个坑都有可运行的复现脚本。

1.2 Go 工程师视角

Go	Python	关键差异
值语义，赋值即拷贝（slice、map、chan 的头部除外）	名称绑定，赋值从不拷贝对象	`b = a` 只是给同一个对象贴第二个标签
interface 隐式满足，编译期检查	鸭子类型；`Protocol` 可做静态检查	类型错误默认推迟到运行时
`error` 作为返回值，`if err != nil`	异常：`raise` / `try` / `except`	错误沿调用栈自动上抛，不需要层层透传
大括号	缩进即语法，约定 4 空格	缩进错了是语法错误
编译期类型检查	类型提示，解释器运行时不检查	注解是给人和检查器看的
`switch v { case x: }` 值比较	`match`/`case` 结构化模式匹配（3.10+）	`case` 里的裸名字是“捕获”，不是“比较”
`for _, v := range xs`	`for v in xs` + 迭代协议	`for` 建立在 `__iter__`/`__next__` 之上
`go.mod` / `go.sum`	`pyproject.toml` + `uv.lock`	新项目默认工具链是 uv（第 3 章）

版本基线。 本书示例以 Python 3.14 为主，语法下限 3.12（PEP 695 泛型语法从 3.12 开始）。截至 2026 年 10 月：3.15 刚发布；3.14 处于 bugfix 维护；3.13 在 2026-10-01 转为只修安全问题；3.10 在 2026 年 10 月结束支持。不要用系统自带的解释器，例如 RHEL 9 的系统 Python 是 3.9（已于 2025-10 结束上游支持，由 Red Hat 自行维护）。用 uv 管理解释器：`uv python pin 3.14`。

1.3 设计哲学的三个分歧

鸭子类型与显式接口

Go 里一个类型能否当 `io.Reader` 用，由编译器检查方法集。Python 的原则是“走起来像鸭子、叫起来像鸭子，就是鸭子”：一个对象能否当文件用，取决于它有没有 `read()` 方法，而不是它继承了谁。这让框架代码非常灵活（PyTorch 里任何实现了 `forward` 的 `nn.Module` 子类都能当一层用），代价是类型错误要到运行时才暴露。

补救手段是类型提示加静态检查器（pyright、mypy，第 3 章），以及 PEP 544 的 `Protocol`。它是“可以被静态检查的鸭子类型”，与 Go interface 最接近：

```
from typing import Protocol

class Serializable(Protocol):
    def to_json(self) -> str: ...        # 只有签名，没有实现

def dump(obj: Serializable) -> str:
    return obj.to_json()                 # 任何有 to_json 方法的对象都能传入
```

异常与 error 值

Go 把错误当普通值显式传递；Python 把错误当控制流：任何一层 `raise`，调用栈逐层展开，直到某层 `try/except` 捕获。两个工程后果：

- 主路径没有 `if err != nil` 的噪音，读起来干净；
- 异常可能从任何地方飞来，所以资源清理必须用 `try/finally` 或 `with`（上下文管理器），不能靠“记得在返回前关掉”。

Python 社区的惯例是 EAFP（Easier to Ask Forgiveness than Permission）：先做，再捕获特定异常，而不是先层层检查。

```
try:
    value = config["lr"]
except KeyError:
    value = 3e-4
# 这个场景其实有更地道的写法：value = config.get("lr", 3e-4)
```

捕获要精确。 `except Exception` 会把编程错误（拼错的属性名、类型错误）也一起吞掉，这比 Go 里忽略 `err` 更隐蔽。只捕获你预期并且知道怎么处理的异常类型。

名称绑定与值语义

这是最底层、也最容易被 Go 工程师低估的分歧。Python 的变量不是装值的盒子，而是贴在对象上的名字。`a = [1, 2]` 创建一个 list 对象并让 `a` 指向它；`b = a` 不拷贝任何东西，只是让 `b` 也指向它。函数传参同理，官方术语叫“共享式传参”（call by sharing）：

```
def append_eos(tokens: list[int]) -> None:
    tokens.append(2)            # 通过共享的对象原地修改：调用方可见

def rebind(tokens: list[int]) -> None:
    tokens = tokens + [2]       # 让形参指向一个新对象：调用方不可见

ids = [1]
append_eos(ids)                 # ids 变成 [1, 2]
rebind(ids)                     # ids 仍是 [1, 2]
```

可以这样记：Python 里所有名字都像指针，但没有取地址和解引用语法，也永远不会自动拷贝对象。第 2 章从对象模型角度把这层讲透。

1.4 高频语法速通

切片。 `s[i:j:k]` 含头不含尾，支持负索引（`s[-1]` 是最后一个元素）和步长（`s[::-1]` 反转）。**关键差异：list 切片产生拷贝，Go slice 共享底层数组。** 而 NumPy 数组和 PyTorch 张量的切片是共享内存的视图（第 4、5 章）。同一个语法，在 list 和数组上行为相反，这是最容易混淆的地方。

推导式。 `[f(x) for x in xs if pred(x)]` 一行完成 map + filter，dict 和 set 同理。圆括号版本 `(f(x) for x in xs)` 是生成器表达式，不立即求值、不占内存。3.12 起推导式在调用处内联执行（PEP 709），不再创建隐藏的函数作用域；3.15 起推导式里可以直接解包（PEP 798）：

```
lists = [[1, 2], [3], [4, 5]]
[*L for L in lists]            # Python 3.15: [1, 2, 3, 4, 5]
```

解包。 `a, b = b, a` 交换不需要临时变量；`first, *rest = xs` 拆出头部；函数定义处的 `*args, **kwargs` 收集多余参数，调用处的 `f(*args, **kw)` 反向展开。日志封装、装饰器、框架里到处是 `**kwargs` 透传。

f-string。 `f"lr={lr}, step={step}"` 内嵌表达式；调试时 `f"{lr=}"` 直接打印“变量名=值”（3.8+）；3.12 起（PEP 701）可以在 f-string 里嵌套同种引号。3.14 新增的 t-string（`t"..."`，PEP 750）不直接拼成字符串，而是保留模板结构，交给渲染函数做转义，适合拼 SQL、HTML 和 prompt。

match-case。 3.10 引入的结构化模式匹配不是 switch，它按形状解构数据：

```
match batch:
    case {"input_ids": ids, "labels": labels}:   # 映射模式：两个键都存在才匹配
        loss = train_step(ids, labels)
    case {"input_ids": ids}:                     # 只有输入：走推理
        logits = infer(ids)
    case _:
        raise ValueError(f"unknown batch format: {type(batch)}")
```

最大的坑：`case RED:` 里的裸名字会**捕获并绑定**一个新变量 `RED`，并且永远匹配成功。要比较值，用字面量或带点的名字（`case Color.RED:`）。坑七有可运行的复现。

海象运算符。 `:=`（3.8，PEP 572）在表达式里赋值，省掉“先算一次、再在 if 里判断一次”：

```
while chunk := f.read(1 << 20):       # 读一块，同时判断是否为空
    process(chunk)

if (n := len(queue)) > 1024:          # := 优先级低于比较运算符，必须加括号
    flush(queue, n)
```

注意 `x := 1, 2` 这种写法在语句层面不合法；写成 `(y := 1, 2)` 时，`y` 绑定的是 `1`，整个括号才是 tuple `(1, 2)`。

1.5 函数：默认参数、变长参数、闭包、生成器

函数是一等对象，可以赋值、传参、作为返回值，这一点与 Go 一致。差异集中在四件事上。

默认参数只求值一次。 `def f(x=[])` 里的 `[]` 在 `def` 语句执行时创建，之后所有调用共享它。正确做法是默认值写 `None`，函数体内再创建可变对象（坑一）。3.15 新增内置的 `sentinel()`（PEP 661），可以精确区分“没传参数”和“传了 None”。

`*args`/`kwargs` 是框架代码的通行证。`**` def train(model, *args, lr=3e-4, **kwargs)：多余的位置参数收进 `tuple` args，多余的关键字参数收进 `dict` kwargs，lr 在 *args` 之后，只能按名字传。

闭包按名字捕获。 闭包里的自由变量在**调用时**才去外层作用域按名字查找，而不是在定义时固化当前值。Go 闭包捕获的也是变量本身，但 Go 1.22 起 `for` 循环每次迭代都新建循环变量，所以 Go 里这个坑已经消失；Python 没有对应改动，`[lambda: i for i in range(3)]` 三个函数都返回 2（坑四）。

生成器是“可暂停的函数”。 函数体里出现 `yield`，调用它就不再执行函数体，而是返回一个生成器对象；每次 `next()` 运行到下一个 `yield` 停住，局部变量和执行位置原样保留。这是惰性、流式处理的基础。生成器只能遍历一次，耗尽后需要重新调用函数。生成器的“冻结—恢复”机制是协程的前身，`async`/`await`（第 3 章）就是沿这条路演化出来的。

1.6 动手实践一：把 Go 风格的抓取器改写成 Pythonic

目标：并发抓取一批 URL 的 `<title>`。先写一个按 Go 直觉“直译”的版本（手动线程 + 队列 + 返回 `(值, 错误)`），再改成 Python 风格（线程池 + 异常 + `with`）。CPU 可跑，需要联网。

```
uv init title-fetch && cd title-fetch
uv python pin 3.14
uv add httpx
```

Go 直译版：

```
def fetch_title_go_style(client: httpx.Client, url: str) -> tuple[str, str | None]:
    """像 Go 一样返回 (结果, 错误)。调用方必须记得检查第二个值。"""
    try:
        resp = client.get(url, timeout=10.0, follow_redirects=True)
    except Exception as exc:  # 把异常压扁成字符串，堆栈和类型信息都丢了
        return "", str(exc)
    if resp.status_code != 200:
        return "", f"status {resp.status_code}"
    m = TITLE_RE.search(resp.text)
    return (" ".join(m.group(1).split()) if m else "<no title>"), None
```

```
def main_go_style() -> dict[str, str]:
    jobs: queue.Queue[str | None] = queue.Queue()
    results: dict[str, str] = {}
    lock = threading.Lock()                         # 对应 sync.Mutex
    client = httpx.Client()

    def worker() -> None:                           # 对应 go func() { for url := range jobs {...} }()
        while (url := jobs.get()) is not None:
            title, err = fetch_title_go_style(client, url)
            with lock:
                results[url] = f"ERROR: {err}" if err is not None else title

    threads = [threading.Thread(target=worker) for _ in range(4)]
    for t in threads:
        t.start()
    for url in URLS:
        jobs.put(url)
```

```
    for _ in threads:                               # 相当于 close(jobs)
        jobs.put(None)
    for t in threads:                               # 相当于 wg.Wait()
        t.join()
    client.close()                                  # 相当于 defer client.Close()，但出异常时会漏掉
    return results
```

Pythonic 版：

```
def fetch_title(client: httpx.Client, url: str) -> str:
    """抓取单个 URL 并提取标题；失败直接抛异常，由调用方决定怎么处理。"""
    resp = client.get(url, timeout=10.0, follow_redirects=True)
    resp.raise_for_status()                         # 非 2xx 抛 HTTPStatusError
    if m := TITLE_RE.search(resp.text):             # walrus：匹配的同时判空
        return " ".join(m.group(1).split())         # 压缩标题里的换行和多余空格
    return "<no title>"
```

```
def main_pythonic() -> dict[str, str]:
    results: dict[str, str] = {}
    with httpx.Client() as client, ThreadPoolExecutor(max_workers=8) as pool:
        futures = {url: pool.submit(fetch_title, client, url) for url in URLS}
        for url, fut in futures.items():
            try:
                results[url] = fut.result()         # 子线程里的异常在这里原样重新抛出
            except httpx.HTTPError as exc:          # 只吞网络类错误，编程错误照常冒泡
                results[url] = f"ERROR: {type(exc).__name__}: {exc}"
    return results
```

两个版本的实测输出相同（其中一个地址是故意写的不存在域名；本机经过公司代理，所以报的是代理的 502）：

```
$ uv run python fetch_titles.py
https://www.python.org                          Welcome to Python.org
https://go.dev                                  The Go Programming Language
https://docs.astral.sh/uv/                      uv
https://this-domain-does-not-exist.invalid      ERROR: ProxyError: 502 Bad Gateway
```

对照着看三点：

1. **`with` 接管了 `defer`。** 直译版的 `client.close()` 写在最后，中途抛异常就会漏掉；`with` 无论正常结束还是异常都会关闭，等价于 `defer client.Close()`，而且一个 `with` 可以同时管理多个资源。
2. **线程池加 `future.result()` 取代了 goroutine + channel + WaitGroup 的骨架。** Python 没有语言级 channel，`queue.Queue` 是库而不是语法。
3. **异常比 `(string, error)` 携带更多信息。** 直译版把异常压扁成字符串，类型和堆栈都丢了；Pythonic 版只捕获 `httpx.HTTPError`，编程错误照常冒泡。这里用线程有效，是因为线程在等待网络时会释放 GIL（第 2 章）。上千并发时，应该改用第 3 章的 asyncio 版本。

1.7 动手实践二：生成器管道流式处理大日志

场景：分析一个每行一条 JSON 的训练日志，按每 10 万步统计 loss 均值，要求内存与文件大小无关。CPU 可跑。

```
def read_lines(path: str) -> Iterator[str]:
    with open(path, encoding="utf-8") as f:
        yield from f                                # 文件对象本身就是逐行迭代器
```

```
def parse_json(lines: Iterable[str]) -> Iterator[dict]:
    for line in lines:
        try:
            yield json.loads(line)
```

```
        except json.JSONDecodeError:              # 坏行跳过，不让整条管道崩掉
            continue
```

```
def loss_windows(records: Iterable[dict], size: int = 100_000) -> Iterator[tuple[int, int, float]]:
    """每 size 条记录产出一次 (起始 step, 结束 step, 平均 loss)；末尾不足 size 的部分也要产出。"""
    total, count, start, last = 0.0, 0, 0, 0
    for rec in records:
        if count == 0:
            start = rec["step"]
        total += rec["loss"]
        count += 1
        last = rec["step"]
        if count == size:
            yield start, last, total / count
            total, count = 0.0, 0
    if count:                                     # 尾部余量：第 1 版漏了这一段
        yield start, last, total / count
```

每一环都是生成器，数据逐行流过，任意时刻内存里只有一行数据加一个累加器。`loss_windows` 末尾的 `if count:` 处理最后不足一个窗口的余量，第 1 版漏掉了这一段，文件行数不是 10 万的整数倍时尾部数据会被静默丢弃。

实测（`python pipeline.py gen N` 生成测试文件，其中混入一行坏数据）：

文件	处理方式	耗时	峰值内存（RSS）
200 万行，86 MiB	生成器管道	1.5 秒	29.6 MiB
800 万行，349 MiB	生成器管道	5.7 秒	30.0 MiB
200 万行，86 MiB	先 `list()` 全部读入再统计	—	720 MiB

文件变成 4 倍，流式处理的峰值内存几乎不变；而把 200 万条记录一次性读成 Python 对象，内存涨到 720 MiB，是文件大小的 8 倍多。Python 对象很“重”：`{"step": 1234567, "loss": 1.2345}` 这样一个两字段的 dict，`sys.getsizeof` 显示 dict 本身 184 字节，两个值对象再加 52 字节，还没算列表里的指针和内存分配器的开销（第 2 章解释原因）。这就是生成器管道相对 Go `bufio.Scanner` 循环的价值，也是第 5 章 PyTorch `IterableDataset` 的心智原型。

1.8 常见坑（全部可运行：`python pitfalls.py`）

```
"""第 1 章常见坑：每一个都可以直接运行看到现象。python pitfalls.py"""
import copy


def section(title: str) -> None:
    print(f"\n== {title}")


section("坑一：可变默认参数")
def add_token(token, tokens=[]):                  # noqa: B006 —— 故意写错
    tokens.append(token)
    return tokens

print(add_token("a"), add_token("b"))             # 两次调用共享同一个 list
print("默认值对象挂在函数上:", add_token.__defaults__)

def add_token_fixed(token, tokens=None):
    tokens = [] if tokens is None else tokens
    tokens.append(token)
    return tokens

print(add_token_fixed("a"), add_token_fixed("b"))
```

```
section("坑二：== 与 is")
a, b = 256, 256
print("256 is 256:", a is b)
x = int("257")                                      # 运行时构造，避开编译期常量合并
y = int("257")
print("257 == 257:", x == y, "| 257 is 257:", x is y)

section("坑三：浅拷贝")
cfg = {"optimizer": {"lr": 3e-4}, "steps": 100}
shallow = cfg.copy()
shallow["optimizer"]["lr"] = 1e-3
print("浅拷贝后原 cfg 的 lr:", cfg["optimizer"]["lr"])
deep = copy.deepcopy(cfg)
deep["optimizer"]["lr"] = 5e-5
print("深拷贝改成 5e-5 后，原 cfg 的 lr 仍是:", cfg["optimizer"]["lr"])

section("坑四：闭包延迟绑定")
callbacks = [lambda: i for i in range(3)]
print("延迟绑定:", [f() for f in callbacks])
callbacks = [lambda i=i: i for i in range(3)]
print("默认参数固化:", [f() for f in callbacks])

section("坑五：迭代中修改列表")
xs = [1, 2, 2, 3, 2, 4]
for v in xs:
    if v == 2:
        xs.remove(v)
print("边遍历边删:", xs)
xs = [1, 2, 2, 3, 2, 4]
print("推导式重建:", [v for v in xs if v != 2])

section("坑六：海象运算符的优先级")
t = (n := 1, 2)
print("t =", t, "| n =", n)                         # n 绑定的是 1，整个括号才是 tuple

section("坑七：match-case 的裸名字是捕获")
RED = "red"
def color_of(v):
    match v:
        case "red":                                 # 字面量：比较值
            return "字面量匹配"
        case _:
            return "其他"
def broken(v):
    match v:
        case RED:                                   # 注意：这里 RED 是新变量，任何值都能匹配
            return f"捕获到 {RED!r}"
print(color_of("blue"), "|", broken("blue"))
```

实测输出：

```
== 坑一：可变默认参数
['a', 'b'] ['a', 'b']
默认值对象挂在函数上: (['a', 'b'],)
['a'] ['b']
== 坑二：== 与 is
256 is 256: True
257 == 257: True | 257 is 257: False
== 坑三：浅拷贝
浅拷贝后原 cfg 的 lr: 0.001
深拷贝改成 5e-5 后，原 cfg 的 lr 仍是: 0.001
== 坑四：闭包延迟绑定
延迟绑定: [2, 2, 2]
默认参数固化: [0, 1, 2]
== 坑五：迭代中修改列表
边遍历边删: [1, 3, 2, 4]
推导式重建: [1, 3, 4]
== 坑六：海象运算符的优先级
```

```
t = (1, 2) | n = 1
== 坑七：match-case 的裸名字是捕获
其他 | 捕获到 'blue'
```

坑	原因	对策
一：可变默认参数	默认值在 `def` 执行时只创建一次，存在 `f.__defaults__` 里被所有调用共享	默认值写 `None`；dataclass 用 `field(default_factory=list)`
二：`==` 与 `is` 混用	`is` 比较对象身份；CPython 缓存了 -5 到 256 的小整数，让 `is` 偶尔“碰巧正确”	比较值一律用 `==`；`is` 只用于 `None`、`True`/`False` 和自定义单例
三：浅拷贝	`dict.copy()`、`list.copy()`、`copy.copy()` 只拷第一层	嵌套结构用 `copy.deepcopy()`；配置对象优先做成不可变（`frozen=True` 的 dataclass、3.15 的 `frozendict`）
四：闭包延迟绑定	闭包按名字在调用时查找	`lambda i=i: i` 用默认参数固化当前值，或 `functools.partial`
五：迭代中修改列表	列表迭代器按下标前进，删除元素后后面的元素前移，被跳过	用推导式重建新列表
六：海象优先级	`:=` 优先级很低	在表达式里总是加括号
七：`case` 裸名字	裸名字是捕获模式，永远匹配并遮蔽外层变量	用字面量或 `Enum.MEMBER` 这样的带点名字

1.9 延伸阅读

- Python 官方教程（The Python Tutorial）：本章所有语法的权威来源。
- Python Data Model 文档：理解“一切皆对象”与魔术方法，读完本章即可通读。
- PEP 572（海象运算符）、PEP 634 – 636（结构化模式匹配）、PEP 544（Protocol）、PEP 798（推导式解包）。

第 2 章 核心原理：对象模型、GIL 与内存管理

2.1 本章目标

- 用“一切皆对象 + 名称绑定”统一解释 Python 的赋值、传参与属性访问。
- 说清 CPython 内存管理的两层机制：引用计数（即时回收）与分代循环 GC（处理引用环），并能动手观测。
- 解释 GIL 保护的是什么、为什么 AI 场景下它通常不是瓶颈，并用实测数据说明 free-threaded 构建的现状。
- 掌握 3.13/3.14/3.15 与 AI Infra 相关的特性和版本时间线。
- 理解属性查找链、描述符与魔术方法，能读懂 `nn.Module` 这类框架的“魔法”。

2.2 Go 工程师视角

Go	Python（CPython）	关键差异
值语义；`*T` 显式指针	一切皆对象，名字都是引用	变量是贴在对象上的标签
tracing GC，周期性标记清扫	引用计数为主 + 分代循环 GC 处理引用环	大部分对象在最后一个引用消失时立即回收
goroutine 多核并行	线程受 GIL 约束（free-threaded 构建除外）	纯 Python 的 CPU 密集多线程不能并行
编译期决定内存布局	每个对象带对象头（引用计数 + 类型指针），属性默认存在 `__dict__` 里	Python 对象比 Go 结构体重得多
struct + 方法集	class + 属性查找链（描述符 → 实例 `__dict__` → 类）	方法本质是存在类上的描述符
运行时内部基本不可见	`sys`、`gc`、`id()` 可以直接观测解释器	原理可以直接写代码验证

2.3 一切皆对象与名称绑定

int、str、函数、类、模块、`None`，乃至“类型”本身，都是对象：有一块内存、一个类型指针、一个引用计数。`type(int)` 是 `type`，`type(type)` 还是 `type`。变量不是 Go 意义上的“存储位置”，而是命名空间（本质上是一个 dict）里名字到对象的映射。由此可以解释三件事：

- **赋值不拷贝。** `b = a` 让 `b` 指向 `a` 所指的对象，引用计数加 1。要拷贝必须显式调用 `copy.copy()`、`copy.deepcopy()` 或类型自己的构造方法。
- **传参是共享式传参。** 形参是实参所指对象的又一个名字。函数内重新绑定形参不影响外部；原地修改可变对象则外部可见。这与 Go 传 slice 的体验接近，区别是 Go 里 slice 是例外，Python 里这是全部。
- **可变性决定一切。** 不可变对象（int、float、str、tuple、frozenset、bytes，3.15 起还有 `frozendict`）没有原地修改的方法，对它们的“修改”必然产生新对象；可变对象（list、dict、set）两者都可以。可变性还决定了能否作 dict 的键（需可哈希）、能否安全地作默认参数。注意 `([],)` 这样的 tuple：tuple 本身不可变，但它里面的 list 照样可变，不可变性不会向内传递。

2.4 内存管理：引用计数 + 循环 GC

CPython 给每个对象维护一个引用计数：赋值、传参、放入容器加 1；名字删除、重新绑定、容器释放减 1；归零时对象立即回收。对照 Go 的 tracing GC，引用计数的好处是回收及时、可预测；代价是每次赋值都有计数开销，而且处理不了引用环：A 引用 B、B 引用 A，即使外部已经没有任何路径能到达它们，计数也永远不为零。

补上这个洞的是 `gc` 模块：一个分代循环垃圾回收器，周期性扫描容器对象，找出不可达的环并回收。一个值得记录的版本插曲：3.14.0 到 3.14.4 使用了新的增量 GC，因为生产环境报告了明显的内存压力，3.14.5 回退到 3.13 的分代 GC，3.15 也沿用分代 GC（见 3.15 的 What's New）。“基准测试好看”和“生产环境稳定”是两回事，这个判断标准在 AI Infra 选型中同样适用。

几个工程要点：

- `id(x)` 返回对象身份（CPython 里就是内存地址），`is` 比较的就是它。对象销毁后地址可能被复用，不要把 `id` 当长期标识存起来。
- `__del__` 不是 C++ 析构函数：调用时机由 GC 决定，解释器退出时可能不调用。释放资源用 `with`，不用 `__del__`。
- PEP 683（3.12）引入“永生对象”：小整数、`None` 等对象的引用计数不再变化。这是 free-threading 改造的前置工作之一。

动手实践一：观测引用计数与循环 GC（CPU 可跑）

```
"""第 2 章实践一：观测引用计数与循环 GC。python refcount_demo.py"""
import gc
import sys

print(sys.version.split()[0])

# 1. 引用计数的增减（getrefcount 的实参本身会临时 +1）
x = [1, 2, 3]
base = sys.getrefcount(x)
print("基线:", base)
y = x
print("y = x 之后:", sys.getrefcount(x) - base, "(相对基线)")
box = [x, x]
print("放进容器两次之后:", sys.getrefcount(x) - base)
del y, box
print("删除 y 和容器之后:", sys.getrefcount(x) - base)

# 2. 引用环：引用计数回收不了，要靠循环 GC
class Node:
    def __init__(self, name: str) -> None:
        self.name = name
        self.peer: "Node | None" = None
    def __del__(self) -> None:                    # 仅用于观察回收时机，生产代码不要依赖 __del__
        print(f"  回收 Node({self.name})")

print("\n不成环：del 之后立刻回收")
solo = Node("solo")
del solo

print("\n成环：del 之后没有回收")
a, b = Node("a"), Node("b")
a.peer, b.peer = b, a
del a, b
print("  （到这里还没有任何回收输出）")
found = gc.collect()                              # 手动跑一轮循环 GC
```

```
print(f"gc.collect() 找到不可达对象 {found} 个")
```

在 3.14.7 上的实测输出：

```
基线: 2
y = x 之后: 1 (相对基线)
放进容器两次之后: 3
删除 y 和容器之后: 0

不成环: del 之后立刻回收
  回收 Node(solo)

成环: del 之后没有回收
  （到这里还没有任何回收输出）
  回收 Node(a)
  回收 Node(b)
gc.collect() 找到不可达对象 7 个
```

两点值得注意：

1. 不成环的对象在 `del` 的那一刻就被回收了，输出紧跟在 `del` 后面。这和 Go“等下一轮 GC”的心智完全不同。
2. 成环的两个对象直到 `gc.collect()` 才被回收。`gc.collect()` 的返回值是“找到的不可达对象数”，它除了两个 Node，还包括它们各自的 `__dict__` 等内部对象，所以不是 2。这个数字在不同版本、不同构建上不同（3.13.12 上是 2，3.14 free-threaded 构建上是 35），不要把它当作稳定的断言条件。

2.5 GIL：它保护什么，为什么 AI 场景下常常无所谓

GIL（Global Interpreter Lock）是 CPython 的一把全局锁：同一时刻只有一个线程能执行 Python 字节码。它存在的根本原因是保护引用计数：没有 GIL，每次计数加减都要加锁或用原子操作，单线程性能会明显下降；同时，海量 C 扩展也因此不必考虑线程安全。

对 Go 工程师最反直觉的推论是：**纯 Python 代码的多线程不能利用多核**。但 GIL 在两种情况下会释放：

1. 线程做阻塞 I/O（网络、磁盘）时，所以 I/O 密集的多线程依然有效，第 1 章的线程池抓网页就是靠这个；
2. C 扩展在长时间计算前主动释放 GIL，NumPy 的矩阵乘、PyTorch 的算子都是如此。

由此得到 AI 场景的关键判断：**训练与推理的重计算发生在 C++/CUDA 层，Python 线程主要负责调度，GIL 大多数时候不在关键路径上。** 训练循环里 Python 侧的开销主要是算子下发和胶水代码；推理框架（如 vLLM）的调度循环是纯 Python 热路径，但那是“单线程有多快”的问题，而不是“多线程能否并行”的问题。

Free-threading 现状（2026 年 10 月）

- **路线图**：PEP 703（2023 年 10 月接受）定了三个阶段。3.13 提供实验性的 free-threaded 构建（可执行文件带 `t` 后缀，如 `python3.13t`）；PEP 779（2025 年 6 月接受）让 3.14 起 free-threaded 成为官方支持的可选构建，但默认构建仍带 GIL；成为默认构建没有时间表。
- **单线程开销**：3.13 的 free-threaded 构建单线程明显变慢（主要因为特化自适应解释器在该模式下被关闭）；3.14 重新启用后，官方给出的单线程开销约为 5% – 10%。下面的实测中，M4 Pro 上两种构建的单线程耗时几乎相同（0.21 秒对 0.20 秒）。

- **技术原理**：去掉 GIL 后，代价转移到了引用计数上：偏向引用计数（对象的属主线程走无锁快路径）、延迟引用计数、每对象一把小锁。不是消灭了锁，而是把一把大锁拆成了很多小锁，并给属主线程开快车道。
- **最大的坑**：C 扩展必须显式声明支持 free-threading，否则解释器在导入它时会为整个进程**静默重新启用 GIL**。用 `sys._is_gil_enabled()` 自查。主流科学计算库（NumPy 等）已提供 free-threaded 版本的安装包，下面的实测在 3.14t 上导入 NumPy 之后 GIL 仍保持关闭；其它库以 py-free-threading.github.io 的兼容性追踪为准。
- **安装**：`uv python install 3.14t`，可执行文件 `python3.14t` 与常规构建共存。3.15 的 PEP 803 引入面向 free-threaded 构建的稳定 ABI（`abi3t`），打包问题正在收敛。
- **注意**：没有 GIL 不等于线程安全。过去被 GIL 粗粒度调度掩盖的“先检查后操作”竞态会暴露出来，共享状态该加锁还得加锁。

动手实践二：同一份代码，两种解释器（CPU 可跑）

```
def cpu_task(n: int) -> int:
    s = 0
    for i in range(n):                          # 纯 Python 字节码：执行期间一直持有 GIL
        s += i * i
    return s
```

```
def run(fn, args, n_threads: int, repeat: int = 3) -> float:
    best = float("inf")
    for _ in range(repeat):                     # 取 3 次中的最好成绩，减少噪声
        ts = [threading.Thread(target=fn, args=args) for _ in range(n_threads)]
        t0 = time.perf_counter()
        for t in ts:
            t.start()
        for t in ts:
            t.join()
        best = min(best, time.perf_counter() - t0)
    return best
```

实测（M4 Pro，8 性能核 + 4 能效核；每个线程执行同样多的计算，“总吞吐”= 线程数 × 单线程耗时 / 实际耗时）：

线程数	3.14.7 常规构建：耗时	总吞吐	3.14.7 free-threaded：耗时	总吞吐
1	0.21 s	1.0×	0.20 s	1.0×
2	0.41 s	1.0×	0.21 s	1.9×
4	0.85 s	1.0×	0.29 s	2.8×
8	1.67 s	1.0×	0.33 s	4.8×

常规构建下，线程越多耗时越长，总吞吐始终是 1 倍：GIL 让它们排队执行。free-threaded 构建下，吞吐随线程数增长，但没有线性扩展：这台机器只有 8 个性能核，而且多个线程同时运行时单核频率会下降。

NumPy 的情况可以单独验证：把 BLAS 限制为单线程（`VECLIB_MAXIMUM_THREADS=1`）后，4 个线程同时做 1500×1500 矩阵乘，在**常规构建**上总吞吐是单线程的 2.1 倍。也就是说，即使有 GIL，NumPy 的计算也能并行，因为它在计算期间释放了 GIL。不限制 BLAS 线程数时，单个矩阵乘本身已经用满多核，多开线程没有收益（实测 1.1 倍）。

2.6 版本时间线与关键特性

截至 2026-10-22（来源：python.org 发布日程与 endoflife.date）：

版本	首发	状态	结束支持
3.15	2026-10-09	最新稳定版，bugfix 维护	2031-10
3.14	2025-10-07	bugfix 维护（最新补丁 3.14.9，2026-10-21）	2030-10
3.13	2024-10-07	2026-10-01 起只修安全问题	2029-10
3.12	2023-10-02	只修安全问题	2028-10
3.11	2022-10-24	只修安全问题	2027-10
3.10	2021-10-04	2026 年 10 月结束支持	已结束

选型建议：新代码面向 3.14 编写，语法下限 3.12。注意 3.15 刚发布，生态还没跟上：例如截至 2026-10-22，Triton 3.8.0 要求 Python < 3.15，vLLM 0.32.0 也要求 Python < 3.15，深度学习项目暂时不要上 3.15。

与 AI Infra 相关的特性：

- 3.13：实验性 free-threaded 构建与实验性 JIT 首次出现（JIT 默认关闭，生产暂不依赖）。
- 3.14：free-threaded 转为官方支持（PEP 779）；t-string（PEP 750）；注解延迟求值（PEP 649/749），前向引用不再需要加引号；`concurrent.interpreters`（PEP 734），每个子解释器有独立的 GIL，是线程与多进程之间的第三条路；Linux 上 `multiprocessing` 默认启动方式从 `fork` 改为 `forkserver`。关于 `from __future__ import annotations`：PEP 749 规定 3.14 里它照常工作，最早要等 3.13 结束支持之后的版本才开始发出弃用警告，再过至少两个版本才会移除。
- 3.15：显式懒加载导入（PEP 810，`lazy import json`，首次使用时才真正加载，启动更快，代价是导入错误推迟到首次使用时才暴露）；内置 `frozendict`（PEP 814，键值都可哈希时它本身可哈希，**不是** dict 的子类，`isinstance(x, dict)` 为 `False`）；内置 `sentinel`（PEP 661）；UTF-8 成为默认编码（PEP 686，`open()` 不再看系统 locale，主要影响 Windows）；新的 `profiling` 包（PEP 799），`profiling.tracing` 是原来的 cProfile，`profiling.sampling` 是新的采样分析器，可以按进程号挂到运行中的进程上。

以上 3.15 特性都在 3.15.0rc1 上验证过（`code/ch02/py315_features.py`）：

```
3.15.0rc1
PEP 798 推导式解包: [1, 2, 3, 4, 5]
PEP 814 frozendict: frozendict({'a': 1}) | 可哈希: True | 是 dict 子类: False
PEP 661 sentinel: MISSING sentinel
PEP 686 UTF-8 模式: 1
PEP 810 lazy import: {"ok": true}
```

2.7 属性查找、描述符与元类

Python 框架源码难读，根源不是语法，而是属性访问被层层拦截。掌握下面这条查找链，大多数“魔法”都能拆开。`obj.x` 的查找顺序：

1. **数据描述符**：`type(obj)` 的类（及其父类）上定义了 `__set__` 或 `__delete__` 的对象，优先于实例 `__dict__`；
2. **实例** `__dict__`；
3. **非数据描述符与普通类属性**（只定义了 `__get__` 的，比如普通方法）；
4. 以上都没找到，才调用 `__getattr__`（如果定义了）。

`__getattribute__` 是每次属性访问都要经过的总入口，上面的查找链就是它的默认实现，一般不要重写它。

描述符就是实现了 `__get__`/`__set__`/`__delete__` 的类的实例。它是对象模型的底层机制：`property` 是描述符；普通方法也是描述符，它的 `__get__` 负责把函数绑定成方法（给第一个参数塞进 `self`）；`classmethod`、`staticmethod` 同样如此。

`nn.Module` 重写了 `__setattr__`：你写 `self.fc = nn.Linear(...)` 时，它拦截这次赋值，把子模块登记进 `self._modules`、把参数登记进 `self._parameters`。这正是 `model.parameters()` 能自动收集全部参数、`model.to("cuda")` 能递归搬运的原因（第 5 章）。还有一个细节：隐式调用的魔术方法只在类型上查找，`len(x)` 走的是 `type(x).__len__`，在实例上塞一个 `__len__` 没有用。

元类是“类的类”：`class` 语句执行时，由元类（默认是 `type`）创建出类对象，ORM 的“定义子类即注册表结构”就靠它。但 3.6 引入的 `__init_subclass__`（子类定义时回调）和 `__set_name__`（描述符得知自己的属性名）覆盖了大部分元类场景，而且简单得多。写新代码优先用这两个钩子，读老框架时才需要直面元类。

动手实践三：描述符做字段校验、`__getattr__` 做懒加载（CPU 可跑）

```
"""第 2 章实践二、三：描述符做字段校验；__getattr__ 做懒加载。python descriptors.py"""
import json
import os
import tempfile


class Range:
    """限定取值范围的数值字段：在类上声明一次，所有实例赋值时自动校验。"""

    def __init__(self, lo: float, hi: float) -> None:
        self.lo, self.hi = lo, hi

    def __set_name__(self, owner: type, name: str) -> None:   # 类创建时回调，拿到属性名
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None:                                       # 通过类访问时返回描述符本身
            return self
        return obj.__dict__[self.name]

    def __set__(self, obj, value: float) -> None:             # 定义了 __set__ → 数据描述符，优先级高于实例 ↩
        __dict__
        if not (self.lo <= value <= self.hi):
            raise ValueError(f"{self.name} must be in [{self.lo}, {self.hi}], got {value}")
        obj.__dict__[self.name] = value


class SamplerConfig:
    temperature = Range(0.0, 2.0)
    top_p = Range(0.0, 1.0)

    def __init__(self, temperature: float = 0.7, top_p: float = 1.0) -> None:
```

```
        self.temperature = temperature                      # 触发 Range.__set__
        self.top_p = top_p


cfg = SamplerConfig()
print("合法:", cfg.temperature, cfg.top_p)
try:
    cfg.top_p = 1.5
except ValueError as e:
    print("越界:", e)
print("类属性是描述符本身:", type(SamplerConfig.__dict__["top_p"]).__name__)
print("实例 __dict__:", cfg.__dict__)


class LazyConfig:
    """首次访问某个配置名时才去读 <dir>/<name>.json，之后命中实例 __dict__，不再走 __getattr__。"""

    def __init__(self, search_dir: str) -> None:
        self._search_dir = search_dir
        self.loads = 0

    def __getattr__(self, name: str):                       # 只有常规查找全部失败时才会调用
        if name.startswith("_"):                             # 防御：拷贝/反序列化时 _search_dir 可能还不存在
            raise AttributeError(name)
        path = os.path.join(self._search_dir, f"{name}.json")
        if not os.path.exists(path):
            raise AttributeError(f"config {name!r} not found at {path}")
        with open(path, encoding="utf-8") as f:
            value = json.load(f)
        self.loads += 1
        setattr(self, name, value)                          # 写回实例 __dict__，下次查找在第 2 步就命中
        return value


with tempfile.TemporaryDirectory() as d:
    with open(os.path.join(d, "train.json"), "w", encoding="utf-8") as f:
        json.dump({"lr": 3e-4}, f)
    lc = LazyConfig(d)
    print("\n第一次:", lc.train, "读文件次数", lc.loads)
    print("第二次:", lc.train, "读文件次数", lc.loads)
    try:
        lc.missing
    except AttributeError as e:
        print("不存在:", e)
```

实测输出：

```
合法: 0.7 1.0
越界: top_p must be in [0.0, 1.0], got 1.5
类属性是描述符本身: Range
实例 __dict__: {'temperature': 0.7, 'top_p': 1.0}

第一次: {'lr': 0.0003} 读文件次数 1
第二次: {'lr': 0.0003} 读文件次数 1
不存在: config 'missing' not found at .../missing.json
```

几个要点：

- `Range` 定义了 `__set__`，是数据描述符，所以 `cfg.top_p = 1.5` 一定会经过它，即使实例 `__dict__` 里已经有同名键。值存在实例 `__dict__` 里，描述符对象本身只有一份，挂在类上。这是 pydantic `Field(ge=..., le=...)` 的机制原型（pydantic v2 的校验在 Rust 内核里实现，但协议相同）。
- `LazyConfig` 第一次访问 `train` 时，正常查找失败，才调用 `__getattr__`；它把结果 `setattr` 回实例 `__dict__`，第二次访问在查找链第 2 步就命中，不再触发 `__getattr__`。
- `__getattr__` 里第一行 `if name.startswith("_"): raise AttributeError` 不是多余的。`copy.copy()`、`pickle` 这类工具会在对象还没完成 `__init__` 时访问属性，这时 `self._search_dir` 不存在，访问它又会进入 `__getattr__`，形成无限递归。第 1 版的写法有这个隐患。

2.8 常见坑

坑	现象	原因	对策
用 `is` 比较数值或字符串	`if step is 1000:` 有时成立有时不成立（3.8 起还会给出 `SyntaxWarning`）	小整数和部分字符串被缓存，`is` 碰巧为真	值比较一律 `==`
可变对象作类属性	两个实例互相看到对方注册的回调	`self.callbacks.append()` 找到的是类属性，修改的是所有实例共享的那一个 list	可变状态在 `__init__` 里挂到实例上；类属性只放常量
引用环持有外部资源	长时间训练后内存（或显存）缓慢上涨，`gc.collect()` 后回落	环要等循环 GC 才回收，环上的对象如果持有张量、文件句柄，资源释放就被推迟	回指用 `weakref`；排查用 `gc.get_referrers`、memray；及时从缓存和回调注册表里摘掉张量
以为 free-threaded 天然线程安全	换成 3.14t 后偶发数据错乱，或者完全没有加速	竞态被暴露；或某个 C 扩展让 GIL 静默重新启用	共享状态照旧加锁；上线前断言 `sys._is_gil_enabled()` 符合预期

2.9 延伸阅读

- Python Data Model：属性查找链、描述符、魔术方法的总纲。
- PEP 703（Making the GIL Optional）、PEP 779（free-threaded 的支持标准）。
- What's New in Python 3.14 / 3.15。
- py-free-threading.github.io：各库 free-threaded 支持状态与 C 扩展改造指南。

第 3 章 工程化：类型、并发与工具链

前两章解决了“能写”和“能看懂”，本章解决“能进生产”。vLLM、Ray、transformers 都是几十万行规模的工程，训练配置、API 载荷、分布式调度全是类型化、可测试的代码。本章讲四件事：类型系统、dataclass 与 pydantic v2、asyncio 与并发模型选择、工具链（uv、ruff、pytest、profiling）。本章的压测客户端在第 8 章会扩展成流式版本，用于压测推理服务。

3.1 本章目标

- 用 `Protocol`、PEP 695 泛型写出能通过 pyright 和 `mypy --strict` 的代码，知道类型检查器怎么选。
- 用 pydantic v2 建模嵌套的训练配置：字段校验、默认值、跨字段校验、拒绝拼错的字段。
- 说清协程与 goroutine 的调度差异，用 `TaskGroup` + `Semaphore` 写结构化并发。
- 根据 I/O 密集还是 CPU 密集、计算是否释放 GIL，在线程、进程、子解释器之间做选择，并用实测数据支撑判断。
- 用 uv 搭建可复现的项目环境，会用 pytest 和采样分析器。

3.2 Go 工程师视角

Go	Python	关键差异
interface（隐式满足）	`Protocol`（结构化子类型，PEP 544）	静态检查的鸭子类型；`@runtime_checkable` 后可以 `isinstance`
泛型 `[T any]`	`def f[T](x: T) -> T`（PEP 695，3.12+）	Python 泛型在运行时不做具体化
编译期类型系统	类型提示，运行时忽略	靠 pyright / mypy / Pyrefly / ty 检查
struct + json tag + validator	dataclass / pydantic v2 `BaseModel`	pydantic 把解析、校验、序列化合三为一
goroutine（M:N 抢占式，多核）	协程（单线程、协作式 event loop）	一个阻塞调用卡死整个 loop
channel	`asyncio.Queue` / `queue.Queue`	队列是库，不是语法
`go.mod` / `go.sum`	uv（`pyproject.toml` + `uv.lock`）	生态历史上碎片化，现在收敛到 uv
`gofmt` / `go vet`	Ruff（格式化 + lint）	一个二进制替代 black + isort + flake8
`go test` + table-driven	pytest + `parametrize` + fixture	fixture 是依赖注入模型，Go 没有直接对应
pprof	cProfile / py-spy / `profiling.sampling`（3.15）	采样分析器可以挂到运行中的进程上

最需要校准的是并发：Go 里“并发”只有一个答案：goroutine。Python 里它是一道选择题，选错了（比如用线程跑纯 Python 的 CPU 密集任务）结果是零加速。

3.3 静态类型：提示而非契约

类型注解对解释器透明：`def f(x: int) -> int` 传进字符串照样运行，只有类型检查器会拦你。注解是给人和工具看的文档，不是运行时契约；运行时要校验，用 pydantic（3.4 节）。

PEP 695（3.12）引入了内联类型参数，不再需要 `TypeVar`：

```
"""第 3 章：类型系统速览。用 pyright / mypy 检查：
    uv run pyright typing_demo.py      uv run mypy --strict typing_demo.py
"""
from typing import Protocol, runtime_checkable


@runtime_checkable
class Serializable(Protocol):                       # ≈ Go interface：只描述"有什么方法"
    def model_dump_json(self) -> str: ...


class Checkpoint:                                   # 没有声明"实现了 Serializable"，结构对上就算满足
    def __init__(self, step: int) -> None:
        self.step = step

    def model_dump_json(self) -> str:
        return f'{{"step": {self.step}}}'


def export(obj: Serializable) -> str:
    return obj.model_dump_json()


def first[T](xs: list[T]) -> T:                     # PEP 695（3.12+）：内联类型参数，不再需要 TypeVar
    return xs[0]


class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, item: T) -> None:
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()


type Vec = list[float]                              # PEP 695 类型别名


def norm(v: Vec) -> float:
    return sum(x * x for x in v) ** 0.5


if __name__ == "__main__":
    print(export(Checkpoint(100)))
    print(isinstance(Checkpoint(1), Serializable))     # runtime_checkable：只查方法名是否存在
    s: Stack[int] = Stack()
    s.push(1)
    print(first([3, 4]), s.pop(), norm([3.0, 4.0]))
    print(Stack[int].__type_params__ if hasattr(Stack[int], "__type_params__") else Stack.__type_params__)
```

Python 泛型在运行时不做具体化：`Stack[int]()` 和 `Stack[str]()` 创建的对象在运行时没有区别，不能像 Go 那样在运行时对 `T` 做分支。需要按类型分发时，只能把类型对象当参数传进去。

`Protocol` 与 Go interface 同构：一个类不需要声明“我实现了 Serializable”，方法签名对上就静态满足。需要运行时判断时加 `@runtime_checkable`，但它只检查方法名存在，不检查签名。

让检查器拦住错误

下面这段代码有两处类型错误，解释器照样运行到第二处才在运行时崩溃：

```
"""故意写错的代码：解释器照跑，类型检查器会拦下。"""
from typing_demo import Stack, export


def main() -> None:
    s: Stack[int] = Stack()
    s.push("not an int")                          # 类型错误 1：往 Stack[int] 里放 str
    print(export(42))                             # 类型错误 2：int 没有 model_dump_json


if __name__ == "__main__":
    try:
        main()
    except AttributeError as e:
        print("运行到第二处才在运行时炸:", e)
```

```
$ python typing_bad.py
运行到第二处才在运行时炸: 'int' object has no attribute 'model_dump_json'

$ pyright typing_demo.py typing_bad.py
typing_bad.py:7:12 - error: Argument of type "Literal['not an int']" cannot be assigned
    to parameter "item" of type "int" in function "push"
typing_bad.py:8:18 - error: Argument of type "Literal[42]" cannot be assigned to parameter
    "obj" of type "Serializable" ... "model_dump_json" is not present
2 errors, 0 warnings, 0 informations

$ mypy --strict typing_demo.py typing_bad.py
typing_demo.py:43: error: Returning Any from function declared to return "float"  [no-any-return]
typing_bad.py:7: error: Argument 1 to "push" of "Stack" has incompatible type "str"; expected "int"
typing_bad.py:8: error: Argument 1 to "export" has incompatible type "int"; expected "Serializable"
Found 3 errors in 2 files
```

第一处错误（往 `Stack[int]` 里放字符串）在运行时根本不会报错，只会在很久以后某个地方取出来用时才出问题。两个检查器都抓住了它。`mypy --strict` 还多报了一处：`sum(...) ** 0.5` 在类型存根里的返回类型是 `Any`，严格模式不允许把 `Any` 当 `float` 返回。改成 `math.sqrt(...)` 即可。不同检查器的严格程度不同，团队要统一一个作为 CI 标准。

类型检查器怎么选（2026-10）

检查器	出品方	实现语言	当前版本	定位
pyright	微软	TypeScript	1.1.414	编辑器场景的事实标准（VS Code 的 Pylance 底座）
mypy	社区	Python（mypyc 编译）	2.4.0	历史最久、插件生态最全；2.0 于 2026-05 发布
Pyrefly	Meta	Rust	1.4.0	2026-05 发布 1.0，主打大型代码库的速度
ty	Astral	Rust	0.0.90	仍处于 0.0.x 预发布阶段，速度快，规范覆盖度仍在追赶

建议：编辑器里用 pyright（或基于它的 Pylance），CI 里选一个作为标准；百万行级别的大仓关注 Pyrefly 和 ty。Ruff 负责格式化和 lint，不管类型；检查器不管风格。两者合起来相当于 Go 的 `gofmt` + `go vet` + staticcheck。

3.14 起注解延迟求值（PEP 649/749）：`def f(x: SomeClass)` 里的前向引用不再需要加引号。

3.4 dataclass 与 pydantic v2：配置即代码

纯数据容器用 dataclass，它自动生成 `__init__`、`__repr__`、`__eq__`：

```
from dataclasses import dataclass, field

@dataclass(slots=True, kw_only=True, frozen=True)
class DatasetSpec:
    path: str
    format: str = "parquet"
    tags: list[str] = field(default_factory=list)   # 可变默认值的标准解法
```

一旦涉及外部输入（JSON 配置、API 请求体、命令行参数），就用 pydantic v2（当前 2.14.x，内核是 Rust 写的 pydantic-core）。对照 Go：pydantic 约等于 `encoding/json` + go-playground/validator + mapstructure 三合一。OpenAI SDK、FastAPI、vLLM 的配置层都基于它。分工是：typing 管静态，pydantic 管运行时边界。

核心 API：`BaseModel` 定义模型；`model_validate()` 从 dict 解析；`model_dump()`/`model_dump_json()` 序列化；`field_validator`/`model_validator` 写校验。v2 相对 v1 的方法名全部加了 `model_` 前缀（`dict()` → `model_dump()`，`parse_obj()` → `model_validate()`）。

动手实践一：pydantic v2 训练配置（CPU 可跑）

```
"""第 3 章实践一：pydantic v2 训练配置。python config.py"""
from pydantic import BaseModel, ConfigDict, Field, ValidationError, field_validator, model_validator


class OptimizerConfig(BaseModel):
    name: str = "adamw"
    lr: float = Field(default=3e-4, gt=0, le=1.0)          # 第 1 版这里没有默认值，下面的默认实例会在导入时报错
    weight_decay: float = Field(default=0.01, ge=0)
    betas: tuple[float, float] = (0.9, 0.999)


class DataConfig(BaseModel):
    train_path: str
    batch_size: int = Field(default=32, gt=0)
    num_workers: int = Field(default=4, ge=0)


class TrainConfig(BaseModel):
    model_config = ConfigDict(extra="forbid")                # 拼错的字段名直接报错，而不是被静默忽略

    model_name: str
    max_steps: int = Field(gt=0)
    seed: int = 42
    optimizer: OptimizerConfig = Field(default_factory=OptimizerConfig)
    data: DataConfig
    tags: list[str] = []                                     # pydantic 会为每个实例拷贝默认值，这里是安全的

    @field_validator("model_name")
    @classmethod
    def name_not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("model_name 不能为空")
        return v

    @model_validator(mode="after")                           # 字段都校验通过之后，再做跨字段检查
    def check_workers(self) -> "TrainConfig":
        if self.data.batch_size >= 512 and self.data.num_workers == 0:
            raise ValueError("batch_size>=512 时 num_workers 必须大于 0")
        return self


if __name__ == "__main__":
```

```
    raw = {
        "model_name": "qwen3-8b-lora",
        "max_steps": "10000",                                  # 字符串会被转成 int（默认的宽松模式）
        "optimizer": {"lr": 1e-4},
        "data": {"train_path": "/data/train.parquet", "batch_size": 128},
    }
    cfg = TrainConfig.model_validate(raw)
    print(cfg.optimizer.lr, type(cfg.max_steps).__name__)
    print(cfg.model_dump_json(exclude={"tags"}))

    bad_inputs = {
        "字段级错误": {"model_name": "", "max_steps": -1, "data": {"train_path": "x"}},
        "跨字段错误": {"model_name": "m", "max_steps": 1,
                      "data": {"train_path": "x", "batch_size": 512, "num_workers": 0}},
        "拼错字段名": {"model_name": "m", "max_step": 1, "data": {"train_path": "x"}},
    }
    for title, bad in bad_inputs.items():
        try:
            TrainConfig.model_validate(bad)
        except ValidationError as e:
            print(f"\n[{title}] {e.error_count()} 个错误")
            for err in e.errors():
                print("  ", ".".join(map(str, err["loc"])) or "<model>", "->", err["msg"])
```

三处与第 1 版不同，前两处是 bug：

1. **第 1 版在导入时就会崩溃。** 它的 `lr: float = Field(gt=0, le=1.0)` 没有默认值，又写了 `optimizer: OptimizerConfig = OptimizerConfig()`，类定义执行到这一行就会因为缺少 `lr` 抛出 `ValidationError`。第 2 版给 `lr` 设了默认值，并改用 `default_factory`。
2. **第 1 版缺了 `@model_validator(mode="after")` 装饰器和方法定义**，跨字段校验的代码悬空。
3. 新增 `extra="forbid"`：配置文件里把 `max_steps` 拼成 `max_step` 时直接报错，而不是被静默忽略、悄悄用了默认值。这类问题在 Go 里有 `json.Decoder.DisallowUnknownFields()` 对应。

实测输出：

```
0.0001 int
{"model_name":"qwen3-8b-lora","max_steps":10000,"seed":42,"optimizer":{"name":"adamw",
 "lr":0.0001,"weight_decay":0.01,"betas":[0.9,0.999]},"data":{"train_path":
 "/data/train.parquet","batch_size":128,"num_workers":4}}

[字段级错误] 2 个错误
   model_name -> Value error, model_name 不能为空
   max_steps -> Input should be greater than 0

[跨字段错误] 1 个错误
   <model> -> Value error, batch_size>=512 时 num_workers 必须大于 0

[拼错字段名] 2 个错误
   max_steps -> Field required
   max_step -> Extra inputs are not permitted
```

注意三点：字符串 `"10000"` 被自动转成了 int（默认的宽松模式；`ConfigDict(strict=True)` 关闭自动转换）；字段级错误一次性聚合返回，每条带字段路径，而不是遇到第一个错就返回；跨字段校验只在字段级校验全部通过后才运行，所以“字段级错误”那组没有报 `num_workers` 的问题。

用 pytest 测它

```
"""第 3 章：pytest 的 fixture 与 parametrize。uv run pytest -q test_config.py"""
import json

import pytest
from pydantic import ValidationError
```

```
from config import TrainConfig


@pytest.fixture
def base_raw() -> dict:                              # fixture ≈ 依赖注入：测试函数按参数名拿到它
    return {"model_name": "m", "max_steps": 10, "data": {"train_path": "x"}}


@pytest.mark.parametrize(                            # parametrize ≈ Go 的 table-driven test
    ("patch", "ok"),
    [
        ({}, True),
        ({"max_steps": 0}, False),
        ({"model_name": "  "}, False),
        ({"data": {"train_path": "x", "batch_size": 512, "num_workers": 0}}, False),
        ({"data": {"train_path": "x", "batch_size": 512, "num_workers": 2}}, True),
    ],
)
def test_validation(base_raw: dict, patch: dict, ok: bool) -> None:
    raw = base_raw | patch
    if ok:
        TrainConfig.model_validate(raw)
    else:
        with pytest.raises(ValidationError):
            TrainConfig.model_validate(raw)


def test_roundtrip(base_raw: dict, tmp_path) -> None:   # tmp_path 是内置 fixture：每个测试一个临时目录
    cfg = TrainConfig.model_validate(base_raw)
    p = tmp_path / "cfg.json"
    p.write_text(cfg.model_dump_json(), encoding="utf-8")
    assert TrainConfig.model_validate(json.loads(p.read_text(encoding="utf-8"))) == cfg
```

```
$ uv run pytest -q test_config.py
......                                                                   [100%]
6 passed in 0.83s
```

pytest 的三个核心机制：`fixture` 是依赖注入加作用域（function/class/module/session），测试函数按参数名拿到它，对照 Go 的 `TestMain` 和子测试准备代码；`parametrize` 就是 table-driven test；`conftest.py` 提供目录级共享的 fixture。常用内置 fixture：`tmp_path`（临时目录）、`monkeypatch`（打补丁）、`caplog`（捕获日志）。常用插件：pytest-asyncio、pytest-cov、pytest-xdist。当前版本是 pytest 9.1（第 1 版写的“8.4.x 线”已经过时）。

3.5 asyncio：协作式调度的单线程世界

这是本章最重要的认知校准。goroutine 是 M:N 抢占式调度，runtime 在任何安全点都能抢占，自动分摊到多核。Python 协程是**单线程协作式**调度：一个 event loop 线程，协程只在 `await` 处主动让出，没有抢占，不用多核。三个推论：

1. 任何不含 `await` 的长计算会独占 loop，其余协程全部饿死。
2. 任何同步阻塞调用（`requests.get`、`time.sleep`、同步数据库驱动）会卡死整个 loop。Go 的 runtime 会把阻塞在系统调用上的 goroutine 挪开，event loop 没有这个机制。这是 Go 工程师写 asyncio 的头号事故来源。
3. asyncio 不能利用多核。它解决的是“单线程同时挂住几千个 I/O 等待”。

event loop 内部是一个回调队列加一个 selector（Linux 上是 epoll）。概念分三层：coroutine（调用 `async def` 函数得到的对象，不调度就不执行）→ Task（`asyncio.create_task` 创建，由 loop 调度运行）→

Future（结果占位符）。

结构化并发三件套（3.11+），优先于裸 `gather` 使用：

- `asyncio.TaskGroup`：对标 Go 的 errgroup，任一任务抛出未捕获异常时自动取消其余任务，异常收敛进 `ExceptionGroup`。
- `asyncio.timeout()`：`async with` 形式的超时。
- `asyncio.to_thread()`：把必须调用的阻塞函数丢到线程池，避免卡住 loop。

取消语义：`CancelledError` 在 `await` 处注入协程，捕获后必须重新抛出，否则会破坏结构化取消。HTTP 客户端：httpx 同时提供同步和异步客户端，API 接近 requests（注意 httpx 的正式版目前仍是 0.28.x，1.0 在预发布中）；aiohttp 是老牌纯异步库。

动手实践二：两个最常见的 asyncio 事故（CPU 可跑）

```
"""第 3 章常见坑一、二：在协程里调阻塞函数；忘记 await。python async_pitfalls.py"""
import asyncio
import time
import warnings


async def good(i: int) -> int:
    await asyncio.sleep(0.1)                        # 让出 event loop
    return i


async def bad(i: int) -> int:
    time.sleep(0.1)                                 # 阻塞整个 loop 线程，其它协程全部排队
    return i


async def offloaded(i: int) -> int:
    await asyncio.to_thread(time.sleep, 0.1)        # 必须用同步库时：丢到线程池
    return i


async def run(fn, n: int = 20) -> float:
    t0 = time.perf_counter()
    async with asyncio.TaskGroup() as tg:
        for i in range(n):
            tg.create_task(fn(i))
    return time.perf_counter() - t0


async def main() -> None:
    for fn in (good, bad, offloaded):
        print(f"20 个协程各等待 0.1s，{fn.__name__:9s}: {await run(fn):.2f}s")
    with warnings.catch_warnings(record=True) as w:
        warnings.simplefilter("always")
        coro = good(1)                              # 只创建了协程对象，什么都没执行
        print("忘记 await 拿到的是:", type(coro).__name__)
        del coro
        import gc
        gc.collect()
        print("回收时的警告:", w[0].message if w else "无")


asyncio.run(main())
```

```
20 个协程各等待 0.1s，good     : 0.10s
20 个协程各等待 0.1s，bad      : 2.14s
20 个协程各等待 0.1s，offloaded: 0.22s
忘记 await 拿到的是: coroutine
```

```
回收时的警告: coroutine 'good' was never awaited
```

`time.sleep` 让 20 个“并发”协程变成了串行，耗时从 0.1 秒变成 2.1 秒；用 `asyncio.to_thread` 卸载后恢复到 0.22 秒（线程池默认有上限，所以比纯异步略慢）。调用 `async def` 函数只创建协程对象，什么都不执行，这和 Go 里“调用函数就执行”的直觉相反。pyright 和 Ruff 都能查出没被 await 的协程。

动手实践三：asyncio + httpx 并发压测客户端（CPU 可跑）

```
async def worker(i: int, client: httpx.AsyncClient, sem: asyncio.Semaphore, url: str,
                 payload: dict | None, latencies: list[float], errors: list[str]) -> None:
    async with sem:                                          # 信号量限并发 ≈ Go 里用带缓冲 channel 当令牌桶
        t0 = time.perf_counter()
        try:
            if payload is None:
                resp = await client.get(url)
            else:
                resp = await client.post(url, json=payload)
            resp.raise_for_status()
            latencies.append(time.perf_counter() - t0)
        except httpx.HTTPError as e:                         # 网络/状态码错误记下来，不让一个请求拖垮整个压测
            errors.append(f"{type(e).__name__}: {str(e).splitlines()[0]}")
```

```
async def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", required=True)
    ap.add_argument("--concurrency", type=int, default=16)
    ap.add_argument("--requests", type=int, default=100)
    ap.add_argument("--payload-file", help="POST 的 JSON 载荷；不填则发 GET")
    ap.add_argument("--timeout", type=float, default=60.0)
    args = ap.parse_args()

    payload = None
    if args.payload_file:
        with open(args.payload_file, encoding="utf-8") as f:
            payload = json.load(f)
    sem = asyncio.Semaphore(args.concurrency)
    latencies: list[float] = []
    errors: list[str] = []
    limits = httpx.Limits(max_connections=args.concurrency)  # 连接池上限与并发数对齐，否则连接池先成瓶颈

    t0 = time.perf_counter()
    async with httpx.AsyncClient(limits=limits, timeout=args.timeout) as client:
        async with asyncio.TaskGroup() as tg:                # 结构化并发：任何未捕获异常都会取消其余任务
            for i in range(args.requests):
                tg.create_task(worker(i, client, sem, args.url, payload, latencies, errors))
    wall = time.perf_counter() - t0                          # 第 1 版漏了这一行，打印时 NameError

    latencies.sort()
    ok = len(latencies)
    print(f"成功 {ok}/{args.requests}  失败 {len(errors)}  总耗时 {wall:.2f}s  QPS {ok / wall:.1f}")
    if ok:
        print(f"延迟 ms: mean={statistics.mean(latencies) * 1e3:.2f} "
              f"p50={pct(latencies, 0.50) * 1e3:.2f} p95={pct(latencies, 0.95) * 1e3:.2f} "
              f"p99={pct(latencies, 0.99) * 1e3:.2f}")
    for e in errors[:2]:
        print("  示例错误:", e)
```

第 1 版的这段代码运行即崩溃：它打印了 `wall` 变量，但从未计算它（第 2 版标注了补上的那一行）。另外两处改进：用最近秩法计算分位数；失败请求单独计数并展示示例，而不是只打印到终端。

用 `python -m http.server` 做靶子，实测：

```
$ python bench.py --url http://127.0.0.1:18000/ --concurrency 1 --requests 2000
成功 2000/2000  失败 0  总耗时 1.93s  QPS 1036.9
延迟 ms: mean=0.88 p50=0.83 p95=1.00 p99=1.18
$ ... --concurrency 8
```

```
成功 2000/2000  失败 0  总耗时 1.23s  QPS 1631.5
延迟 ms: mean=4.30 p50=4.13 p95=4.80 p99=6.02
$ ... --concurrency 32
成功 2000/2000  失败 0  总耗时 1.13s  QPS 1768.7
延迟 ms: mean=15.75 p50=15.08 p95=16.85 p99=44.07
```

并发从 1 加到 32，QPS 只从 1037 涨到 1769，延迟却涨了 18 倍：`http.server` 是单线程服务，早就饱和了，多出来的并发只是在排队。这是一条典型的“吞吐饱和、延迟上升”曲线，第 8 章压测推理服务时会看到同样形状的曲线。

三个设计点：`httpx.Limits` 的连接上限要与信号量对齐，否则连接池先成瓶颈；`TaskGroup` 保证任何非预期异常都会取消整个压测，而不是静默泄漏任务；`--payload-file` 留给第 8 章塞 OpenAI 格式的请求体。

3.6 并发模型选择：实测说话

判断维度只有两个：任务是在等 I/O 还是在算？计算是否发生在会释放 GIL 的 C 扩展里？

```
def bench(executor_cls, fn, arg, jobs: int, workers: int) -> float:
    with executor_cls(max_workers=workers) as ex:
        list(ex.map(fn, [arg] * workers))          # 预热：进程/解释器的启动成本不计入
        t0 = time.perf_counter()
        list(ex.map(fn, [arg] * jobs))
        return time.perf_counter() - t0
```

同一个纯 Python CPU 任务（8 个任务、8 个 worker），在常规构建和 free-threaded 构建上的实测：

执行方式	3.14.7 常规构建	3.14.7 free-threaded
串行	0.82 s	0.81 s
`ThreadPoolExecutor`	0.85 s（1.0×）	0.19 s（4.3×）
`ProcessPoolExecutor`	0.12 s（7.0×）	0.10 s（7.8×）
`InterpreterPoolExecutor`（3.14+）	0.12 s（7.1×）	0.18 s（4.6×）
32 个 0.2 秒的 I/O 等待，32 线程	0.21 s（串行需 6.4 s）	0.21 s

结论：

场景	选择	依据
上千并发 HTTP/RPC	asyncio + httpx/aiohttp	单线程挂海量连接，开销最低
几十个只能同步调用的 SDK	`ThreadPoolExecutor`	等 I/O 时释放 GIL（上表最后一行）
纯 Python CPU 密集，常规构建	`ProcessPoolExecutor` 或 `InterpreterPoolExecutor`	线程池零加速（上表第二行）
NumPy/PyTorch 重计算	线程即可	C 扩展计算时释放 GIL（第 2 章实测 2.1×）
进程隔离 + 共享大块数据	`multiprocessing` + `shared_memory`	零拷贝共享内存段
新项目、CPU + I/O 混合	可评估 free-threaded + 线程	官方支持但非默认；C 扩展需声明支持

子解释器池（PEP 734）在常规构建上和进程池一样快，启动比进程轻，对象默认不共享，需要 pickle 传参，接口与另外两个池一致，切换成本很低。

`multiprocessing` 在 3.14 有一个重要行为变化：Linux 上默认启动方式从 `fork` 改为 `forkserver`（macOS 和 Windows 早就是 `spawn`）。`fork` 和线程混用很危险（子进程可能继承一把已被持有的锁，直接死锁）。新方式下子进程是干净的解释器，代价是任务函数和参数都必须能被 pickle。

3.7 工具链：uv + Ruff + pytest + profiling

uv（Astral，Rust 实现，当前 0.13.x）是新项目默认的包管理器，一个工具替代 pip + venv + pyenv + pipx + pip-tools：

```
uv init myproj && cd myproj
uv python pin 3.14            # 锁定解释器版本
uv add httpx pydantic         # 写入 pyproject.toml 和 uv.lock，自动建 venv
uv add --dev ruff pytest      # 开发依赖
uv run python main.py         # 不用手动激活 venv
uv sync --frozen              # 部署：严格按 lock 文件复现
uvx ruff check .              # 不安装、直接跑一次性工具
uv python install 3.14t       # 安装 free-threaded 解释器
```

对照 Go：`uv add` ≈ `go get`，`uv.lock` ≈ `go.sum`。

profiling 三档工具：

- cProfile：确定性追踪，逐调用计数，开销大，适合开发期。3.15 起位于 `profiling.tracing`。
- py-spy：Rust 写的采样分析器，从独立进程读目标进程内存，不用重启、不用改代码，`py-spy record -o flame.svg --pid <pid>` 一条命令出火焰图，体验与 `go tool pprof` 抓线上服务一致。
- `profiling.sampling`（3.15，PEP 799）：标准库自带的采样分析器，`python -m profiling.sampling attach --flamegraph -o profile.html <pid>` 挂到运行中的进程，还有 `--live` 交互模式（类似 top）。

内存分析用 memray，行级热点用 line_profiler。

3.8 常见坑

坑	现象	对策
协程里调阻塞函数	QPS 和串行一样，或高并发下服务假死	I/O 换异步库；必须用同步库时 `asyncio.to_thread()`
忘记 `await`	函数“没执行”，`RuntimeWarning: coroutine ... was never awaited`	入口用 `asyncio.run()`，并发用 `TaskGroup`；开启检查器
pydantic v1/v2 API 混用	照抄旧教程，`.dict()`、`parse_obj()`、`@validator` 报弃用或报错	锁定 `pydantic>=2`；`ConfigDict` 替代内部 `class Config`
交互环境里多进程序列化失败	`PicklingError`，或子进程卡在导入阶段	任务函数放到独立模块；lambda、闭包、嵌套函数都不能 pickle，改用顶层函数或 `functools.partial`
用线程加速纯 Python CPU 任务	8 线程和单线程一样快（3.6 节实测）	用进程池或子解释器池；或评估 free-threaded 构建
把 asyncio 当多核方案	压测客户端打满一个核，加并发无济于事	多进程各跑一个 loop；服务端用 Uvicorn/Granian 的多 worker 模式

3.9 延伸阅读

- typing 官方文档（Protocol、泛型、PEP 695 语法）；pydantic v2 文档与 v1→v2 迁移指南。
- asyncio 文档的 Task、TaskGroup、timeout 部分。
- uv、pytest、py-spy 官方文档；Python 3.15 `profiling` 模块文档。

第 4 章 数值计算与 GPU：NumPy、CUDA 模型与 Triton

本章进入 AI Infra 的物理层：数据在内存里怎么摆、计算为什么慢、GPU 为什么快，以及“用 Python 写 GPU kernel”是怎么回事。三个主角：NumPy 定义了整个科学计算生态的数组内存模型（PyTorch 张量的语义大量继承自它）；CUDA 编程模型是读懂一切 GPU 性能分析的基础；Triton 是当前写 kernel 的主流入口，`torch.compile` 默认生成的就是 Triton kernel。

4.1 本章目标

- 用“数据指针 + 形状 + 步长 + 元素类型”解释 ndarray 的行为，准确判断一个操作返回视图还是拷贝。
- 说清广播规则，并通过实测理解纯 Python 循环与向量化之间的差距。
- 知道 NumPy 2.x 的两处破坏性变更（NEP 50 标量类型提升、`copy=False` 语义反转），能安全迁移老代码。
- 画出 CUDA 的执行层级与显存层级，说清 kernel 启动开销与 CUDA Graph。
- 读懂并能手写一个 Triton vector add kernel。

4.2 Go 工程师视角

Go	Python 数值栈	关键差异
slice（指针 + len + cap）	ndarray（数据指针 + shape + strides + dtype）	思想同构；strides 让“跨步视图”成为一等公民
`s[1:3]` 共享底层数组	NumPy 基本切片返回视图（共享）；**list 切片是拷贝**	list 和数组行为相反，别混
零拷贝优化	视图、stride 技巧、`np.asarray` 不拷贝	思路一致，工具不同
cgo 调 C	Python 调 NumPy ufunc / ATen / cuBLAS	Python 只是调度层，重计算在编译好的 C/CUDA 里
goroutine（M:N 调度）	CUDA 线程（SIMT，grid/block/warp 硬件调度）	GPU 没有抢占式调度器，一次发射上万个线程
pprof 看 CPU 热点	Nsight Systems / Nsight Compute 看 kernel 时间线	GPU 是异步设备，分析单位是 kernel

ndarray 就是一个加强版的 slice header。Go 的 slice 只记录“从哪开始、多长”，步长隐含为元素大小；ndarray 把步长显式化、多维化，于是转置、隔行取样、广播都变成“改 header、不搬数据”的零拷贝操作。

4.3 ndarray：一个加强版 slice header

元素 `(i, j)` 的地址是 `data + i*strides[0] + j*strides[1]`，strides 的单位是字节。默认是 C 连续（行优先，与 Go 的多维数组展平方式一致）。转置 `a.T` 只交换 shape 和 strides，与原数组共享内存。由此推出最重要的规则：

- **基本切片返回视图**：`a[1:3]`、`a[:, ::2]`、`a.T`、`reshape`（能不改内存布局时）都不拷贝，修改视图会改到原数组。用 `np.shares_memory(a, b)` 验证。
- **花式索引总是拷贝**：`a[[0, 2, 5]]`（整数数组索引）和 `a[a > 0]`（布尔掩码）返回新数组。口诀：用“坐标列表”取是拷贝，用“区间”取是视图。
- **广播**：形状不同的数组运算时，维度从右对齐，长度为 1 的维被“虚拟拉伸”，实现上就是把那一维的步长设为 0，同一份数据反复读，不复制。`(N,1) + (1,M) → (N,M)`。

向量化为什么快：NumPy 的 ufunc 在 C 层做紧凑循环并利用 SIMD 指令，循环体里没有 Python 对象、没有解释器开销，数据在缓存里连续流过。纯 Python 循环的每个元素都要经历“装箱成对象、类型分发、解释执行”。

NumPy 2.x 的破坏性变更（当前 2.5.4，2026-10-10 发布；2.5 系列要求 Python ⩾ 3.12）：

1. NEP 50：Python 标量参与运算时不再按“值”决定结果类型。Python 的 int/float 是“弱类型”，跟随数组的 dtype：`float32` 数组乘 `2.0` 结果仍是 `float32`；`int8` 数组加 `300`（放不进 int8）直接抛 `OverflowError`，1.x 时代会悄悄提升成更宽的类型。
2. `copy=False` **语义反转**：1.x 里表示“能不拷就不拷”；2.x 里表示“绝不允许拷贝，做不到就报错”。要表达“按需拷贝”，用 `np.asarray()`。

4.4 动手实践一：向量化、视图与访存局部性（CPU 可跑）

```
"""第 4 章实践一：向量化 vs 循环、view vs copy、访存局部性、NumPy 2 的两处破坏性变更。

uv run python bench_numpy.py
"""
import timeit

import numpy as np

print("numpy", np.__version__)
rng = np.random.default_rng(0)

# ---------- 1. 行归一化：纯 Python 循环 vs 广播 ----------
a = rng.random((1000, 1000))
rows = a.tolist()                                   # 预先转成 Python list，循环版只计"纯 Python 计算"


def python_loop():
    out = []
    for row in rows:
        s = sum(row)
        out.append([x / s for x in row])
    return out


def numpy_vec():
    return a / a.sum(axis=1, keepdims=True)         # keepdims 让结果形状为 (N,1)，再广播回 (N,M)


assert np.allclose(np.array(python_loop()), numpy_vec())
t_loop = min(timeit.repeat(python_loop, number=1, repeat=3))
t_vec = min(timeit.repeat(numpy_vec, number=20, repeat=3)) / 20
print(f"纯 Python 循环: {t_loop * 1e3:8.2f} ms")
```

```
print(f"NumPy 广播   : {t_vec * 1e3:8.2f} ms   加速比 ~{t_loop / t_vec:.0f}x")

# 不写 keepdims 会怎样：(1000,1000) / (1000,) 按"列"广播，形状不报错，结果却是错的
wrong = a / a.sum(axis=1)
print("忘写 keepdims：形状", wrong.shape, "| 每行之和是否为 1:", np.allclose(wrong.sum(axis=1), 1))

# ---------- 2. view vs copy ----------
b = np.arange(10)
v = b[2:6]                                      # 基本切片 → view
c = b[[2, 3, 4, 5]]                             # 花式索引 → copy
v[0] = 999
c[1] = -1
print("\nview 改到原数组:", b[2] == 999, "| copy 不影响原数组:", b[3] == 3)
print("shares_memory(view):", np.shares_memory(b, v), "| shares_memory(fancy):", np.shares_memory(b, c))
m = np.arange(12).reshape(3, 4)
print("转置只改 strides：", m.strides, "->", m.T.strides, "| 共享内存:", np.shares_memory(m, m.T))

# ---------- 3. 访存局部性：同样多的元素，按行读 vs 按列读 ----------
big = rng.random((4096, 4096))


def sum_rows():                                 # 逐行求和：每行在内存里连续
    return sum(float(big[i, :].sum()) for i in range(0, 4096, 4))


def sum_cols():                                 # 逐列求和：相邻元素相隔 4096×8 字节
    return sum(float(big[:, j].sum()) for j in range(0, 4096, 4))


t_r = min(timeit.repeat(sum_rows, number=3, repeat=3)) / 3
t_c = min(timeit.repeat(sum_cols, number=3, repeat=3)) / 3
print(f"\n各读 1024 条、每条 4096 个元素：按行 {t_r * 1e3:.1f} ms，按列 {t_c * 1e3:.1f} ms（{t_c / t_r:.1f} 倍）")

# ---------- 4. NumPy 2 的两处破坏性变更 ----------
x8 = np.array([100], dtype=np.int8)
r = x8 + np.int8(100)                           # NEP 50：结果保持 int8，溢出回绕
print("\nNEP 50: int8(100)+int8(100) =", r, r.dtype)
try:
    np.array([1], dtype=np.int8) + 300          # Python int 是"弱类型"，300 放不进 int8 → 直接报错
except OverflowError as e:
    print("NEP 50: int8 数组 + 300 ->", type(e).__name__, e)
print("NEP 50: float32 数组 * 2.0 的 dtype =", (np.ones(2, np.float32) * 2.0).dtype)
try:
    np.array(a.tolist()[:2], copy=False)        # 2.x：copy=False 表示"绝不允许拷贝"
except ValueError as e:
    print("copy=False ->", type(e).__name__, str(e).split(".")[0])
print("np.asarray 按需拷贝:", np.asarray(b) is b)
```

实测输出：

```
numpy 2.5.4
纯 Python 循环:    21.56 ms
NumPy 广播   :     0.37 ms   加速比 ~58x
忘写 keepdims：形状 (1000, 1000) | 每行之和是否为 1: False

view 改到原数组: True | copy 不影响原数组: True
shares_memory(view): True | shares_memory(fancy): False
转置只改 strides：  (32, 8) -> (8, 32) | 共享内存: True

各读 1024 条、每条 4096 个元素：按行 1.2 ms，按列 13.2 ms（11.4 倍）

NEP 50: int8(100)+int8(100) = [-56] int8
NEP 50: int8 数组 + 300 -> OverflowError Python integer 300 out of bounds for int8
NEP 50: float32 数组 * 2.0 的 dtype = float32
copy=False -> ValueError Unable to avoid copy while creating an array as requested
np.asarray 按需拷贝: True
```

四个观察：

1. **向量化快 58 倍。** 循环版的计时只包含纯 Python 计算（`tolist()` 在计时之外），比第 1 版把转换也算进去更公平。

2. **忘写 `keepdims` 不会报错，只会算错。** `a.sum(axis=1)` 的形状是 `(1000,)`，广播时被当作 `(1, 1000)` 按列去除，形状完全合法，结果却是错的。这比报错更危险：方阵上一定不会报错。
3. **访存局部性的代价是一个数量级。** 同样读 1024 × 4096 个元素，按列读（相邻元素在内存里相隔 32 KiB）比按行读慢 11.4 倍。第 1 版的实验拿“只读 1/64 元素的跨步视图”去和“读全部元素”比，两边工作量不同；在本机复现它，结果是全量连续求和 1.88 ms、跨步求和 0.20 ms，与第 1 版预期的“跨步更慢”正好相反。
4. **NEP 50 的效果：** `int8` 的 100 + 100 回绕成 -56，类型保持 int8。

4.5 CUDA 最小知识：不写 CUDA 也必须懂

执行层级。 GPU 由多个 SM（流式多处理器）组成。一个 kernel 启动时按 grid → block → thread 三级组织，每个 block 驻留在一个 SM 上；block 内每 32 个线程组成一个 warp，同一 warp 的线程同步执行同一条指令（SIMT）。推论：warp 内的 if-else 分支发散会让两个分支串行执行。对照 Go：goroutine 由 runtime 抢占式地分发到少量 P 上；CUDA 线程没有调度器，是“一次发射成千上万个，硬件按 warp 批量推进”。

显存层级。 从快到慢：寄存器 → shared memory（block 内共享、程序员手动管理）→ L2 → HBM 全局显存（延迟数百个时钟周期）。GPU 优化大多是三件事：提高算术强度（每读一个字节做更多运算）、合并访存（一个 warp 的线程读连续地址）、用 shared memory 复用数据。逐元素运算（加法、激活函数）几乎必然受限于 HBM 带宽，称为 memory-bound，这是理解“算子融合”价值的前提。

kernel 启动是异步的。 CPU 把 kernel 放进 stream 就返回，GPU 在后台执行；每次启动有几微秒的 CPU 侧开销。PyTorch 里 `x * 2 + 1` 是两次启动；小算子很多的网络里，启动开销可能超过计算本身。`tensor.item()`、`print(tensor)`、`.cpu()` 都会让 CPU 等 GPU 跑完，是性能分析的头号干扰源。

CUDA Graph： 把一串 kernel 启动录制成静态图，之后一次 `replay()` 整体重放，几乎消除 CPU 侧启动开销。约束：形状固定、内存地址固定、图内没有依赖数据的控制流、不能同步。`torch.compile(mode="reduce-overhead")` 内部就用它；vLLM 对 decode 阶段做 CUDA Graph 捕获（第 8 章）。

Python 这么慢，为什么训练还快？ PyTorch 是三层结构：Python 前端 + C++ 核心（ATen）+ CUDA kernel。Python 层只做分发，重计算全在编译好的 C++/CUDA 里，这些代码执行时释放 GIL，GPU kernel 更是异步发射，CPU 只负责排队。这与 Go 里通过 cgo 调 C 本质相同：宿主语言只是调度层。

4.6 Triton：用 Python 写 GPU kernel

Triton（当前 3.8.0，2026-08-28 发布；Linux 上安装 PyTorch 时会一并安装；注意它要求 Python < 3.15）是一种 Python 语法的 GPU kernel 语言和编译器。`torch.compile` 的 Inductor 后端在 GPU 上默认生成 Triton kernel；vLLM、Liger-Kernel 等组件也大量使用它。同一份源码可以运行在 NVIDIA 和 AMD GPU 上。

编程模型是 block 级的 SPMD：你不写单个线程，而是写“一个 program 实例处理一块数据”。没有 `threadIdx`，用 `tl.program_id(0)` 知道自己是第几块，用 mask 处理边界：

CUDA C	Triton	说明
`blockIdx.x`	`tl.program_id(axis=0)`	当前 program 是第几块
`if (i < n)` 边界判断	`mask = offsets < n`	在 load/store 时生效，越界地址根本不访问

CUDA C	Triton	说明
`kernel<<<grid, block>>>`	`kernel[grid](...)`	grid 通常是按 `BLOCK_SIZE` 计算块数的函数
线程内逐元素处理	`tl.arange(0, BLOCK_SIZE)` 一次处理一整块	编译器把块内向量操作映射到 warp

融合的价值在于访存而不是计算。 `y = x * 2 + 1` 在 PyTorch eager 模式下是两个 kernel，中间结果 `x * 2` 要写回 HBM 再读出来；融合成一个 kernel 后只读写一次。逐元素运算是 memory-bound 的，少一次 HBM 往返就接近一倍加速。这和 Go 里"少一次内存拷贝、少一次系统调用"的优化直觉完全同构。

生产案例：据 PyTorch 官方博客（2025-10）介绍，vLLM V1 内置的纯 Triton attention 后端用约 800 行 Triton 代码，在 H100 上达到了与 FlashAttention-3 相当的端到端性能，并成为 AMD MI300 部署的默认后端。到今天，"能读懂、能改写 Triton kernel"比"能手写 CUDA"更常用。

4.7 动手实践二：Triton vector add（需要 GPU）

```
"""第 4 章实践二：Triton vector add。需要 NVIDIA/AMD GPU（Linux 上 pip install torch 会带上 triton）。

python vector_add.py
"""
import torch
import triton
import triton.language as tl


@triton.jit                                          # 首次调用时 JIT 编译成 GPU kernel
def add_kernel(x_ptr, y_ptr, out_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)                      # ≈ CUDA 的 blockIdx.x：我是第几个块
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)   # 本块负责的一整段下标
    mask = offsets < n_elements                      # 尾块越界保护
    x = tl.load(x_ptr + offsets, mask=mask)          # 被 mask 掉的地址根本不访问
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(out_ptr + offsets, x + y, mask=mask)


def add(x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    out = torch.empty_like(x)
    grid = lambda meta: (triton.cdiv(x.numel(), meta["BLOCK_SIZE"]),)   # 需要多少个 program
    add_kernel[grid](x, y, out, x.numel(), BLOCK_SIZE=1024)
    return out


def bench(fn, iters: int = 100) -> float:
    for _ in range(10):                              # 预热（含 JIT 编译）
        fn()
    torch.cuda.synchronize()
    start, end = torch.cuda.Event(enable_timing=True), torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(iters):
        fn()
    end.record()
    torch.cuda.synchronize()                         # kernel 是异步的，必须同步后再读计时
    return start.elapsed_time(end) / iters * 1e3     # elapsed_time 单位是 ms，换算成 μs


if __name__ == "__main__":
    assert torch.cuda.is_available(), "需要 GPU"
    torch.manual_seed(0)
    n = 98_432                                       # 故意不是 1024 的整数倍，覆盖 mask 路径
    x, y = torch.rand(n, device="cuda"), torch.rand(n, device="cuda")
    torch.testing.assert_close(add(x, y), x + y)
```

```
    print("正确性通过")
    big = 1 << 26                                  # 6400 万个 float32，256 MiB：足够大才看得到带宽
    xb, yb = torch.rand(big, device="cuda"), torch.rand(big, device="cuda")
    for name, fn in [("triton", lambda: add(xb, yb)), ("torch", lambda: xb + yb)]:
        us = bench(fn)
        gbps = 3 * big * 4 / (us * 1e-6) / 1e9     # 读 x、读 y、写 out 共三次
        print(f"{name:6s}: {us:8.1f} μs/次   有效带宽 {gbps:6.0f} GB/s")
```

本机没有 NVIDIA GPU，这段代码没有在本机运行（已通过语法检查）。与第 1 版相比改了两处：

- **测试规模。** 第 1 版用约 10 万个元素测带宽，数据只有 400 KB，能放进 L2 缓存，kernel 也短到以启动开销为主，测不出真实带宽。第 2 版用 6400 万个元素（每个数组 256 MiB），并直接打印有效带宽。在 H100 这类卡上，两者都应接近显存带宽上限，数值相近。
- **计时单位。** `torch.cuda.Event.elapsed_time` 返回毫秒，第 2 版明确换算为微秒。

三个必须理解的点：`BLOCK_SIZE: tl.constexpr` 是编译期常量，参与 kernel 特化；mask 不是事后过滤，而是 load/store 指令级的访存抑制；计时必须用 CUDA Event 并在循环后同步，直接对单次调用用 `time.perf_counter()` 量到的只是启动开销。

4.8 常见坑

坑	现象	对策
以为切片产生新数组	改了“切片”，原始 batch 数据被污染	需要独立副本时 `.copy()`；排查用 `np.shares_memory`
循环里逐元素改数组	预处理脚本跑几个小时，热点是一个 Python for	先想能否用广播/ufunc 表达；`np.vectorize` 只是语法糖，不加速；真要自定义逐元素逻辑用 Numba，或写成 Triton/C++
NEP 50 引起的类型变化	升级 NumPy 2.x 后出现溢出、精度变化或 `OverflowError`	关键计算前显式 `astype`；按官方 2.0 迁移指南逐项检查
`copy=False` 报错	`ValueError: Unable to avoid copy`	按需拷贝用 `np.asarray(x)`
用墙钟时间测 GPU kernel	测出来快得离谱	用 CUDA Event，前后同步；警惕 `.item()`、`print(tensor)` 这类隐式同步
广播方向记反	形状对了，结果错了（4.4 节第 2 点）	逐行操作永远写 `keepdims=True` 或 `[:, None]`

4.9 延伸阅读

- NumPy 文档：strides 与视图；NumPy 2.0 迁移指南（NEP 50、`copy` 语义）。
- CUDA C++ Programming Guide；CUDA C++ Best Practices Guide（异步执行与 CUDA Graph）。
- Triton 官方教程第一课（vector add）。
- PyTorch 博客：Enabling vLLM V1 on AMD GPUs with Triton（2025-10）。

第 5 章 PyTorch 深入：从张量到训练循环

本章把第 4 章的知识收拢到 PyTorch 上：张量怎么存内存，梯度怎么“自动”算出来，`nn.Module` 背后用了哪些 Python 机制，`torch.compile` 如何把 eager 代码编译成融合 kernel。读完本章，你应该能不依赖任何高层封装，写出并读懂一个完整的训练循环。

5.1 本章目标

- 说清张量的内存模型（storage + stride），以及与 NumPy 互转时的零拷贝陷阱。
- 说清 autograd 的规则：梯度累加、计算图释放、`no_grad` 与 `inference_mode` 的区别、`clone` 与 `detach` 的区别。
- 理解 `nn.Module` 如何用 `__setattr__` 注册参数，`state_dict` 的键怎么组织。
- 写出设备无关的混合精度训练循环，知道 bf16 为什么通常不需要 `GradScaler`。
- 理解 `torch.compile` 的两段式管线，知道什么时候它有用、什么时候没用，并用实测数据说话。

5.2 Go 工程师视角

Go	PyTorch	要点
slice header（指针 + len + cap）	Tensor（storage + shape + stride + dtype）	stride 的单位是元素个数；转置、切片只改 stride
cgo 调 C 库	Python 分发到 ATen（C++）+ CUDA kernel	Python 只是调度层
异步 I/O + 同步点	CUDA kernel 异步执行；`.item()`、`.cpu()` 强制同步	`.item()` 相当于每次都 `wg.Wait()`
GC 管理对象生命周期	计算图是活的引用图，`backward()` 后默认释放	要二次反向需 `retain_graph=True`
`unsafe.Pointer` 强转出来的共享 slice	`torch.from_numpy()` 共享底层内存	改一边影响另一边
errgroup 拉起 worker	`DataLoader(num_workers=N)` 多进程数据管道	用多进程绕过 GIL，跨进程传 batch

5.3 张量内存模型

张量与 NumPy 数组是同构设计，只是 stride 的单位换成了元素个数，底层缓冲区叫 storage。多个张量可以共享一个 storage：切片、转置、`view()` 都返回新的张量对象，但指向原来的 storage。

`view()` 与 `reshape()` 的分工：`view()` 保证零拷贝，做不到就报错；`reshape()` 尽量零拷贝，做不到就拷贝。`clone()` 拷贝数据但保留计算图连接（梯度能流回源张量）；`detach()` 共享数据但切断计算图。

```
"""第 5 章：张量内存模型与 autograd 的几条规则。python tensor_basics.py"""
import numpy as np
import torch

a = torch.arange(12).reshape(3, 4)
b = a.T
print("stride:", a.stride(), "->", b.stride(), "| b 连续吗:", b.is_contiguous())
print("共享 storage:", a.untyped_storage().data_ptr() == b.untyped_storage().data_ptr())

try:
```

```
    b.view(12)                                  # view 保证零拷贝，做不到就报错
except RuntimeError as e:
    print("非连续张量 view ->", str(e).split(".")[0])
r = b.reshape(12)                               # reshape 尽量零拷贝，做不到就拷贝
print("reshape 后还共享内存吗:", r.untyped_storage().data_ptr() == a.untyped_storage().data_ptr())

arr = np.zeros(3, dtype=np.float32)
t = torch.from_numpy(arr)                       # 零拷贝：两边共享同一块内存
arr[0] = 7
print("改 NumPy 数组后 tensor:", t.tolist())
t2 = torch.tensor(arr)                          # torch.tensor 总是拷贝
arr[1] = 8
print("torch.tensor 拷贝出来的不受影响:", t2.tolist())

# autograd 规则一：梯度会累加
w = torch.tensor(2.0, requires_grad=True)
for _ in range(2):
    (w * 3).backward()
print("\n两次 backward 不清零，w.grad =", w.grad.item())

# 规则二：图在 backward 后释放
y = (w * w).exp()
y.backward()
try:
    y.backward()
except RuntimeError as e:
    print("第二次 backward ->", str(e).split(".")[0])

# 规则三：no_grad 与 inference_mode
with torch.no_grad():
    z1 = w * 2
with torch.inference_mode():
    z2 = w * 2
print("no_grad 结果 requires_grad:", z1.requires_grad, "| inference tensor:", z2.is_inference())
try:
    (z2 * w).backward()                         # inference tensor 不能再进入需要梯度的计算
except RuntimeError as e:
    print("inference tensor 参与求导 ->", str(e).split(".")[0])

# 规则四：clone 保留图，detach 切断图
x = torch.ones(2, requires_grad=True)
print("clone().grad_fn:", type(x.clone().grad_fn).__name__, "| detach().requires_grad:", x.detach().requires_grad)
```

实测输出：

```
stride: (4, 1) -> (1, 4) | b 连续吗: False
共享 storage: True
非连续张量 view -> view size is not compatible with input tensor's size and stride
    (at least one dimension spans across two contiguous subspaces)
reshape 后还共享内存吗: False
改 NumPy 数组后 tensor: [7.0, 0.0, 0.0]
torch.tensor 拷贝出来的不受影响: [7.0, 0.0, 0.0]

两次 backward 不清零，w.grad = 6.0
第二次 backward -> Trying to backward through the graph a second time
    (or directly access saved tensors after they have already been freed)
no_grad 结果 requires_grad: False | inference tensor: True
inference tensor 参与求导 -> Inference tensors cannot be saved for backward
clone().grad_fn: CloneBackward0 | detach().requires_grad: False
```

逐条对应四个规则：

1. **梯度是累加的。** 两次 `backward` 不清零，`w.grad` 从 3 变成 6。所以训练循环每步都要 `zero_grad`；梯度累积（多个小 batch 攒一次更新）正是靠这个设计。推荐 `zero_grad(set_to_none=True)`（当前版本的默认值）：把 `.grad` 设为 `None` 而不是填零，省一次显存写入。
2. **图在 `backward` 后释放。** 要对同一张图二次反向，第一次就要传 `retain_graph=True`。
3. **`no_grad` 与 `inference_mode` 不同。** 前者只是不建图；后者更激进，连原地修改的版本计数都不记录，更快，但产出的张量之后不能再参与任何需要梯度的计算。验证和推理代码优先用 `inference_mode`。

4. `torch.from_numpy` 零拷贝，`torch.tensor` 总是拷贝。 前者只支持 CPU 张量，且 dtype 必须是 NumPy 支持的（NumPy 没有 bf16）。

注意：判断两个张量是否共享内存用 `untyped_storage().data_ptr()`；第 1 版用的 `t.storage()` 返回的 `TypedStorage` 已被弃用。

设备迁移：`t.to("cuda")` 是跨设备拷贝，返回新张量；同设备调用是空操作。CPU 与 GPU 之间完全异步，CPU 把 kernel 放进 stream 就返回。主机到设备的拷贝可以用锁页内存（`pin_memory=True`）配合 `non_blocking=True` 与计算重叠，`DataLoader(pin_memory=True)` 就是做这个的。

5.4 autograd：动态计算图

PyTorch 的 autograd 是“边运行边建图”（define-by-run）：每次前向运算即时构建一张有向无环图。每个张量记录产生它的算子（`grad_fn`），`loss.backward()` 从标量出发逆拓扑序遍历，用链式法则把梯度累加进叶子节点的 `.grad`。

为什么用动态图？因为计算图就是你写的 Python 控制流本身：`if`、`for`、提前 `return` 都合法，调试时 `print` 任何中间值都是真实数值，异常堆栈指向你写的那一行。代价是图信息随前向结束而消散，跨算子的优化无从谈起，`torch.compile` 正是为了补上这一块。

动手实践一：40 行手写 autograd（CPU 可跑）

```
"""第 5 章实践一：40 行手写 autograd，并用 PyTorch 核对结果。python micrograd.py"""
import math


class Value:
    def __init__(self, data: float, _children: tuple["Value", ...] = ()) -> None:
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None            # 每个节点挂一个"局部链式法则"闭包
        self._prev = set(_children)

    def __add__(self, other: "Value") -> "Value":
        out = Value(self.data + other.data, (self, other))
        def _backward() -> None:                 # d(out)/d(self) = 1
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other: "Value") -> "Value":
        out = Value(self.data * other.data, (self, other))
        def _backward() -> None:                 # d(out)/d(self) = other
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def tanh(self) -> "Value":
        t = math.tanh(self.data)
        out = Value(t, (self,))
        def _backward() -> None:                 # d tanh(x)/dx = 1 - tanh²
            self.grad += (1 - t * t) * out.grad
        out._backward = _backward
        return out

    def backward(self) -> None:
        topo: list[Value] = []
        visited: set[Value] = set()
        def build(v: Value) -> None:             # DFS 后序：子节点先进列表
            if v not in visited:
                visited.add(v)
                for child in v._prev:
```

```
                    build(child)
                topo.append(v)
        build(self)
        self.grad = 1.0
        for v in reversed(topo):                    # 逆拓扑序：每个节点在用到自己的梯度前已经收齐
            v._backward()


x, w, b = Value(2.0), Value(-0.5), Value(0.3)
y = (x * w + b).tanh() * x                          # x 被用了两次：梯度要从两条路径累加
y.backward()
print(f"y = {y.data:.6f}  dy/dx = {x.grad:.6f}  dy/dw = {w.grad:.6f}  dy/db = {b.grad:.6f}")

import torch                                        # noqa: E402 —— 用 PyTorch 核对

xt, wt, bt = (torch.tensor(v, dtype=torch.float64, requires_grad=True) for v in (2.0, -0.5, 0.3))
yt = torch.tanh(xt * wt + bt) * xt
yt.backward()
print(f"PyTorch:     dy/dx = {xt.grad.item():.6f}  dy/dw = {wt.grad.item():.6f}  dy/db = {bt.grad.item():.6f}")
```

```
y = -1.208736  dy/dx = -1.239107  dy/dw = 2.538958  dy/db = 1.269479
PyTorch:     dy/dx = -1.239107  dy/dw = 2.538958  dy/db = 1.269479
```

手写版与 PyTorch 的结果在 6 位小数上完全一致。比第 1 版多做了两件事：加了 `tanh`，让计算图有非线性；让 `x` 在表达式里出现两次（`tanh(x·w + b) · x`），这样 `x.grad` 必须从两条路径累加才对。`_backward` 闭包里的 `+=` 就是 PyTorch“梯度累加”的源头；`build()` 的后序遍历保证每个节点在使用自己的梯度之前，已经收齐了所有下游传回的梯度。

5.5 `nn.Module`：`__setattr__` 与参数注册

第 2 章讲过属性赋值可以被拦截。`nn.Module` 是这个机制最漂亮的工程应用：你写 `self.fc = nn.Linear(784, 128)` 时，`Module.__setattr__` 拦截赋值，发现值是 `nn.Parameter` 或子 `Module`，就登记进 `self._parameters` / `self._modules`。于是：

- `model.parameters()` 递归遍历 `_modules` 树收集全部参数，直接交给优化器；
- `model.to("cuda")` 就地迁移所有已注册的参数和 buffer。把普通张量直接赋给 `self.x`（不包成 `Parameter`、也不用 `register_buffer()`），`.to()` 就会漏掉它；
- `model.state_dict()` 把参数和 buffer 摊平成 `{"fc.weight": tensor, ...}`，checkpoint 存的就是它，`load_state_dict` 按键名对齐。

理解了 `_parameters` 字典，第 6 章 FSDP 的参数分片和第 7 章 LoRA 的参数注入都不再神秘：它们操作的就是这棵树。

调用模型写 `model(x)`，不要写 `model.forward(x)`。 `Module.__call__` 在 `forward` 前后包了一层 hook 调度，`torch.compile`、FSDP、量化工具、profiler 都挂在这层钩子上。直接调 `forward()` 会绕过它们：模型能跑，但分片、编译捕获全部失效。

5.6 DataLoader、混合精度与 torch.compile

DataLoader。 `Dataset` 定义 `__getitem__`/`__len__`，`DataLoader` 负责组 batch、打乱和预取。`num_workers=N` 用 N 个进程并行加载与预处理，绕过 GIL；`pin_memory=True` 让 batch 放在锁页内存里；`persistent_workers=True` 让 worker 跨 epoch 存活，省去每轮重建进程的开销。

混合精度。 旧 API `torch.cuda.amp.*` 已弃用，用设备无关的 `torch.amp`：

```
scaler = torch.amp.GradScaler("cuda", enabled=(dtype == torch.float16))   # 只有 fp16 需要
with torch.autocast(device_type="cuda", dtype=dtype):                      # 只包前向和 loss
    loss = loss_fn(model(x), y)
scaler.scale(loss).backward()      # scaler 未启用时等价于 loss.backward()
scaler.step(optimizer)
scaler.update()
```

fp16 的动态范围小，梯度容易下溢，需要 `GradScaler` 做 loss scaling；bf16 的指数位与 fp32 相同，动态范围足够，通常不需要 scaler。第 1 版的示例同时用了 bf16 和 `GradScaler`，没有错，但多余。另外，混合精度省的是计算和激活显存：优化器里仍然持有 fp32 主权重和优化器状态（第 6 章的 16 字节/参数公式）。

`torch.compile`：TorchDynamo 在运行时捕获 Python 字节码，把能静态化的部分切成计算图，遇到依赖数据的 Python 控制流就“断图”、回退 eager；图经过 AOTAutograd 处理前向和反向，再交给 TorchInductor 生成代码：GPU 上默认生成 Triton kernel，CPU 上生成 C++/OpenMP 代码。三个常用模式：

模式	机制	适用
`default`	kernel 融合，编译最快	日常
`reduce-overhead`	叠加 CUDA Graph，消除启动开销	小 batch、启动开销占比高
`max-autotune`	对矩阵乘等做自动调优	编译最慢，推理收益最高

首次执行有编译开销，要预热后再计时；形状变化会触发重新编译，可以用 `torch._dynamo.mark_dynamic()` 声明动态维度。

什么时候 compile 没用？ 模型时间主要花在大矩阵乘或 attention 上时，eager 已经在调用 cuBLAS/FlashAttention 这类高度优化的 kernel，融合能省的很少。下面的实测就是这种情况。

5.7 动手实践二：完整训练循环 + 四档对比

```
def run(mode: str, device: str, X: torch.Tensor, y: torch.Tensor, steps: int) -> tuple[float, float]:
    model = make_model(device)
    m = torch.compile(model) if "compile" in mode else model
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    loss_fn = nn.BCEWithLogitsLoss()
    amp_dtype = None
    if "amp" in mode:                              # CUDA 上优先 bf16；fp16 才需要 GradScaler
        amp_dtype = torch.bfloat16 if device != "cuda" or torch.cuda.is_bf16_supported() else torch.float16
    scaler = torch.amp.GradScaler(device, enabled=(amp_dtype == torch.float16))

    def step() -> torch.Tensor:
        with torch.autocast(device_type=device, dtype=amp_dtype or torch.float32, enabled=amp_dtype is not None):
            loss = loss_fn(m(X).squeeze(-1), y)   # autocast 只包前向
        opt.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()             # scaler 未启用时等价于 loss.backward()
        scaler.step(opt)
        scaler.update()
        return loss.detach()

    for _ in range(10):                            # 预热：编译、kernel 选择都发生在前几步
        step()
    sync(device)
    t0 = time.perf_counter()
```

```
    for _ in range(steps):
        last = step()                          # 循环里不调用 .item()，避免每步同步
    sync(device)
    return (time.perf_counter() - t0) / steps * 1e3, last.item()
```

第 1 版的这段代码有三个问题：

1. **计时循环里没有反向传播和优化器更新**，测的只是前向，却标为“训练耗时”。
2. **在 CPU 上会崩溃**：计时后无条件调用 `torch.cuda.synchronize()`，而文中说“CPU 可全程跑通”。
3. **四档共用一个模型对象**，前一档训练后的权重和 compile 包装会带进下一档。

第 2 版每档重新建模型，计时循环包含完整的训练步，并按设备选择同步方式。实测（模型约 110 万参数，batch 8192）：

模式	CPU（M4 Pro）	Apple GPU（MPS）
eager	28.9 ms/step	12.6 ms/step
AMP（bf16）	5,676 ms/step	10.5 ms/step
compile	30.3 ms/step	12.7 ms/step
compile + AMP	5,725 ms/step	10.8 ms/step

三个结论，都和“教程默认印象”不同：

1. **在这台机器的 CPU 上，bf16 自动混合精度慢了约 200 倍。** 这说明在 Apple M4 的 CPU 上，PyTorch 2.14 的 bf16 矩阵乘走的是慢速路径（具备 AMX-BF16 等指令的 x86 服务器 CPU 情况会不同）。混合精度首先是 GPU 上的优化，在 CPU 上要先测再用。
2. **在 MPS 上，bf16 带来约 1.2 倍加速。** NVIDIA 数据中心卡上通常更明显，但具体倍数取决于模型的计算密度。
3. **`torch.compile` 在这个模型上没有收益。** 这个 MLP 的时间几乎全在 1024×1024 的矩阵乘上，可融合的逐元素操作（GELU、偏置加法）占比很小。第 1 版“compile 再叠加 1.2 – 1.5×”是一个没有前提的经验数。要看到 compile 的收益，模型里要有大量小的逐元素操作，或者 batch 小到 kernel 启动开销占主导。

还有两个工程细节：计时前后必须同步，否则量到的是“排队速度”；循环里不要调用 `loss.item()`（见常见坑 1）。

5.8 常见坑

坑	现象	对策
训练循环里每步 `loss.item()`	GPU 利用率忽高忽低，吞吐远低于预期	每 N 步取一次；或在 GPU 上累加 `running += loss.detach()`，N 步后再 `.item()`
验证时忘记 `model.eval()` 和 `inference_mode()`	验证显存暴涨，Dropout/BatchNorm 行为异常	验证函数开头固定写这两行，结束后 `model.train()`
张量设备不一致	`Expected all tensors to be on the same device`；或 `.to()` 写了没接返回值	数据进模型前统一 `.to(device)`；`forward` 里新建张量时指定 `device=x.device`；持久张量用 `register_buffer`

坑	现象	对策
原地操作破坏计算图	`one of the variables needed for gradient computation has been modified by an inplace operation`	调试时 `torch.autograd.set_detect_anomaly(True)`；对参与求导的中间张量避免带下划线的原地方法
`from_numpy` 的“假拷贝”	改了 NumPy 数组，模型输入跟着变	需要独立副本用 `torch.tensor(arr)`
`num_workers>0` 时递归启动进程	`RuntimeError: An attempt has been made to start a new process...`	训练入口包在 `if __name__ == "__main__":` 里
CPU 上开 bf16 autocast	慢几十到几百倍（5.7 节）	混合精度只在 GPU 上默认开启；CPU 上先测

5.9 延伸阅读

- PyTorch 文档：Autograd mechanics；Automatic Mixed Precision（`torch.amp`）；`nn.Module`；`torch.utils.data`。
- `torch.compile` 官方教程（Dynamo 捕获与断图）。
- Karpathy 的 micrograd 仓库与配套视频。

第 6 章 分布式训练：DDP、FSDP 与并行策略

当模型参数加优化器状态超过单卡显存，或者想把训练时间从几周压到几天，就必须面对分布式。好消息是，你积累的分布式系统直觉大部分有效：集合通信是传输层，torchrun 是进程编排加服务发现，ZeRO 分片是分库分表，DDP 的权重一致性论证和状态机复制是同一个逻辑。本章把这些映射坐实，并用一个 CPU 就能跑的最小例子亲手验证。

6.1 本章目标

- 说清 NCCL 提供的集合通信原语（all-reduce、broadcast、all-gather、reduce-scatter）及其在训练中的角色。
- 画出并行策略地图：DP、TP、PP、CP、EP 各切什么、通信什么、什么时候用。
- 说清 DDP 的分桶梯度同步与通信计算重叠，以及 torchrun 的进程编排。
- 理解 ZeRO 1/2/3 的显存切分，掌握 FSDP2 `fully_shard` 的包装方式。
- 用“16 字节/参数”估算全量训练显存，用带宽估算通信时间，并据此选择并行方案。

6.2 Go 工程师视角

Go / K8s 世界	训练集群世界	要点
gRPC / RDMA 传输层	NCCL + NVLink / InfiniBand	集合通信库；ring all-reduce 是带宽最优的算法之一
systemd 拉起多进程	`torchrun --nproc-per-node=8`	每卡一个进程，注入 `RANK`、`WORLD_SIZE`、`LOCAL_RANK`
etcd / 服务发现	rendezvous（c10d）	所有进程先“会合”交换地址，再建通信组
K8s 重启策略	`torchrun --max-restarts=3`、弹性 `--nnodes=1:2`	worker 挂了，全员从最近的 checkpoint 重启
gang scheduling（Volcano 等）	训练任务是全有或全无	一张卡挂 = 全任务停；checkpoint 频率 ≈ WAL 刷盘频率的权衡
分库分表	ZeRO / FSDP 参数分片	DTensor 的 shard / replicate 布局
TCP 批量发送的权衡	DDP 梯度分桶（默认约 25 MB）	桶太小通信次数多，太大则与计算重叠得少
分布式 tracing / verbose 日志	`torch.profiler`、Nsight、`NCCL_DEBUG=INFO`	后者 ≈ gRPC 的详细日志

6.3 NCCL 与集合通信

NCCL（NVIDIA Collective Communications Library）是 GPU 集群的“传输层”，提供 AllReduce、Broadcast、Reduce、AllGather、ReduceScatter 等原语，针对 PCIe、NVLink、RoCE、InfiniBand 做了拓扑感知优化。最需要建立直觉的是 all-reduce：N 张卡各有一份梯度，all-reduce 之后每张卡都拿到全局之和（再除以 N 得到平均）。ring all-reduce 里每个节点只和左右邻居通信，每张卡的收发量约为数据量的 2(N−1)/N 倍，几乎与节点数无关，这是它能水平扩展的原因。

用通信量把原语钉在训练流程里（以下“参数量”指梯度或权重的字节数）：

- DDP：每步一次梯度 all-reduce，每卡收发约 2 倍参数量。
- FSDP（ZeRO-3）：前向 all-gather 权重 + 反向再 all-gather 一次 + 反向 reduce-scatter 梯度，每卡约 3 倍参数量。

这个数字直接决定拓扑设计。用 `code/ch06/memory_calc.py` 算一下：

```
7B  DDP 28 GB / FSDP 42 GB; 节点间 100 Gbps 上 DDP 纯通信约  2.24 s
7B  DDP 28 GB / FSDP 42 GB; 节点间 400 Gbps 上 DDP 纯通信约  0.56 s
7B  DDP 28 GB / FSDP 42 GB; NVLink ~900 GB/s 上 DDP 纯通信约 0.03 s
```

（按理想带宽计算的下限，实际还有协议开销。）节点内 NVLink 和节点间网络的带宽差一到两个数量级，所以**高频通信必须留在 NVLink 域内，跨节点通信必须被计算掩盖**。带宽不是细节，是架构约束。

6.4 并行策略地图

策略	切什么	通信什么	适用场景
DP（数据并行）	batch	每步一次梯度 all-reduce	模型放得下时的默认选择
TP（张量并行）	单层权重矩阵按行或列切	层内 all-reduce，非常频繁	单卡放不下一层；基本只在节点内用
PP（流水并行）	按层切到不同卡	微批次的激活点对点传递	很深的模型；有流水线气泡，用 1F1B 调度压缩
CP（上下文并行）	序列维度	attention 的 K/V 交换（如 ring attention）	长序列训练
EP（专家并行）	MoE 的不同专家	token 路由的 all-to-all	MoE 模型

选型可以压缩成几个问题：模型加优化器状态放得下一张卡吗？放得下就用 DDP。放不下但放得下“一层”吗？用 FSDP（ZeRO-3）。单层都放不下，或者要追求极限的算力利用率？在节点内叠加 TP。序列超长？再叠加 CP。EP 是 MoE 模型的独立维度。这个顺序背后的原则是：**把通信频率最高的维度放进带宽最高的物理域**。

6.5 DDP：分桶同步与通信计算重叠

`DistributedDataParallel` 的核心不是"反向全部算完再同步"，而是边算边同步：

1. **梯度按桶分组**（默认约 25 MB，可用 `bucket_cap_mb` 调）。一个桶里的梯度全部就绪后立即异步发起 all-reduce，与后续层的反向计算重叠。反向是从后往前算的，所以大部分通信被计算掩盖。
2. **all-reduce 后各 rank 拿到相同的平均梯度，各自独立执行 `optimizer.step()`。** 初始权重相同 + 每步梯度相同 ⇒ 权重永远相同。这与状态机复制的论证一致：相同初始状态 + 相同输入序列 ⇒ 相同状态。DDP 构造时还会从 rank 0 广播一次权重，作为初始一致性的保证。
3. **梯度累积**（多个微批次攒一次更新）时，前 N−1 个微批次的反向要包在 `with model.no_sync():` 里，否则每个微批次都白做一次 all-reduce。

torchrun 做三件事：按 `--nproc-per-node` 拉起 worker 进程；注入 `RANK`（全局序号）、`WORLD_SIZE`（总进程数）、`LOCAL_RANK`（节点内序号）、`MASTER_ADDR/MASTER_PORT`；worker 里调用 `dist.init_process_group()` 时，所有进程通过 rendezvous 会合并建立通信组。用 Go 工程师的话说：torchrun = systemd（拉起 + 注入环境变量）+ etcd（服务发现）+ K8s 重启策略（弹性容错）。多机时每个节点各跑一次 torchrun，只有 `--nnodes`、`--node-rank`、`--master-addr` 不同，就像给 StatefulSet 的每个 Pod 注入序号。

6.6 动手实践一：torchrun + DDP 最小完整例子（CPU 可跑）

没有 GPU 时用 gloo 后端，两个进程跑在 CPU 上，代码不用改：

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--same-data", action="store_true")
    ap.add_argument("--steps", type=int, default=5)
    args = ap.parse_args()

    use_cuda = torch.cuda.is_available()
    dist.init_process_group("nccl" if use_cuda else "gloo")   # RANK / WORLD_SIZE / MASTER_* 由 torchrun 注入
    rank, world = dist.get_rank(), dist.get_world_size()
    try:
        if use_cuda:
            local_rank = int(os.environ["LOCAL_RANK"])
            torch.cuda.set_device(local_rank)
            device = torch.device("cuda", local_rank)
            ddp_kwargs = {"device_ids": [local_rank]}
        else:
            device = torch.device("cpu")
            ddp_kwargs = {}                                   # CPU/gloo 不传 device_ids

        torch.manual_seed(0)                                  # 各 rank 初始权重相同（DDP 构造时还会从 rank 0 广播一次）
        model = DDP(nn.Linear(8, 1).to(device), **ddp_kwargs)
        opt = torch.optim.SGD(model.parameters(), lr=0.1)
        loss_fn = nn.MSELoss()

        seed = 1000 if args.same_data else 1000 + rank        # 默认每个 rank 一份不同数据，模拟数据分片
        g = torch.Generator().manual_seed(seed)
        x = torch.randn(16, 8, generator=g).to(device)
        y = torch.randn(16, 1, generator=g).to(device)

        for step in range(args.steps):                        # 第 1 版这里缺了训练循环
            loss = loss_fn(model(x), y)
            opt.zero_grad(set_to_none=True)
            loss.backward()                                   # 梯度按 bucket 就绪即 all-reduce，与反向计算重叠
            local_grad = model.module.weight.grad.clone()     # all-reduce 之后：已是全局平均梯度
            opt.step()
            # 把每个 rank 的本地 loss 收集起来看看
            losses = [torch.zeros(1, device=device) for _ in range(world)]
            dist.all_gather(losses, loss.detach().reshape(1))
            if rank == 0:
                print(f"step {step}: 各 rank 本地 loss = {[round(l.item(), 4) for l in losses]}  "
```

```
                    f"|grad| = {local_grad.norm().item():.4f}")

        w = model.module.weight.detach().clone()
        gathered = [torch.zeros_like(w) for _ in range(world)]
        dist.all_gather(gathered, w)
        if rank == 0:
            same = all(torch.equal(gathered[0], t) for t in gathered[1:])
            print(f"{world} 个 rank 的权重逐位相同: {same}")
    finally:
        dist.destroy_process_group()                          # 异常路径也要收尾，否则其它 rank 会挂在集合通信上
```

第 1 版的这段代码不能运行：它在两页之间缺了整个训练循环，`loss.backward()` 引用的 `loss` 和打印里的 `step` 都没有定义。第 2 版补上了循环，并且每步把各 rank 的本地 loss 收集起来打印，让“数据不同、梯度相同”可以直接看到。另外加了 `try/finally`，异常时也能销毁通信组。

```
$ torchrun --nproc-per-node=2 --master-port=29511 ddp_demo.py
step 0: 各 rank 本地 loss = [0.7578, 0.7318]  |grad| = 0.9796
step 1: 各 rank 本地 loss = [0.6389, 0.6165]  |grad| = 0.7116
step 2: 各 rank 本地 loss = [0.5775, 0.5484]  |grad| = 0.5329
step 3: 各 rank 本地 loss = [0.5448, 0.5054]  |grad| = 0.4124
step 4: 各 rank 本地 loss = [0.5268, 0.4767]  |grad| = 0.3295
2 个 rank 的权重逐位相同: True

$ torchrun --nproc-per-node=2 --master-port=29512 ddp_demo.py --same-data
step 0: 各 rank 本地 loss = [0.7578, 0.7578]  |grad| = 1.4292
...
step 4: 各 rank 本地 loss = [0.432, 0.432]  |grad| = 0.4133
2 个 rank 的权重逐位相同: True
```

怎么读这组输出：

- 默认模式下两个 rank 的本地 loss 不同（数据不同），但权重始终**逐位**相同（用 `torch.equal` 而不是近似比较），因为 all-reduce 之后它们拿到的是同一份平均梯度。
- `--same-data` 模式下两个 rank 的 loss 完全一样，平均梯度等于各自的梯度，all-reduce 什么也没改变，相当于白算了一份。这就是常见坑 3“数据没按 rank 切分”的样子：训练看起来一切正常，实际上 N 张卡在做同一件事。

在 macOS 上运行时，torchrun 会提示“Redirects are currently not supported in MacOs”，可以忽略；如果遇到网卡选择问题，设置 `GLOO_SOCKET_IFNAME=lo0`。有 GPU 时把 `--nproc-per-node` 改成卡数即可，设置 `NCCL_DEBUG=INFO` 可以看到 NCCL 选择的拓扑和建组过程。

6.7 ZeRO 1/2/3 与显存构成

显存四件套：参数、梯度、优化器状态、激活。bf16 混合精度 + Adam 的全量训练，每个参数约 16 字节：fp32 主权重 4 + fp32 一阶矩 4 + fp32 二阶矩 4 + bf16 参数 2 + bf16 梯度 2。ZeRO（DeepSpeed 论文提出，也是 FSDP 的理论基础）把这些状态切到 N 张卡上：

阶段	切分对象	每卡每参数字节	N=8 时
不切（DDP）	—	16	16
ZeRO-1	优化器状态	12/N + 4	5.5
ZeRO-2	+ 梯度	12/N + 2/N + 2	3.75
ZeRO-3	+ 参数本身	16/N	2

ZeRO-3 的代价是通信：前向时按需 all-gather 权重、用完即释放，反向再 all-gather 一次并 reduce-scatter 梯度。类比分库分表：数据是分片的，用的时候先"JOIN"拉全量再计算。

ZeRO 切的是前三件，激活不在其中。 激活显存大致与 batch × 序列长度 × 隐藏维 × 层数成正比，长序列、大 batch 时会反超参数状态成为大头，这就是"参数都切成 1/8 了为什么还 OOM"的答案。对应手段是激活检查点（activation checkpointing）：前向只保存每层的输入，反向时重算层内的中间激活，大约多付出一次前向的计算量（训练总计算量增加约三分之一），换来层内激活不再常驻显存。

6.8 框架格局（2026-10）

框架	现状	定位
FSDP2（`fully_shard`）	PyTorch 原生 ZeRO-3 实现，基于 DTensor 重写	新项目默认选择；每个参数仍是真实张量，便于与 TP、PP 组合、做部分冻结和混合精度
DeepSpeed	仍在活跃维护（当前 0.19.8）	ZeRO-Offload、部分 MoE 训练等存量方案
Megatron-Core	当前 0.19.2（2026-09，要求 Python ⩾ 3.12）	千亿参数级预训练，TP/PP/CP/EP 全栈；侵入式，不适合入门
torchtitan	PyTorch 官方"干净房间"参考实现，当前 0.3.0	用纯 PyTorch 原语（DTensor + FSDP2 + `torch.compile`）做多维并行；读源码理解分布式训练的最佳入口

torchtitan 在 2026 年 10 月的官方博客更新里，新增了对 DeepSeek-V4、Qwen3.5、Qwen3-VL、Kimi K3 等模型的多维并行支持，并推出了强化学习框架 TitanRL。

中小集群的甜点位是混合分片（hybrid shard）：节点内做 FSDP 分片，节点间做数据并行复制，把高频的 all-gather 和 reduce-scatter 限制在 NVLink 域内。

FSDP2 包装骨架（需要多张 GPU）

```
from torch.distributed.fsdp import fully_shard

for block in model.layers:      # 先逐个 block 分片：每个 block 前向时 all-gather 自己的权重，用完释放
    fully_shard(block)
fully_shard(model)              # 再兜底：embedding、输出层等剩余参数

# 优化器必须在 fully_shard 之后创建：分片后参数变成 DTensor，之前持有的引用会失效
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
```

启动方式与 DDP 相同。练习：在多卡机器上分别跑 DDP 版和 FSDP 版，打印 `torch.cuda.max_memory_allocated()`，验证"每卡显存从与模型大小成正比，降到与模型大小 / N 成正比"。

6.9 动手实践二：显存与通信估算（CPU 可跑）

```python
def full_ft_bytes_per_param(zero: int, n: int) -> float:
    """bf16 混合精度 + Adam：fp32 主权重 4 + m 4 + v 4 + bf16 参数 2 + bf16 梯度 2 = 16 字节/参数。"""
    opt, grad, param = 12.0, 2.0, 2.0
    if zero >= 1:
        opt /= n
    if zero >= 2:
        grad /= n
    if zero >= 3:
        param /= n
    return opt + grad + param
```

```
每卡每参数字节数（N=8 卡）
  ZeRO-0: 16.00 B    ZeRO-1:  5.50 B    ZeRO-2:  3.75 B    ZeRO-3:  2.00 B

全量微调静态显存（参数+梯度+优化器状态，不含激活）
  Qwen3-0.6B:     9.6 GB  →  ZeRO-3 切到 8 卡后每卡    1.2 GB
  Qwen3-8B  :   131.2 GB  →  ZeRO-3 切到 8 卡后每卡   16.4 GB
  70B       :  1120.0 GB  →  ZeRO-3 切到 8 卡后每卡  140.0 GB
```

三个结论：

1. Qwen3-8B 全量训练的静态状态约 131 GB，单张 80 GB 卡放不下；8 卡 ZeRO-3 后每卡 16.4 GB，剩余显存留给激活和通信缓冲。
2. 70B 全量训练的静态状态约 1.1 TB，一个 8×80 GB 节点（640 GB）放不下，至少需要 2 个节点；加上激活和缓冲，实际通常需要更多。这就是混合分片和激活检查点登场的地方。
3. 70B 用纯 DDP 根本不可能（每卡都要放下 1.1 TB），讨论跨节点 DDP 的通信时间对它没有意义；对 7B 这种单卡放得下权重的模型，跨节点 100 Gbps 的 DDP 每步纯通信就要 2 秒以上，必须靠分桶重叠来掩盖。

6.10 常见坑

坑	现象	对策
每个 rank 都打印、都存 checkpoint	日志重复 N 份；多进程同时写同一文件导致 checkpoint 损坏	IO 前判断 `rank == 0`；保存后 `dist.barrier()`
忘记 `destroy_process_group()`	训练结束进程不退出，挂在集合通信上	`try/finally` 里调用（6.6 节）
数据没按 rank 切分	loss 正常下降，但 N 张卡在做同一件事（6.6 节 `--same-data`）	用 `DistributedSampler`，并在每个 epoch 开头调 `sampler.set_epoch(epoch)`
NCCL 网络问题	多机训练挂在 `init_process_group`，或报连接失败	`NCCL_DEBUG=INFO` 看卡在哪一步；确认 `MASTER_ADDR/PORT` 可达；多网卡时设置 `NCCL_SOCKET_IFNAME`；用 `ib_write_bw` 检查 IB 链路；先用最小 all-reduce 脚本隔离问题
梯度累积没用 `no_sync()`	开了累积后吞吐不升反降	前 N−1 个微批次的反向包在 `with model.no_sync():` 里
FSDP 里优化器创建时机错误	分片后 loss 不更新，或报形状不匹配	顺序固定为：构建模型 → `fully_shard` → 创建优化器

6.11 延伸阅读

- PyTorch 文档：DDP 教程、`torch.distributed`、FSDP2（`fully_shard`）教程。
- ZeRO 论文（arXiv:1910.02054）。
- torchtitan 仓库与 2026-10 的 TorchTitan 更新博客。
- NCCL 文档与 NVIDIA 技术博客。

第 7 章 大模型微调：HuggingFace 生态与 LoRA

现实中很少有人从零写预训练循环。绝大多数工程场景是：拿一个开源基座，用 LoRA 在自己的指令数据上做监督微调（SFT），产出几十 MB 的 adapter，合并进主干后交给推理引擎（第 8 章）部署。本章走完这条链路，并在一台没有 NVIDIA GPU 的笔记本上把它实际跑通。

7.1 本章目标

- 画出 HuggingFace 生态地图，知道各组件的职责和当前版本约束。
- 说清 LoRA 的低秩分解原理，以及 QLoRA 的“NF4 + 双重量化 + 分页优化器”。
- 会算 LoRA 的可训练参数量，会估算 LoRA/QLoRA 的显存。
- 独立完成一次 Qwen3-0.6B + LoRA 的 SFT：构造数据、训练、保存 adapter、合并、生成验证。
- 避开版本、配置和供应链安全上的常见坑。

7.2 Go 工程师视角

Go 世界	HuggingFace 世界	要点
标准库 + 框架（net/http + chi）	transformers（模型、tokenizer）+ Trainer	Trainer ≈ 内置最佳实践的服务端框架
同一份 handler 挂到不同 server	Accelerate：同一份训练代码跑单卡、多卡、FSDP	代码与拓扑解耦
列存 + 流式读取	datasets：Arrow 内存映射 + streaming	大数据集不用全量加载进内存
中间件 / 插件	PEFT：往 `nn.Module` 树里注入 adapter 层	第 5 章 `_modules` 树的直接应用
增量发布	LoRA adapter：几 MB 到几十 MB 的 safetensors	基座不动，只发布增量
OS swap	QLoRA 的分页优化器	显存峰值时把优化器状态换到 CPU 内存
供应链安全（锁版本、校验 hash）	加载模型 = 解析外部输入	恶意 config 或 checkpoint 可导致远程代码执行

供应链安全需要单独强调。 2026 年 transformers 披露了多个可导致任意代码执行的漏洞，例如：

- CVE-2026-1839：`Trainer` 恢复 checkpoint 时用 `torch.load()` 加载 RNG 状态文件而没有加 `weights_only=True`，恶意 checkpoint 可执行任意代码，修复于 5.0.0rc3；
- CVE-2026-4372：恶意 `config.json` 里的 `_attn_implementation_internal` 字段可以让 `from_pretrained()` 从攻击者的仓库下载并执行代码，绕过 `trust_remote_code` 检查，修复于 5.3.0。

把模型和 checkpoint 当作“不可信输入”，与 Go 里对待用户上传的文件是同一级别：transformers 至少升到 5.3.0；只加载可信组织发布的 safetensors 权重；生产环境锁定全部依赖版本并校验 hash。

7.3 生态地图（2026-10-22 核查）

组件	职责	当前版本
transformers	模型定义、权重加载、Trainer、`generate`	5.20.2（v5 于 2026-01-26 正式发布）
Accelerate	启动抽象：同一份代码跑单卡、多卡、FSDP、DeepSpeed	1.15.0
datasets	数据集加载，Arrow 列存内存映射 + 流式	5.1.0
PEFT	LoRA、QLoRA、DoRA 等参数高效微调	0.21.2
TRL	SFT、DPO、GRPO 等后训练 Trainer	1.16.0（要求 Python ⩾ 3.11）
tokenizers	Rust 实现的分词器	0.23.3（1.0 处于 RC 阶段）

第 1 版写“transformers 2026-04 才发布首个 v5 RC”，不对：v5 的首个 RC 是 2025-12-01，正式版是 2026-01-26。transformers v5 要求 PEFT ⩾ 0.18；照老教程 `pip install peft` 而不加版本约束，可能装到旧版。

Trainer 怎么定位：它把日志、checkpoint、梯度累积、混合精度、分布式启动都内置了，代价是自定义逻辑要走 callback 和 hook。经验法则：SFT、DPO 这类标准任务用 TRL 的 Trainer；需要非常规 loss 组合、多模型交互、自定义调度时，回到第 5 章的手写循环 + Accelerate 做分布式平移，反而更清楚。

词表大的模型要注意 logits 的显存。 logits 张量的形状是 batch × 序列长度 × 词表大小。Qwen3 的词表约 15 万：batch 8、序列 8192 时，fp32 的 logits 就是 $8 \times 8192 \times 151936 \times 4$ 字节 ≈ 40 GB。朴素实现会先把完整的 logits 放进显存再算交叉熵；“融合输出层 + 分块交叉熵”（如 Liger-Kernel 的 fused linear cross entropy）把这一步融掉，长序列训练时能大幅降低峰值显存。

7.4 LoRA：低秩分解

预训练权重 W 是 $d \times k$ 的满秩矩阵，但微调带来的变化 ΔW 往往“内在秩”很低。LoRA 冻结 W，把增量参数化为两个低秩矩阵的乘积：

$$W' = W + \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

A 随机初始化、B 初始化为 0，所以训练开始时 $BA = 0$，模型行为与基座完全一致。前向时是一条旁路：$y = xW + \frac{\alpha}{r}(xA^\top)B^\top$（具体转置取决于实现），梯度只流过低秩分支。

可训练参数量怎么算。 以本章用的 Qwen3-0.6B 为例：28 层，隐藏维 1024，16 个 Q 头、8 个 KV 头、头维 128。所以 `q_proj` 是 1024→2048，`k_proj`、`v_proj` 是 1024→1024，`o_proj` 是 2048→1024。r=16 时每层 LoRA 参数为：

模块	输入 + 输出	×r
q_proj	1024 + 2048	49,152
k_proj	1024 + 1024	32,768
v_proj	1024 + 1024	32,768
o_proj	2048 + 1024	49,152
每层合计		163,840

28 层合计 4,587,520 个参数，占总参数 6.0 亿的 0.76%。下面实测 `print_trainable_parameters()` 打印的正是这个数。第 1 版写"约 0.3%"，与实际不符。

工程上的三个好处：优化器只维护 adapter 的状态，显存大头消失；adapter 产物很小（下面实测 18 MB），可以进 Git 或制品库做版本管理；推理前 `merge_and_unload()` 把 $\frac{\alpha}{r}BA$ 加回 W，没有额外推理延迟。

两个超参的直觉：r 决定 adapter 的容量，任务与基座差距越大（新领域术语、新输出格式）越要往大调，但收益会递减、过拟合风险上升，先用 r=16 跑通再调。`target_modules` 只打 attention 的 q/k/v/o 最省；把 MLP 的 `gate_proj`/`up_proj`/`down_proj` 也加进去，通常能进一步提升效果，代价是 adapter 体积和训练时间增加。

7.5 QLoRA：把基座压进 4 bit

QLoRA 在 LoRA 之上把冻结的基座量化到 4 bit：NF4（针对正态分布权重设计的 4 bit 数据类型）；双重量化（对量化常数再量化一次，每参数再省约 0.37 bit）；分页优化器（显存峰值时把优化器状态换页到 CPU 内存）。计算时基座权重反量化成 bf16 参与矩阵乘，存储和搬运是 4 bit。QLoRA 论文报告，4 bit 基座 + 16 bit adapter 的组合在多个基准上追平了 16 bit 全量微调。

显存账本（与第 6 章的 16 字节/参数对照）：

方案	基座权重每参数	7B 模型大致需求（含激活，量级）
全量训练（bf16 + Adam）	16 字节	约 112 GB 静态状态，需多卡 ZeRO/FSDP
LoRA（bf16 基座冻结）	2 字节 + 少量 adapter 状态	约 20 – 30 GB，24 GB 卡要控制序列长度
QLoRA（4 bit 基座）	约 0.5 字节 + 少量 adapter 状态	约 6 – 12 GB，16 GB 卡可跑

激活显存是另一条曲线，长序列时会成为大头，用 `gradient_checkpointing=True` 以重算换显存（第 6.7 节）。

基座选型。 教学和验证用 Qwen3-0.6B：Apache 2.0 许可，笔记本上就能训练。Qwen3 系列 2025-04-29 发布，稠密模型覆盖 0.6B 到 32B，另有 MoE 的 30B-A3B 与 235B-A22B。实战建议 Qwen3-4B/8B + QLoRA + 24 GB 单卡。选基座时把许可证当作依赖审计的一部分；基座能力决定微调的天花板，adapter 只是让迁移变便宜，不会让 0.6B 学会只有 8B 才知道的知识。

7.6 动手实践：Qwen3-0.6B + LoRA SFT 全链路

环境：NVIDIA GPU（≥16 GB）可加 `--qlora`；Mac 上自动用 Apple GPU（MPS）；纯 CPU 也能跑通，只是慢。

```
uv add "transformers>=5.3" "peft>=0.21" "trl>=1.16" "datasets>=5.1" "accelerate>=1.15"
uv add bitsandbytes          # 只有 QLoRA 需要，仅支持 NVIDIA GPU
python sft_lora.py --steps 30
```

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--steps", type=int, default=30)
    ap.add_argument("--qlora", action="store_true", help="4-bit 基座，仅 NVIDIA GPU")
    args = ap.parse_args()
    device = "cuda" if torch.cuda.is_available() else ("mps" if torch.backends.mps.is_available() else "cpu")

    # 1. 数据：messages 格式，SFTTrainer 会自动套用模型的 chat template
    ds = Dataset.from_list([{"messages": [{"role": "user", "content": q}, {"role": "assistant", "content": a}]}
                            for q, a in QA] * 20)
```

```
tok = AutoTokenizer.from_pretrained(MODEL_ID)
if tok.pad_token is None:
    tok.pad_token = tok.eos_token

# 2. 基座
if args.qlora:
    from peft import prepare_model_for_kbit_training
    from transformers import BitsAndBytesConfig
    bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                             bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_use_double_quant=True)
    model = AutoModelForCausalLM.from_pretrained(MODEL_ID, quantization_config=bnb, device_map="auto")
    model = prepare_model_for_kbit_training(model)        # 第 1 版缺这一步：处理 LN 精度与梯度检查点的输入梯度
else:
    dtype = torch.bfloat16 if device == "cuda" else torch.float32
    model = AutoModelForCausalLM.from_pretrained(MODEL_ID, dtype=dtype).to(device)

# 3. 注入 LoRA
cfg = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                 target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], task_type="CAUSAL_LM")
model = get_peft_model(model, cfg)
model.print_trainable_parameters()

# 训练前先看一次基座的回答，作为对照
def ask(m, q: str) -> str:
    prompt = tok.apply_chat_template([{"role": "user", "content": q}], tokenize=False,
                                     add_generation_prompt=True, enable_thinking=False)
    inputs = tok(prompt, return_tensors="pt").to(m.device)
    with torch.inference_mode():
        out = m.generate(**inputs, max_new_tokens=80, do_sample=False)
    return tok.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens=True).strip()

with model.disable_adapter():
    print("训练前:", ask(model, QA[0][0])[:120])

# 4. 训练
trainer = SFTTrainer(
    model=model, processing_class=tok, train_dataset=ds,
    args=SFTConfig(output_dir="./out", per_device_train_batch_size=2, gradient_accumulation_steps=4,
                   max_steps=args.steps, learning_rate=2e-4, logging_steps=5, save_strategy="no",
                   report_to=[], gradient_checkpointing=args.qlora,
                   bf16=(device == "cuda"), use_cpu=(device == "cpu")),
)
t0 = time.perf_counter()
trainer.train()
print(f"训练 {args.steps} 步耗时 {time.perf_counter() - t0:.0f}s")
for rec in trainer.state.log_history:
    if "loss" in rec:
        print(f"  step {rec['step']:>3}  loss {rec['loss']:.4f}")
trainer.save_model("./out/adapter")                       # 只保存 adapter，几 MB 到几十 MB

# 5. 合并：QLoRA 时不要合并进 4-bit 基座，而是重新加载 16-bit 基座再合并
if args.qlora:
    base = AutoModelForCausalLM.from_pretrained(MODEL_ID, dtype=torch.bfloat16, device_map="auto")
    merged = PeftModel.from_pretrained(base, "./out/adapter").merge_and_unload()
else:
    merged = model.merge_and_unload()                     # W' = W + (α/r)·BA，之后推理没有额外开销
merged.save_pretrained("./out/merged")
tok.save_pretrained("./out/merged")
print("训练后:", ask(merged, QA[0][0])[:120])
```

与第 1 版相比改了四处：

1. **QLoRA 加了 `prepare_model_for_kbit_training`**。它处理 4 bit 模型里归一化层的精度，并让梯度检查点在输入嵌入被冻结时仍能正常反传。
2. **QLoRA 不合并进 4 bit 基座**。第 1 版直接对 4 bit 模型调用 `merge_and_unload()`：把 bf16 的 BA 加到 4 bit 权重上会引入舍入误差，合并后的模型也不是推理引擎期望的标准权重。正确做法是重新加载 bf16 基座，再加载 adapter 合并。
3. **先用 `disable_adapter()` 生成一次，作为训练前的对照**，否则“生成验证”看不出微调有没有效果。

4. **Qwen3 默认开启思考模式**，生成验证时 `apply_chat_template` 传 `enable_thinking=False`，否则输出前面会先出现一段思考内容。

本机实测（Apple M4 Pro，MPS，float32，30 步）：

```
trainable params: 4,587,520 || all params: 600,637,440 || trainable%: 0.7638
训练前: 在 Go 语言中，`slice` 和 `array` 是两个非常重要的类型，它们在语法和用途上有一些
        显著的区别。以下是它们的主要区别：### 1. **定义方式** ...
训练 30 步耗时 26s
  step   5  loss 3.6470
  step  10  loss 2.2237
  step  15  loss 1.4298
  step  20  loss 0.7941
  step  25  loss 0.4077
  step  30  loss 0.2965
训练后: array 是定长值类型，赋值会整体拷贝；slice 是底层数组的视图（指针 + len + cap），
        赋值只拷贝这三个字段，多个 slice 可以共享底层数组。
```

产物检查：`out/adapter/adapter_model.safetensors` 只有 18 MB，没有全量权重；`out/merged/` 是合并后的完整模型，可以直接交给推理引擎。

怎么看这个结果：基座原本给出的是一篇泛泛的 Markdown 长答；30 步后，模型逐字复述了训练数据里的答案。在只有 3 条样本、重复 20 次的演示里，过拟合正是我们想看到的信号：它证明数据格式、chat template、LoRA 注入、训练、合并、生成这条链路是通的。它**不**证明模型学会了什么新能力。

从演示到生产，差距主要在数据，不在代码。 SFT 的共识是“质量远比数量重要”：几千条覆盖目标场景、格式一致、风格统一的高质量数据，通常胜过十万条噪声数据。迭代闭环建议这样搭：先固定一个 50 – 200 条的评测集（人工标注预期答案的关键点或评分规则），每轮训练后自动生成并打分，把“调 r、调学习率、加数据”变成有指标对照的实验。这与做性能优化时先建基准、再改代码是同一个纪律。

扩展到 24 GB 单卡 + 8B 模型：`MODEL_ID` 换成 `Qwen/Qwen3-8B`，加 `--qlora`（4 bit 基座约 5 GB）；`per_device_train_batch_size=4`、`gradient_accumulation_steps=4`；序列长度从 2048 起步；`target_modules` 扩到全部线性层。多卡时用 `accelerate launch --multi_gpu` 启动，同一份脚本即可。

部署方式二选一：要么像本例一样合并后存全量权重；要么保留 adapter 目录，部署时由推理引擎同时加载基座和 adapter（vLLM 支持多 LoRA 热加载，适合“一个基座挂多个业务 adapter”）。

7.7 常见坑

坑	现象	对策
transformers v5 与旧 PEFT 不兼容	照老教程安装后导入报错或行为异常	显式写版本约束；`pip check` 验证；老教程里的 `torch_dtype`、`use_auth_token` 等参数按 v5 文档改名
没设置 pad token	`Asking to pad but the tokenizer does not have a padding token`，或 loss 不降	`tok.pad_token = tok.eos_token`；不要拿普通 token 当 pad
训练和推理的 chat template 不一致	微调后输出格式混乱、不停冒出特殊 token	全链路统一用 `apply_chat_template`；SFTTrainer 对 messages 格式会自动套模板；换基座时重新检查模板（Qwen3 还要注意思考模式开关）
bf16/fp16 与硬件不匹配	loss 瞬间 NaN，或报类型不匹配	`torch.cuda.is_bf16_supported()` 探测；bf16 需要 Ampere（A100/RTX 30 系）及以上；V100、T4 只能用 fp16 + loss scaling

坑	现象	对策
加载不可信 checkpoint	环境被植入恶意代码	transformers ⩾ 5.3.0；只加载可信来源的 safetensors；锁定版本并校验 hash
把 adapter 当全量模型部署	推理引擎加载时报缺权重	合并后存全量权重，或部署时同时指定基座与 adapter
QLoRA 直接合并进 4 bit 基座	合并后效果变差，或权重格式不被推理引擎接受	重新加载 bf16 基座再合并（7.6 节第 2 点）

7.8 延伸阅读

- transformers、PEFT、TRL、Accelerate 官方文档（v5 迁移说明、`LoraConfig` 参数、`SFTConfig`）。
- LoRA 论文（arXiv:2106.09685）、QLoRA 论文（arXiv:2305.14314）。
- Qwen3 官方博客（模型规格与许可）。
- CVE-2026-1839、CVE-2026-4372 的官方公告（GitHub Security Advisory）。

第 8 章 推理引擎：vLLM 与 SGLang

训练或微调出的模型，最终要变成一个有 SLA 的在线服务。在 Go 世界里你会用负载均衡 + 应用服务器 + 连接池解决这件事；在大模型推理里，这个角色由推理引擎扮演，它同时是 HTTP 服务器、请求调度器、显存分配器和 kernel 编排器。本章讲清 vLLM 与 SGLang 的核心机制和关键参数，并把“高并发后端”的经验直接映射过来。

8.1 本章目标

- 用操作系统虚拟内存分页的类比讲清 PagedAttention，说清 continuous batching 为什么能大幅提高吞吐。
- 理解 vLLM V1 引擎的架构，正确设置 `gpu_memory_utilization`、`max_num_seqs`、`max_num_batched_tokens`。
- 说清 SGLang 的 RadixAttention 与结构化输出在哪些场景占优，以及为什么新项目不要再用 TGI。
- 理解 TTFT 与 TPOT 的定义，用自己写的流式压测脚本测出延迟与吞吐的权衡曲线。

8.2 Go 工程师视角

高并发后端（Go）	推理引擎	说明
每连接一个 goroutine + 调度器	continuous batching（迭代级调度）	每个解码步重新组 batch，请求随时进出
sync.Pool / 定长块分配器	PagedAttention 的 KV block 管理	默认 16 个 token 一个 block，逻辑块到物理块的映射，消除碎片
信号量 / 限流器	`max_num_seqs` + `max_num_batched_tokens`	并发序列上限 + 单步 token 预算，超出部分排队
缓存命中率	前缀缓存（prefix caching）命中率	公共前缀的 KV 直接复用，省掉重复计算
健康检查 + `/metrics`	vLLM 的 `/metrics`（Prometheus 格式）	队列深度、KV 池占用、延迟直方图
内存耗尽 → OOM	KV 池耗尽 → 抢占 → 尾延迟飙升	V1 抢占时丢弃 KV、之后重算

三个思维差异要提前校准：

1. 推理请求是**有状态的长请求**，一次请求在 GPU 上驻留几十毫秒到几分钟，全程占用显存。
2. 它的“内存”是 HBM 显存，贵，而且换出到主机内存的代价很高。
3. 解码阶段的瓶颈是**显存带宽**而不是算力：每生成一个 token 都要把整个模型权重读一遍。这和 CPU 服务“CPU 打满才算满”的直觉完全不同。

8.3 PagedAttention：把 KV cache 当虚拟内存管理

自回归解码每生成一个 token，都要和之前所有 token 的 Key/Value 做注意力。为了不每次都重算，引擎把每个 token 的 K/V 缓存在显存里，这就是 KV cache。一个 8B 级模型处理 4K 上下文，KV cache 就是 GiB 量级（第 9 章给出精确公式）。

朴素实现按最大长度预分配连续显存，就像在 Go 里给每个连接 `make([]byte, MaxInt32)`：绝大部分请求远短于上限，显存浪费在内部碎片上；长请求还需要大块连续空间，产生外部碎片。

PagedAttention（Kwon 等，SOSP 2023；论文报告相对当时的基线吞吐提升 2 – 4 倍）把操作系统的分页思想搬了过来：

- **定长分页**：KV cache 切成固定大小的 block（vLLM 在 CUDA 上默认每块 16 个 token），物理 block 可以散落在显存任意位置。
- **页表**：每个请求一张 block table，记录“第 i 个逻辑块在哪个物理块”，attention kernel 查表读取。
- **按需分配**：写满当前块时才申请下一块。
- **引用计数共享**：多个请求有相同前缀时，前缀的物理 block 引用计数加一，直接复用；这是前缀缓存的物理基础，也天然支持写时复制（同一个 prompt 采样多个回复）。

剩下的浪费只有每个请求最后一个没写满的块。

8.4 Continuous batching：迭代级调度

传统的静态批处理像一辆坐满才发、全员到齐才让下车的班车：一批请求一起进 GPU，要等最长的那条生成完才能整批释放，短请求的算力全在陪跑。

continuous batching（源自 Orca，OSDI 2022；论文在 GPT-3 175B 上报告相对 FasterTransformer 在相同延迟下最高 36.9 倍吞吐）把调度粒度从“一批请求”改成“一次迭代”：每完成一个解码步，调度器就检查一遍：有请求生成完了，立即移出 batch、释放 KV block；队列里有等待的请求，立即插进来填满空位。代价是调度器本身成了热点，每一步都要做一次调度决策，vLLM V1 为此把调度循环放进独立进程，与 GPU 执行异步流水。

V1 的两个默认特性：

- **前缀缓存**：同一个 system prompt、同一套 few-shot 示例、agent 多轮对话的历史，都是公共前缀。V1 默认开启，实现是“按 block 内容做哈希 + LRU 淘汰”。它对首 token 延迟收益很大，而据 vLLM V1 发布博客，命中率为 0 时吞吐损失也不到 1%，所以默认开启是安全的。
- **chunked prefill**：一条 8K token 的 prompt 做预填充会独占 GPU 数百毫秒，期间所有解码请求被卡住。chunked prefill 把长 prompt 切成块，每个调度步混排“一块预填充 + 若干解码”，两类请求互不饿死。这是单节点上消除预填充/解码干扰的首选手段，比拆架构（PD 分离，第 9 章）便宜得多。

投机解码：用一个小的草稿模型一次猜 k 个 token，大模型并行验证，接受正确的前缀、丢弃猜错的部分，把串行生成变成并行验证。它主要在低并发、延迟敏感的场景有收益；batch 越大，GPU 本来就越忙，收益越小。

8.5 vLLM V1 架构与版本线

版本事实（以 PyPI 和官方博客为准）：V1 引擎在 2025 年初以 alpha 发布，v0.8.0（2025-03）起成为默认；V0 在 v0.10.0（2025-07）宣布弃用，v0.11（2025-10）起代码移除。截至 2026-10-22，vLLM 最新版本是 0.32.0（要求 Python 3.11 – 3.14）。官方报告 V1 相对 V0 最高 1.7 倍吞吐。任何 v0.8 之前的教程、任何让你设置 `VLLM_USE_V1=0` 的文章，都已过时。

V1 是三层分离的多进程架构，用 Go 的话说就是“HTTP 层、调度层、执行层拆成不同进程，用共享内存队列连接”：

1. **API 前端进程**：OpenAI 兼容的 HTTP 服务，负责请求解析、分词、流式回包。
2. **EngineCore 进程**：调度循环所在。每一步决定哪些请求跑、各跑多少 token；CPU 侧的调度、采样与 GPU 执行异步重叠，避免 GPU 等 CPU。
3. **Worker / ModelRunner**：驱动 GPU kernel，管理 KV block 池、CUDA Graph 的捕获与回放。

解码这种“每一步算子序列都一样”的路径被捕获成 CUDA Graph 回放，消除逐算子的启动开销。但图捕获本身要占显存，这是新手最容易误判的 OOM 来源。

8.6 三个参数决定服务的性格

- `gpu_memory_utilization`：这不是“KV cache 的占比”，而是整个实例可用显存占总显存的比例。这份预算要装下模型权重、激活、CUDA Graph 和 KV block 池；引擎启动时先扣掉前三项，剩下的全部划给 KV 池。当前文档标注的默认值是 0.92，不同版本有过调整（第 1 版写的 0.9），以你所用版本的 `vllm serve --help` 为准。同一张卡上还要跑别的进程，就要调低。
- `max_num_seqs`：同时解码的序列上限，相当于服务端的限流阀。它同时决定 CUDA Graph 按多大的 batch 捕获（越大越占显存），超出的请求进入等待队列。调它就是在“单请求延迟”和“总吞吐”之间选点。
- `max_num_batched_tokens`：单个调度步的 token 预算。它必须不小于 `max_num_seqs`（每个活跃序列每步至少要解码 1 个 token），否则引擎拒绝启动。调大它允许一步里塞更多预填充，提高吞吐，但可能拉高 TPOT。

其他常用项：`max_model_len`（截断上下文，KV 需求等比缩小）、`--enable-prefix-caching`（V1 默认开）、投机解码相关参数。KV 池耗尽时，V1 的抢占策略是丢弃被抢占请求的 KV，之后重新计算：重算走的是预填充路径，计算密集、吞吐高，往往比在 HBM 和主机内存之间来回搬运 GiB 级数据更划算。

8.7 SGLang、TGI 与其他引擎

SGLang（LMSYS 维护，发布频繁，2026-10 时为 0.5.22）有两个结构性卖点：

- **RadixAttention**：用基数树（radix tree）组织 KV cache，自动共享任意公共前缀，多轮对话、agent 分叉轨迹这类“部分重叠”也能命中。可以理解为：vLLM 的哈希前缀缓存是按块精确匹配，RadixAttention 是最长前缀匹配的 trie。
- **结构化输出**：默认用 XGrammar 做约束解码，支持 JSON Schema、正则、EBNF。对下游是 Go 服务的团队，“输出必然是合法 JSON”意味着可以删掉整套解析重试逻辑（vLLM 同样支持结构化输出，也可以用 XGrammar 后端）。

TGI（HuggingFace Text Generation Inference）已于 2025-12-11 进入维护模式，只接受小的修复，官方建议迁移到 vLLM 或 SGLang。旧教程里的 TGI 一律视为过时。

TensorRT-LLM 是 NVIDIA 自研引擎，FP8 kernel 实现好、单节点极限性能强，但工程链较重，适合 NVIDIA 全栈、愿意为性能投入的团队。NVIDIA Dynamo 不是引擎，而是坐在 vLLM、SGLang、TensorRT-LLM 之上的编排层（1.0 于 2026-03 发布），做大规模 PD 分离和 KV 感知路由，第 9 章展开。

选型共识：单机单卡直接 vLLM；前缀重的 agent/RAG 场景 SGLang 有结构优势；大规模 PD 分离和 KV 路由用 Dynamo 或 llm-d 这类编排层；TGI 只出现在存量系统。

8.8 性能指标：TTFT、TPOT 与正确的压测口径

按 vLLM 的指标口径：

- TTFT（首 token 延迟）= 排队时间 + 预填充时间，衡量“多久开始说话”。交互式应用一般要求几百毫秒以内。
- TPOT（每输出 token 时间）=（端到端延迟 − TTFT）/（输出 token 数 − 1），衡量“说得流不流畅”。它与 ITL（相邻 token 间隔）在普通解码下数值接近，开启投机解码后两者会分离。
- 吞吐：输出 tokens/s 与请求数/s，必须和并发度或请求速率一起报告。只报单请求的数字是误导，推理服务的全部艺术在于并发下的延迟—吞吐权衡。

参考量级（社区常见的数量级，非本机实测）：8B 级模型、单张 A100、FP16，并发 1 时 TTFT 约 50 – 100 ms、TPOT 约 15 – 20 ms；并发拉到几十时，整卡输出吞吐可达每秒上千 token，但单请求的体验变差。

8.9 动手实践一：vLLM 离线批量推理（需要 NVIDIA GPU）

```
# offline_batch.py
import time
from vllm import LLM, SamplingParams

prompts = [f"用一句话解释分布式系统中的概念 {i}：" for i in range(100)]
llm = LLM(model="Qwen/Qwen3-1.7B", gpu_memory_utilization=0.85, max_model_len=2048)
sampling = SamplingParams(temperature=0.7, top_p=0.9, max_tokens=256)

t0 = time.perf_counter()
outputs = llm.generate(prompts, sampling)          # 引擎内部自动做 continuous batching
elapsed = time.perf_counter() - t0
n = sum(len(o.outputs[0].token_ids) for o in outputs)
print(f"请求 {len(outputs)}  输出 token {n}  耗时 {elapsed:.2f}s  吞吐 {n / elapsed:.0f} tokens/s")
```

首次运行会下载模型并做 CUDA Graph 捕获。启动日志里有一行 KV cache 大小（单位是 token 数），这是预算扣完权重和图之后划给 KV 池的部分，第 9 章的容量规划要与它对账。与 transformers 的 `generate` 循环相比，吞吐通常高一个数量级，差距来自 continuous batching 和 PagedAttention。（第 1 版这里用的是 Qwen2.5 系列，与第 7 章的 Qwen3 不一致，第 2 版统一为 Qwen3。）

8.10 动手实践二：`vllm serve` 起 OpenAI 兼容服务（需要 NVIDIA GPU）

```
vllm serve Qwen/Qwen3-8B --max-num-seqs 128 --gpu-memory-utilization 0.9 --port 8000

curl http://localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d '{
  "model": "Qwen/Qwen3-8B",
  "messages": [{"role": "user", "content": "用三句话介绍 Kubernetes"}],
  "max_tokens": 128}'

curl -s http://localhost:8000/metrics | grep -E "^vllm:(num_requests|kv_cache_usage|time_to_first)"
```

任何 OpenAI 客户端只要改 `base_url` 就能用。`/metrics` 里的指标名请注意：V1 的 KV 占用指标是 `vllm:kv_cache_usage_perc`（第 1 版写的 `vllm:gpu_cache_usage_perc` 是旧名字）；前缀缓存没有现成的“命中率”指标，要用计数器 `vllm:prefix_cache_hits` / `vllm:prefix_cache_queries` 在 PromQL 里自己算比值；TPOT 相关的直方图是 `vllm:request_time_per_output_token_seconds` 和 `vllm:inter_token_latency_seconds`。第 9 章的告警规则会用到它们。

8.11 动手实践三：流式压测脚本（CPU 可跑）

官方工具有 `vllm bench serve`（支持随机或 ShareGPT 数据集、请求速率、最大并发，直接输出 TTFT/TPOT/ITL 的均值和分位数）和 NVIDIA 的 genai-perf，生产压测优先用它们。自己写一个仍然值得：第 3 章的 asyncio 在这里落地，你也会真正理解这些指标是怎么算出来的。

```
async def one_request(client: httpx.AsyncClient, url: str, model: str, max_tokens: int,
                      sem: asyncio.Semaphore) -> dict:
    payload = {
        "model": model,
        "messages": [{"role": "user", "content": "写一段 200 字左右的分布式系统简介"}],
        "max_tokens": max_tokens,
        "temperature": 0.7,
        "stream": True,
        "stream_options": {"include_usage": True},              # 让服务端在最后一个 chunk 里报告精确 token 数
    }
    async with sem:                                             # 第 1 版声明了 sem 却没用，并发实际不受控
        t_start = time.perf_counter()                           # 第 1 版缺这一行，运行即 NameError
        ttft, n_chunks, usage_tokens = None, 0, None
        async with client.stream("POST", f"{url}/v1/chat/completions", json=payload) as resp:
            resp.raise_for_status()
            async for line in resp.aiter_lines():
                if not line.startswith("data:"):
                    continue
                data_str = line[len("data:"):].strip()
                if data_str == "[DONE]":
                    break
                data = json.loads(data_str)
                if data.get("usage"):
                    usage_tokens = data["usage"].get("completion_tokens")
                if data.get("choices") and data["choices"][0].get("delta", {}).get("content"):
                    n_chunks += 1
                    if ttft is None:
                        ttft = time.perf_counter() - t_start
        e2e = time.perf_counter() - t_start
    tokens = usage_tokens or n_chunks                           # 没有 usage 时用 chunk 数近似
    tpot = (e2e - ttft) / (tokens - 1) if ttft is not None and tokens > 1 else None   # vLLM 的 TPOT 口径
    return {"ttft": ttft, "e2e": e2e, "tpot": tpot, "tokens": tokens}
```

```
async def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--url", default="http://localhost:8000")
    ap.add_argument("--model", required=True)
    ap.add_argument("--concurrency", type=int, default=16)
    ap.add_argument("--requests", type=int, default=64)
    ap.add_argument("--max-tokens", type=int, default=256)
    args = ap.parse_args()
```

```
    sem = asyncio.Semaphore(args.concurrency)
    timeout = httpx.Timeout(connect=10.0, read=300.0, write=30.0, pool=None)
    limits = httpx.Limits(max_connections=args.concurrency)
    async with httpx.AsyncClient(timeout=timeout, limits=limits) as client:
        t0 = time.perf_counter()
        results = await asyncio.gather(*[one_request(client, args.url, args.model, args.max_tokens, sem)
                                         for _ in range(args.requests)])
        wall = time.perf_counter() - t0

    ttfts = [r["ttft"] for r in results if r["ttft"] is not None]
    tpots = [r["tpot"] for r in results if r["tpot"] is not None]
    e2es = [r["e2e"] for r in results]
    total_tokens = sum(r["tokens"] for r in results)            # 第 1 版用了这个变量却没定义
    print(f"并发 {args.concurrency:>3}  请求 {len(results)}  墙钟 {wall:6.2f}s")
    print(f"  TTFT  mean {statistics.mean(ttfts) * 1e3:7.1f} ms  p50 {pct(ttfts, .5) * 1e3:7.1f}  p99 ↩
        {pct(ttfts, .99) * 1e3:7.1f}")
    print(f"  TPOT  mean {statistics.mean(tpots) * 1e3:7.2f} ms  p99 {pct(tpots, .99) * 1e3:7.2f}")
    print(f"  E2E   mean {statistics.mean(e2es):7.2f} s")
    print(f"  吞吐  {total_tokens / wall:8.1f} tokens/s   {len(results) / wall:6.2f} req/s")
```

第 1 版的这个脚本有三处运行即崩溃或结果错误的问题： 计时起点 `t_start` 从未赋值（第一个请求就 `NameError`）；声明了信号量却没有 `async with sem`，`--concurrency` 参数形同虚设，所有请求同时发出；最后打印吞吐时用了未定义的 `total_tokens`。第 2 版全部修正，并加了 `stream_options.include_usage`，让服务端在最后一个 chunk 里报告精确的 token 数，不再只能用 chunk 数近似。

没有 GPU 也能调通：模拟服务

为了在没有 GPU 的机器上验证压测脚本，`code/ch08/mock_llm_server.py` 用 aiohttp 实现了一个 OpenAI 兼容的流式接口。它模拟推理引擎的两个关键行为：首 token 之前等待一段与 prompt 长度和当前负载相关的时间；之后每隔一段时间吐一个 token，而且同时在解码的请求越多，每一步越慢（模拟 batch 变大）。

```
python mock_llm_server.py --port 18001 &
for c in 1 8 32 64; do python bench_stream.py --url http://127.0.0.1:18001 --model mock \
    --concurrency $c --requests $((c*4)) --max-tokens 64; done
```

实测：

并发	TTFT 均值	TTFT p99	TPOT 均值	端到端均值	输出吞吐
1	47 ms	110 ms	11.8 ms	0.79 s	81 tokens/s
8	57 ms	129 ms	13.5 ms	0.91 s	552 tokens/s
32	95 ms	109 ms	18.1 ms	1.23 s	1,627 tokens/s
64	151 ms	176 ms	24.6 ms	1.70 s	2,373 tokens/s

这是模拟数据，绝对值没有意义，但曲线的形状和真实引擎一致：**并发从 1 到 64，总吞吐涨了 29 倍，单请求的 TPOT 却变差了一倍。**把并发扫一遍得到的就是延迟—吞吐权衡曲线，`max_num_seqs` 决定你停在曲线的哪个位置。对真实服务跑同样的扫描，再与 `vllm bench serve` 的报告对照，两者口径一致时数值应当接近。

这个脚本就是 Go 里“errgroup + 信号量压 HTTP 服务”的 Python 版：goroutine 换成 task，`sem <- struct{}{}` 换成 `async with sem`，其余是同一门手艺。

8.12 常见坑

坑	现象	对策
`max_num_batched_tokens` 小于 `max_num_seqs`	服务起不来	保证前者 ⩾ 后者；想减少预填充对解码的干扰，靠默认开启的 chunked prefill，不要把 token 预算压到并发上限以下
把 TTFT 当整体延迟	报告“平均延迟 200 ms”，用户却说要等 5 秒	体感延迟 ≈ TTFT + 输出 token 数 × TPOT；报告同时给三个数，并按输出长度分桶
忽略前缀缓存命中率	评测集上 TTFT 50 ms，真实流量 500 ms	评测 prompt 共享长 system prompt，命中率接近 100%；真实流量不是。压测数据的 prompt 结构要对齐真实流量，并用 `prefix_cache_hits / prefix_cache_queries` 监控
把 CUDA Graph 占用误判为泄漏	按公式算只要 30 GB，80 GB 卡却起不来；或调大 `max_num_seqs` 后显存“凭空”少了	启动日志分别打印权重、图、KV 池，逐项核对；显存紧张时先降 `max_num_seqs`
多副本用轮询负载均衡	前缀缓存命中率很低	多轮对话的后续请求被打到别的副本，前缀全要重算。副本少时做会话亲和；规模大了用 KV 感知路由（第 9 章）

8.13 延伸阅读

- vLLM 文档：V1 架构、引擎参数、指标设计（Metrics）、`vllm bench serve`。
- PagedAttention 论文（SOSP 2023）；Orca 论文（OSDI 2022）。
- SGLang 文档与 RadixAttention 论文；XGrammar。
- HuggingFace TGI 文档首页的维护模式说明。

第 9 章 AI Infra 工程：容量规划、量化、调度与可观测

第 8 章解决了“把模型跑成服务”。本章解决接下来的四个工程问题：显存够不够用（KV cache 容量规划），不够用怎么办（量化），流量大了怎么扩（PD 分离与编排），线上出了问题怎么看（可观测）。方法和你在 Go 后端做容量规划、成本优化、Prometheus 监控完全一样，只是资源从内存和 CPU 换成了显存和 SM。

9.1 本章目标

- 默写 KV cache 显存公式，能直接从 `config.json` 算出每 token 字节数，并完成“给定 GPU 和并发目标，反推可行上下文”的容量规划。
- 说清 FP8、INT4（AWQ/GPTQ）、bitsandbytes、FP8 KV cache 各自的适用边界，知道 A100 上 FP8 为什么不提速。
- 说清 PD 分离为什么把预填充和解码拆到不同节点、依赖什么网络条件、为什么中小规模先别上。
- 描述 Ray 与 K8s 在推理侧的角色，以及 GPU 调度的基本手段。
- 搭起“vLLM 指标 + DCGM + Prometheus”的监控分层，并解释为什么 `nvidia-smi` 的 GPU-Util 不等于真实利用率。

9.2 Go 工程师视角

Go 后端容量与运维	AI Infra 对应物	关键差异
每连接内存 × 并发数	每 token KV 字节 × 上下文 × 并发	显存不能超卖、不能 swap，算错就是抢占
压缩、降精度存储	权重量化 + KV cache 量化	有损，需要精度评测背书
读写分离、冷热分离	PD 分离（预填充/解码分池）	预填充计算密集，解码带宽密集
K8s + 水平扩缩容	KubeRay / GPU Operator / Kueue	GPU 是离散的整卡资源，装箱比 CPU 苛刻
Prometheus + node_exporter	vLLM `/metrics` + DCGM exporter	GPU-Util=100% 不等于在做有效工作
pprof / go tool trace	Nsight Systems / Nsight Compute / torch profiler	分层一致：时间线级、kernel 级、长期监控

最重要的校准：**GPU 服务的资源是离散、刚性、不可超卖的。** 主机内存超了还有 swap 兜底，CPU 超了只是变慢；显存超了就是 OOM 或抢占，尾延迟直接爆炸。所以 AI Infra 的容量规划是硬约束问题，必须能算、能对账。

9.3 KV cache 显存公式

$$M_{token} = 2 \times L \times n_{kv} \times d_{head} \times b, \qquad M_{total} = M_{token} \times \text{seq_len} \times \text{concurrency}$$

逐项解释：系数 2 是 K 和 V 各存一份；L 是层数；n_{kv} **必须用** `config.json` **里的** `num_key_value_heads`：GQA/MQA 模型的 KV 头数远少于 Q 头数，用错会成倍高估；d_{head} 优先读 `head_dim`，没有时才用 `hidden_size / num_attention_heads`；b 是每个元素的字节数，BF16/FP16 为 2，FP8 为 1。

Qwen3-8B 的 `config.json`（2026-10-22 从 HuggingFace 读取）：36 层、32 个 Q 头、8 个 KV 头、`head_dim` 128、最大上下文 40960。代入：

$$2 \times 36 \times 8 \times 128 \times 2 = 147{,}456 \text{ B} = 144 \text{ KiB/token}$$

跑满 40960 上下文的一个请求就是 144 KiB × 40960 = 5.625 GiB；8 个这样的并发是 45 GiB。KV cache 对序列长度是线性的（平方级的是 attention 的计算量），所以“上下文翻倍、显存翻倍”精确成立。

有两类架构会让公式变成上界：滑动窗口注意力（部分层只保留固定窗口的 KV，不随上下文增长）和 MLA（DeepSeek 系列把 KV 压缩成低维潜向量）。生产上把公式当上界用永远安全，做精细规划时再查具体架构。

9.4 动手实践一：KV cache 容量规划脚本（CPU 可跑）

```
def kv_bytes_per_token(cfg: dict, kv_bytes: int) -> int:
    """每 token KV 字节 = 2(K 和 V) × 层数 × KV 头数 × 头维 × 每元素字节。"""
    layers = cfg["num_hidden_layers"]
    kv_heads = cfg.get("num_key_value_heads", cfg["num_attention_heads"])   # GQA：必须用 KV 头数
    head_dim = cfg.get("head_dim") or cfg["hidden_size"] // cfg["num_attention_heads"]
    return 2 * layers * kv_heads * head_dim * kv_bytes
```

```
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default="https://huggingface.co/Qwen/Qwen3-8B/raw/main/config.json")
    ap.add_argument("--params-b", type=float, default=8.19, help="参数量（十亿），以模型卡为准")
    ap.add_argument("--weight-bytes", type=float, default=2, help="权重每参数字节：BF16=2，FP8=1，INT4≈0.5")
    ap.add_argument("--kv-bytes", type=int, default=2, help="KV 每元素字节：BF16=2，FP8=1")
    ap.add_argument("--gpu-gib", type=float, default=80)
    ap.add_argument("--util", type=float, default=0.90, help="对应 vLLM 的 --gpu-memory-utilization")
    ap.add_argument("--overhead-gib", type=float, default=2.0, help="激活 + CUDA graph 的保守预留")
    a = ap.parse_args()

    cfg = load_config(a.config)
    per_tok = kv_bytes_per_token(cfg, a.kv_bytes)               # 第 1 版的 per_tok_kib 没有定义
    weights = a.params_b * 1e9 * a.weight_bytes / GIB
    budget = a.gpu_gib * a.util
    pool = budget - weights - a.overhead_gib
    print(f"层数 {cfg['num_hidden_layers']}  KV 头 {cfg.get('num_key_value_heads')}  "
          f"Q 头 {cfg['num_attention_heads']}  头维 {cfg.get('head_dim')}  最大上下文 ↩
              {cfg.get('max_position_embeddings')}")
    print(f"显存预算 {budget:.1f} GiB = 权重 {weights:.1f} + 预留 {a.overhead_gib:.1f} + KV 池 {pool:.1f}")
    print(f"每 token KV {per_tok / 1024:.0f} KiB   KV 池可容纳 {pool * GIB / per_tok / 1e3:.0f}K tokens")
    print(f"\n{'并发':>4} {'上下文':>7} {'KV 需求 GiB':>11} {'结论':>6}")
    for conc, ctx in [(1, 40960), (8, 40960), (10, 40960), (16, 8192), (64, 4096), (128, 4096), (256, 2048)]:
        need = per_tok * conc * ctx / GIB
        print(f"{conc:>4} {ctx:>7} {need:>11.2f} {'可行' if need <= pool else '放不下':>6}")
    if cfg.get("num_key_value_heads") and cfg["num_key_value_heads"] != cfg["num_attention_heads"]:
        mha = per_tok * cfg["num_attention_heads"] // cfg["num_key_value_heads"]
        print(f"\n若误把 Q 头数当 KV 头数：每 token {mha / 1024:.0f} KiB，高估 {cfg['num_attention_heads'] // ↩
            cfg['num_key_value_heads']} 倍")
```

第 1 版的这个脚本运行即崩溃：它打印了 `per_tok_kib`，却从未定义这个变量。第 2 版改为直接读 `config.json`（本地文件或 URL），所有参数都可以从命令行覆盖，并且自动给出“误把 Q 头数当 KV 头数”会高估多少。

```
$ python kv_capacity.py
层数 36  KV 头 8  Q 头 32  头维 128  最大上下文 40960
显存预算 72.0 GiB = 权重 15.3 + 预留 2.0 + KV 池 54.7
每 token KV 144 KiB   KV 池可容纳 399K tokens

  并发     上下文   KV 需求 GiB     结论
   1   40960        5.62     可行
   8   40960       45.00     可行
  10   40960       56.25    放不下
  16    8192       18.00     可行
  64    4096       36.00     可行
 128    4096       72.00    放不下
 256    2048       72.00    放不下

若误把 Q 头数当 KV 头数：每 token 576 KiB，高估 4 倍
```

第 1 版的示例表格挑了 7 组“全部可行”的组合，看不出边界在哪里。第 2 版刻意放了几组放不下的：同一张 80 GB 卡，满上下文最多撑 9 个并发；4K 上下文最多撑 97 个。把 KV 改成 FP8（`--kv-bytes 1`），每 token 降到 72 KiB，池子能容纳 797K tokens，可行并发直接翻倍。

验证分两步：先手算核对 144 KiB/token；再真正起服务（`vllm serve Qwen/Qwen3-8B --gpu-memory-utilization 0.9`），启动日志会打印 KV cache 能容纳的 token 数，与脚本估算对账，差异应在“预留开销”的量级内。差太多，说明 CUDA Graph 或激活占用超出预期，回头调 `max_num_seqs` 或预留假设。

9.5 量化：用精度评测换来的显存

量化不是免费的压缩，每一次量化决策都应该是“精度评测数据 + kernel 支持矩阵”的函数。大样本的公开研究（例如 Neural Magic 团队在 ACL 2025 发表、覆盖 50 万次以上评测的论文 arXiv:2411.02355）给出的格局可以归纳为三档：

第一档：FP8 W8A8（权重和激活都是 8 bit）。 精度基本无损，是 Hopper/Blackwell 上的默认推荐。要分清两件事：**省显存**到处都成立；**算力加速**只有具备 FP8 Tensor Core 的 GPU（Ada、Hopper、Blackwell）才成立。A100（Ampere）运行 FP8 模型只能走“权重 8 bit、计算 16 bit”的 Marlin kernel：权重省一半显存，计算时反量化回 16 bit，不提速。把 FP8 模型部署到 A100 上期待性能翻倍，是最常见的代际误配。

第二档：INT4 权重（AWQ / GPTQ，只量化权重）。 精度恢复通常在 96% – 100%，两种算法在学术基准上的差距很小，校准集的质量比算法名字更重要。显存约为 BF16 的四分之一，是显存紧张场景的主力。工具链已经换代：AutoAWQ 已停止维护，由 vLLM 团队维护的 llm-compressor（PyPI 包名 `llmcompressor`）接替；AutoGPTQ 已归档，由 GPTQModel 接替。vLLM 加载 INT4 模型时，确认走的是 Marlin 一类的高效 kernel。

第三档：bitsandbytes（8 bit / 4 bit 加载）。 一行参数就能把显存压下来，适合快速验证“这个模型这张卡能不能跑”，但高吞吐服务场景不如专门的 FP8/Marlin kernel，定位是调试手段。

别忘了 KV cache 也能量化。 FP8 KV cache 让 KV 显存减半，等价于同样显存下并发或上下文翻倍（9.4 节实测），质量损失通常很小。长上下文业务里，它往往是性价比最高的一档：不动权重和计算路径，只动缓存。

选型口诀：Hopper/Blackwell 上用 FP8 W8A8；显存紧张用 INT4（配高效 kernel）；KV 压力大先开 FP8 KV cache；GGUF 只用于 llama.cpp/Ollama 这类本地场景，不做服务端高吞吐。上线前**必须过业务评测集**：96% – 100% 是学术基准上的统计结论，你的业务分布不一定落在平均值附近。

9.6 PD 分离：两种负载拆到两个池子

预填充是计算密集型，解码是带宽密集型。混在同一批 GPU 上，两种 SLO 会互相干扰：一条长 prompt 的预填充会拉高所有解码请求的 TPOT。chunked prefill 是单节点上的软件解法；PD 分离是集群级的架构解法：预填充池只负责“读 prompt、产出首 token 和 KV”，然后把 KV 通过高速网络传给解码池，后者专职逐 token 生成。两个池子可以独立扩缩容、独立调 batch 策略，甚至用不同代际的 GPU。

这套架构已经在生产中验证：

- DeepSeek 在 2025-02 公开的 V3/R1 推理系统概览中采用 PD 分离：预填充的部署单元是 4 个节点（32 张 H800），解码的部署单元是 18 个节点（144 张 H800）。（第 1 版写的“3 个预填充节点 + 9 个解码节点”不是 DeepSeek 的官方部署。）
- LMSYS 在 2025-05 用 SGLang 在 H100 集群上开源复现了这套架构，吞吐接近 DeepSeek 公布的数据。
- 学术上，Mooncake（Kimi 的服务系统）获得 FAST 2025 最佳论文，DistServe 发表于 OSDI 2024。

生态侧：NVIDIA Dynamo（1.0 于 2026-03 发布，当前 1.5.x）提供 PD 分离、KV 感知路由、NIXL 传输和多级 KV 缓存；llm-d 是 K8s 原生的方案；vLLM 引擎自身的 PD 分离仍标注为实验特性，通过 KVConnector 接口接入 NIXL、Mooncake、LMCache 等传输实现。

但对中小规模，要先泼冷水。 原因有三：

1. **KV 传输是硬成本。** 长上下文请求的 KV 是 GB 级的，必须走 RDMA（InfiniBand/RoCE）或 NVLink；退回 TCP 时传输延迟会吃掉收益。规则很直白：KV 传输的耗时必须小于它所消除的解码停顿。
2. **分池的取整损耗。** 小集群拆成两个池子后，各自都难以把整卡用满。
3. **大多数 TPOT 抖动有更便宜的解法。** 先确认 chunked prefill 已开、`max_num_batched_tokens` 合理、前缀缓存命中率正常，再考虑拆架构。

几十张卡以下的团队：先把单节点参数和 SLO 调好，PD 分离是规模上来之后自然出现的选项。

9.7 Ray 与 K8s：推理服务的编排层

单个 vLLM 实例是单机进程；把它变成多副本、自动扩缩、灰度发布的服务，需要编排层。

Ray 生态。 Ray 当前版本 2.59（2026-10），2025 年捐赠给 PyTorch 基金会。2026-07-30，Nscale 宣布收购 Ray 的商业公司 Anyscale（官方未披露金额，媒体报道约 16.5 亿美元，交易预计 2026 年下半年完成）。推理侧最有用的是 `ray.serve.llm`：把“模型 + 副本数 + 扩缩策略”声明成配置，底层用 vLLM，Ray Serve 负责多副本编排和流量分发；上 K8s 用 KubeRay。

安全上必须知道的一件事：CVE-2025-62593。 Ray Dashboard 曾经只靠“检查 User-Agent 是否以 Mozilla 开头”来阻止浏览器发起的请求，而 Firefox 和 Safari 允许网页脚本改写这个请求头；配合 DNS rebinding，开发者只要在本机运行着 Ray 时打开一个恶意网页，就可能被远程执行命令。漏洞在 Ray 2.52.0（2025-11）修复，CISA 于 2026-08-17 将其加入已知被利用漏洞目录。要点有三：升级到 2.52.0 以上；**2.52.0 新增的 token 认证默认是关闭的，需要手动开启**；Dashboard 默认端口 8265 只绑定本机地址，不要用 `--dashboard-host=0.0.0.0` 暴露出去。这与“不要把 Go 的 pprof 端口暴露到公网”是同一条纪律。（第 1 版写的“需 ⩾ 2.56”不准确。）

K8s + GPU 调度。 底座是 NVIDIA GPU Operator，一次装齐驱动、容器工具链和 DCGM exporter。整卡分配就是熟悉的 `nvidia.com/gpu: 1`；细粒度共享用 MIG（硬件级切分，A100/H100 支持）或 time-slicing（时间片复用，没有隔离保证，只适合开发环境）。GPU 是离散资源，装箱问题比 CPU 苛刻得多，批调度和配额靠 Volcano、Kueue 等调度器补充；再往上还有 KServe 等模型服务 operator。对 Go 工程师的类比：K8s 把 GPU 当成"不能超卖、不能热迁移"的特殊资源调度，Deployment 写法不变，调度语义接近裸金属分配。

9.8 可观测分层：和 Go 后端同一套 Prometheus 纪律

第一层：应用指标（vLLM `/metrics`）。 关键指标（按 vLLM 当前指标设计文档核对）：

指标	类型	含义
`vllm:num_requests_running` / `vllm:num_requests_waiting`	Gauge	运行中 / 排队中的请求数（≈ 消息队列积压深度）
`vllm:kv_cache_usage_perc`	Gauge	KV 池占用率，0 – 1
`vllm:prefix_cache_queries` / `vllm:prefix_cache_hits`	Counter	前缀缓存查询数 / 命中数，比值即命中率
`vllm:time_to_first_token_seconds`	Histogram	TTFT
`vllm:inter_token_latency_seconds`	Histogram	相邻输出之间的间隔
`vllm:request_time_per_output_token_seconds`	Histogram	每个请求的平均 TPOT
`vllm:e2e_request_latency_seconds`	Histogram	端到端延迟

第 1 版用到的 `vllm:gpu_cache_usage_perc`、`vllm:prefix_cache_hit_rate`、`vllm:time_per_output_token_seconds` 在 V1 中都不存在了，照抄会得到一组永远为空的面板和永远不触发的告警。SGLang 有同构的 `sglang:*` 指标。

第二层：GPU 硬件指标（DCGM exporter，默认端口 9400）。 最重要的认知纠偏：`DCGM_FI_DEV_GPU_UTIL`（也就是 `nvidia-smi` 里的 GPU-Util）的含义是"采样窗口内是否至少有一个 kernel 在执行"，一个只用了 1% SM 的 kernel 也能把它顶到 100%。真实的 SM 利用率看 `DCGM_FI_PROF_SM_ACTIVE`。再加上 `DCGM_FI_DEV_FB_USED`（显存占用）和 XID 错误（硬件故障前兆），就是完整的硬件视图。解码阶段的瓶颈常在显存带宽，"Util 很高、吞吐很低"往往是带宽打满了。

第三层：按需 profiling。 Nsight Systems（`nsys`）做时间线级分析，开销低，能看清调度空隙、kernel 间隙、CPU 与 GPU 是否重叠；Nsight Compute（`ncu`）做单个 kernel 的深入分析；torch profiler 能看调用栈但开销较大，不要在生产常开。对应到 Go：nsys ≈ `go tool trace`，ncu ≈ 函数级基准测试，torch profiler ≈ 全量 pprof。

Grafana 面板建议四行一屏：流量（请求速率、排队深度）、延迟（TTFT/TPOT 分位数）、资源（SM active、显存占用、KV 池占用、前缀命中率）、成本（每千 token 成本）。这与 Go 微服务的 RED + USE 方法论完全同构。

9.9 动手实践二：Prometheus 抓取与告警（指标名已按 V1 修正）

```
# prometheus.yml 片段
scrape_configs:
  - job_name: vllm
    scrape_interval: 15s
    static_configs:
      - targets: ["vllm-server:8000"]     # vllm serve 默认暴露 /metrics
  - job_name: dcgm
    scrape_interval: 15s
    static_configs:
      - targets: ["gpu-node:9400"]        # DCGM exporter（DaemonSet 部署）
```

```
# alerts.yml 片段
groups:
  - name: vllm-slo
    rules:
      - alert: VllmKVCacheNearFull        # 资源将尽：KV 池打满会触发抢占，尾延迟飙升
        expr: vllm:kv_cache_usage_perc > 0.95
        for: 1m
      - alert: VllmQueueBacklog           # 容量不足：排队持续不为零
        expr: vllm:num_requests_waiting > 0
        for: 5m
      - alert: VllmTTFTp99High            # SLO 失守：p99 TTFT 超过 500 ms
        expr: |
          histogram_quantile(0.99,
            sum by (le) (rate(vllm:time_to_first_token_seconds_bucket[5m]))) > 0.5
        for: 5m
      - alert: VllmPrefixHitRateLow       # 前缀命中率塌落（例如负载均衡策略被改成了轮询）
        expr: |
          rate(vllm:prefix_cache_hits[10m]) / rate(vllm:prefix_cache_queries[10m]) < 0.3
        for: 15m
```

四条告警分别对应“资源将尽、容量不足、SLO 失守、缓存失效”，和 Go 服务的“连接池将满、队列积压、P99 超标、缓存命中率下降”一一对应。注意 `histogram_quantile` 前要先按 `le` 聚合多实例的分桶。面板搭好之后，用第 8 章的压测脚本打一轮流量，四条曲线应该同时动起来。

9.10 常见坑

坑	现象	对策
把 GPU-Util 当真实利用率	Util 长期 100%，加并发吞吐还在涨；或 Util 100% 但吞吐很低	看 `DCGM_FI_PROF_SM_ACTIVE`，结合 tokens/s 与显存带宽判断
量化格式与 GPU 代际错配	FP8 模型部署到 A100 后吞吐不升	确认 GPU 有 FP8 Tensor Core；确认 INT4 走高效 kernel；用现役工具（llm-compressor、GPTQModel）制作模型
小集群照搬 PD 分离	拆池后吞吐下降、P99 变差	几十卡以下先调单节点参数；确认有 RDMA 网络、且预填充/解码比例长期失衡再考虑
按平均上下文做容量规划	偶发长文档请求让 KV 池瞬间见底，大面积抢占	按 P99 上下文规划；用 `max_model_len` 或业务侧截断限制上限；长文档与短对话分池
Ray Dashboard 暴露	安全扫描报 8265 可未授权访问	升级到 ⩾ 2.52.0 并开启 token 认证；Dashboard 只绑定本机或放在认证代理后
照抄过时的指标名	面板一片空白，告警永不触发	以当前版本的 `/metrics` 输出和官方指标文档为准（9.8 节）
量化后不做评测直接上线	某类业务输出质量明显下降	量化模型先过业务评测集，评测管线和监控一样常驻

9.11 延伸阅读

- vLLM 文档：Metrics 设计、量化、分离式预填充（Disaggregated Prefilling）。
- “Give Me BF16 or Give Me Death? Accuracy-Performance Trade-Offs in LLM Quantization”（arXiv:2411.02355）。
- DeepSeek-V3/R1 推理系统概览（2025-02）；Mooncake（FAST 2025）；DistServe（OSDI 2024）。
- NVIDIA Dynamo、llm-d 项目文档；DCGM exporter 指标手册。
- Ray 安全公告 GHSA-q279-jhrf-cc6v 与 Anyscale 关于 CVE-2025-62593 的说明（2026-08-19）。

第 10 章 90 天路线图与实战项目

前 9 章把零件都交到了你手上。本章回答最后一个问题：怎么在 90 天里把这些零件变成肌肉记忆。答案是路线图加项目制，和当年学 Go 的路径同构：先写 CLI 和小工具，再写服务，最后读 runtime 源码。

10.1 本章目标

- 说清三个阶段的依赖关系和每个阶段可检验的验收标准。
- 拿到 8 个递进项目的目标、验收标准、预估耗时与硬件需求，并能按自己的条件裁剪顺序。
- 按 torchtitan → vLLM → transformers 的顺序读源码。
- 客观评估 Go 背景的优势与短板。

10.2 设计原则

Go 世界的成长经验	本路线图的对应设计	同一个逻辑
先写 CLI 和爬虫，再写微服务	第 1 – 30 天只做语法、工程化与单机脚本	先建立语言直觉，再叠加系统复杂度
用 side project 驱动学习	8 个项目都要求可验收的产出	知识变成可运行的产物才算学会
学并发先懂调度器	推理先讲 PagedAttention，再调引擎参数	机制先于 API
读 net/http 和 runtime 源码进阶	torchtitan / vLLM / transformers 阅读清单	读生产级代码是进阶的捷径

Go 工程师学 Python 最大的风险不是学不会，而是用 Go 的标准要求自己：想一次性“正确地”掌握一切再动手。Python 生态的正确姿势是“先跑起来再迭代”：notebook 里验证想法，脚本里跑通链路，最后才做工程化。所以本章的验收标准都写成“能演示的产出”，而不是“能默写的知识”。这和“先 profile 再优化”是同一条纪律：让证据决定下一步。

10.3 三个 30 天

三个阶段的功能不同，顺序不能调换：

- **第 1–30 天，消除陌生感。** 让 Python 的写法、工具链、错误信息不再消耗你的注意力。
- **第 31–60 天，建立机制理解。** 训练栈（autograd、并行策略、显存构成）是概念密度最高的部分，需要整块时间。
- **第 61–90 天，回到主场。** 推理服务与 AI Infra 本质是高并发系统工程，Go 经验在这里产生复利。

训练栈依赖 Python 熟练度（第 5 章的参数注册、第 6 章的进程编排都建立在第 2、3 章之上）；推理是训练的逆问题，不懂前向、反向与显存构成，就无法理解 KV cache 为什么存在。节奏按每天 2 – 3 小时设计：工作日推进章节，周末做项目。

第 1–30 天：语法与工程化（第 1–4 章）

周	重点	产出
第 1 周	第 1 章：语法、推导式、生成器、七个坑	跑通第 1 章的全部实践；项目 1 开工
第 2 周	第 2 章：名称绑定、引用计数、GIL、描述符	在本机复现第 2 章的 GIL 对比实验，能向同事讲清“为什么 AI 代码大多不关心 GIL”
第 3 周	第 3 章：类型、pydantic v2、asyncio、uv、pytest	项目 1 交付；项目 2 开工
第 4 周	第 4 章：数组内存模型、CUDA 概念、Triton	向量化与访存局部性的对比报告

验收：不看文档写出带 `Protocol` 标注、通过 pyright 的模块；压测客户端能稳定发出指定并发并正确统计分位数；能画出 ndarray 的“指针 + 形状 + 步长”结构，解释视图与拷贝的代价差异；项目 1、2 通过验收。

第 31–60 天：训练栈（第 5–7 章）

周	重点	产出
第 5 周	第 5 章：张量内存模型、autograd、`nn.Module`	手写 autograd 并与 PyTorch 逐位核对
第 6 周	第 5 章后半：DataLoader、混合精度、`torch.compile`	项目 3：可复现的训练性能基准仓库
第 7 周	第 6 章：NCCL、并行策略、DDP、FSDP2、显存公式	项目 4：DDP/FSDP 结业项目 + 显存估算与实测对照
第 8 周	第 7 章：HF 生态、LoRA/QLoRA	跑通 Qwen3-0.6B LoRA SFT 全链路

验收：给定任意参数量，算出全量训练与 LoRA 的显存；解释 DDP 的分桶机制为什么能让通信与计算重叠；LoRA 模型能加载、能生成，能说出 r 和 alpha 的作用，能手算可训练参数量。

第 61–90 天：推理与 AI Infra（第 8–9 章）

周	重点	产出
第 9 周	第 8 章：PagedAttention、continuous batching、引擎参数	`vllm serve` 起服务，用项目 2 的脚本测出 TTFT/TPOT 曲线
第 10 周	第 8 章：压测口径、引擎选型	项目 5（网关）与项目 6（多轮对话服务）
第 11 周	第 9 章：KV 公式、量化、PD 分离判断	项目 8：KV 容量规划审计
第 12 周	第 9 章：编排与可观测	项目 7：微调 → 部署闭环，配齐四行一屏的监控面板

验收：从任意模型的 `config.json` 算出每 token KV 字节并给出容量结论；网关交付限流、流式响应、Prometheus 指标三件套；给团队做一次 30 分钟的“推理服务容量规划”分享。

10.4 源码阅读：torchtitan → vLLM → transformers

这个顺序是刻意安排的。torchtitan 代码量最小、目标单一，适合先建立“生产级训练循环长什么样”的骨架认知；vLLM 规模中等、设计文档完善，而且与后端经验最同构（调度器、内存池、路由都是熟面孔），投入产出比最高；transformers 最大最杂，放在最后，带着具体问题去查，不要试图通读。

仓库	入口	建议顺序	收获
torchtitan	训练主循环	主循环 → 模型定义 → 并行化（parallelize）→ distributed 目录	并行策略如何注入模型；checkpoint 与容错
vLLM	`vllm/v1/` 目录	OpenAI 入口 → engine core → scheduler → KV cache manager → GPU model runner	一条请求的完整生命周期；PagedAttention 在代码里的形态
transformers	`src/transformers/models/qwen3/`	单模型的 modeling 文件 → `generation/` → `trainer.py`（按需）	任意模型的实现约定；PEFT 的注入点

具体文件路径随版本变化，以你克隆时的主分支为准；IDE 跳转加调试器断点比任何阅读指南都可靠。方法沿用读 Go 源码的习惯：读 vLLM 的调度器就像当年读 runtime 的调度器，先找主循环，再找状态机，最后看数据结构。建议安排在第 61 – 90 天，这时推理侧的概念已经就位，读代码是在验证理解，而不是扫盲。

10.5 实战项目清单

8 个项目按章节依赖递进，都建立在前 9 章的实践代码之上，是扩展而不是另起炉灶。

#	项目	依赖章节	预估耗时	硬件
1	Pythonic 日志分析管道	1 – 3	2 – 3 天	CPU
2	通用 LLM 流式压测客户端	3、8	3 – 4 天	CPU（可先对模拟服务开发）
3	训练性能基准仓库	4 – 5	3 – 5 天	CPU 或 Apple GPU 可跑；NVIDIA GPU 更佳
4	分布式训练结业项目	5 – 6	5 – 7 天	多卡；可用 gloo 在 CPU 上降级
5	OpenAI 兼容推理网关	3、8	4 – 6 天	单卡 24 GB
6	带前缀缓存的多轮对话服务	8 – 9	4 – 6 天	单卡 24 GB
7	LoRA 微调 + 部署闭环	7 – 8	5 – 7 天	微调可在 Apple GPU 上做；部署需单卡 24 GB
8	KV cache 容量规划审计	9	3 – 4 天	CPU（对账需要 GPU 实例）

项目 1：Pythonic 日志分析管道。 在第 1 章的生成器管道上扩展：pydantic v2 定义统计结果的结构并输出 JSON 报告；补全 pytest 测试，通过 pyright 检查；用 uv 管理。验收：处理 10 GB 日志时峰值内存恒定（用第 1 章的 `peak_rss_mib` 测），测试全绿，类型检查无错误。

项目 2：通用 LLM 流式压测客户端。 把第 8 章的 `bench_stream.py` 做成专业工具：按输出长度分桶统计 TTFT、TPOT、端到端延迟；支持按请求速率（而不只是固定并发）施压；输出 Prometheus 格式。验收：先对第 8 章的模拟服务调通，再对任意 OpenAI 兼容端点压到 256 并发，结果与 `vllm bench serve` 同场景对比，偏差可解释。这是后面所有实验的测量基础，值得做扎实。

项目 3：训练性能基准仓库。 把第 5 章的四档对比脚本工程化：配置用 pydantic 建模，结果落盘成表格，README 记录硬件与结论。验收：别人克隆后一条命令复现你的数据；报告要解释混合精度和 compile 各

自的收益来源，以及**在什么条件下它们没有收益**（第 5.7 节的反例就是很好的起点）。

项目 4：分布式训练结业项目。 在第 6 章的 DDP 示例上包一个小型 Transformer，先 DDP 跑通，再用 `fully_shard` 改造；完成显存估算，与 `torch.cuda.max_memory_allocated` 的实测逐项对照。验收：估算与实测偏差小于 20%，并能解释差异来源（激活、碎片、通信缓冲）。没有多卡时用 gloo 在 CPU 上跑通流程，报告注明哪些项无法验证。

项目 5：OpenAI 兼容推理网关。 包装 vLLM 引擎：FastAPI 实现 `/v1/chat/completions` 的子集，SSE 流式输出，asyncio 信号量限流，pydantic 校验请求。验收：与原生 `vllm serve` 的 TTFT 差距小于 10%；超过并发上限时返回 429 而不是无限排队；延迟指标暴露在 `/metrics`。这个项目让你理解引擎外面那层“经典后端”该长什么样，恰好是你最熟的部分。

项目 6：带前缀缓存的多轮对话服务。 在项目 5 上支持多轮对话：历史拼接策略、system prompt 模板化、多副本时的会话亲和路由，用 `prefix_cache_hits / prefix_cache_queries` 观测命中率。验收：固定 system prompt 场景下命中率超过 80%；做一组对照实验，展示关闭亲和路由后命中率的塌落。

项目 7：LoRA 微调 + 部署闭环。 串起第 7、8、9 章：用第 7 章的脚本在领域数据上做 LoRA SFT，合并权重，通过项目 5 的网关部署，用项目 2 的工具对比微调前后的行为与延迟。验收：“数据 → 训练 → 合并 → 部署 → 压测 → 报告”整条链路文档化，报告里至少包含一次失败与回滚的记录（例如 chat template 不一致、QLoRA 合并进 4 bit 基座导致效果下降）。出过一次事故并写进报告，价值远大于一次顺利跑通。

项目 8：KV cache 容量规划审计。 把第 9 章的脚本升级成审计工具：从 `config.json` 自动读取结构参数，输入目标 GPU 型号、并发与 SLO，输出容量报告；再与一个运行中的 vLLM 实例对账（启动日志里的 KV 池大小、`vllm:kv_cache_usage_perc` 的实测值）。验收：复现 Qwen3-8B 的 144 KiB/token；对账偏差可解释（CUDA Graph、预留、FP8 KV 开关）。

10.6 Go 工程师的优势与短板

优势，决定了你应该在哪里加杠杆：

优势	在 AI Infra 的兑现点
并发与调度直觉	continuous batching 就是迭代级调度；asyncio 是协作式的 goroutine
分布式系统经验	集合通信、并行策略、进程编排、弹性训练，都是老朋友换了名字
可观测与容量规划纪律	Prometheus 与 RED 方法论直接平移；KV 容量规划就是一次容量审计
工程化习惯	类型检查、测试、锁版本的纪律，让你一上手就超过多数脚本式代码的工程水位

短板，每一条都有具体的补偿方法：

1. **动态类型直觉缺失。** 看到 `**kwargs` 穿过六层调用，完全不知道里面是什么。对策：自己的代码一律类型检查 + pydantic 边界校验；读框架源码时直接下断点看运行时的值，比静态推断快得多。
2. **科学计算思维缺位。** 拿到数组下意识写 for 循环。对策：每次想写循环处理数值，先问能否向量化或广播；用第 4 章的“步长”视角思考内存布局。
3. **数值敏感度不足。** 遇到 NaN、溢出、loss scale 时没有直觉，当成逻辑 bug 排查半天。对策：反复跑第 5 章的混合精度实验；遇到 NaN 先怀疑数据类型与精度，再怀疑逻辑，这个顺序和 Go 里恰好相反。

4. **对“实验文化”水土不服。** 想把每个探索脚本都写成一次性正确的工程。对策：明确区分两种模式：探索期允许 notebook 和“脏脚本”，结论验证后再工程化。这不是降低标准，而是这个领域的标准工作流。

10.7 结语

90 天结束时，你的身份不是“会写 Python 的 Go 工程师”，而是一个理解训练与推理机制的系统工程师。Python 会在这 90 天里退到工具层：它不再是你需要“学”的东西，而是你观察显存、调度、并行、量化这些真问题时顺手的语言。真正的壁垒从来不是语法，而是机制理解加上工程判断。前者这本书已经交付，后者需要你用这 8 个项目亲手挣来。

附录 A 第 1 版勘误对照

页码为第 1 版 PDF 的文件页序。“严重度”中，**阻断**表示代码无法运行或结论错误，**事实**表示版本、日期、名称等与实际不符，**改进**表示表述不严谨或缺少前提。

A.1 代码无法运行或结果错误

#	第 1 版位置	问题	严重度	第 2 版处理
1	p28 – 29 §3 实践一	`OptimizerConfig.lr` 没有默认值，`TrainConfig` 的类定义里又写了 `OptimizerConfig()`，导入模块时即抛 `ValidationError`	阻断	3.4 节：`lr` 加默认值，改用 `default_factory`
2	p28 – 29 §3 实践一	跨字段校验缺少 `@model_validator(mode="after")` 装饰器和方法定义，代码悬空	阻断	3.4 节补全；新增 `extra="forbid"` 拒绝拼错的字段
3	p30 – 31 §3 实践二	打印用到变量 `wall`，但从未计算	阻断	3.5 节补上，并改用最近秩法计算分位数
4	p47 – 48 §5 实践 2	计时循环只有前向，没有反向和优化器更新，却标为训练耗时	阻断	5.7 节：计时循环包含完整训练步
5	p48 §5 实践 2	计时后无条件调用 `torch.cuda.synchronize()`，CPU 上直接报错；文中却说“CPU 可全程跑通”	阻断	按设备选择同步方式；在 CPU 和 MPS 上实测
6	p47 §5 实践 2	四档共用一个模型对象，前一档的训练状态和 compile 包装带入下一档	改进	每档重新建模型
7	p54 – 55 §6 实践 1	两页之间缺失训练循环，`loss`、`step` 未定义	阻断	6.6 节补全并实测，增加 `--same-data` 对照与 `try/finally`
8	p72 – 73 §8 实践 3	`t_start` 未赋值，第一个请求即 `NameError`	阻断	8.11 节修复
9	p72 – 73 §8 实践 3	声明了信号量但没有 `async with sem`，`--concurrency` 失效，所有请求同时发出	阻断	修复，并补充模拟服务以便无 GPU 时调通
10	p74 §8 实践 3	使用未定义的 `total_tokens`	阻断	修复；改用 `stream_options.include_usage` 获取精确 token 数
11	p80 – 81 §9 实践 1	使用未定义的 `per_tok_kib`	阻断	9.4 节修复；改为直接读 `config.json`
12	p10 – 11 §1 实践二	`loss_windows` 不产出末尾不足一个窗口的余量；打印区间硬编码 `+99_999`	改进	1.7 节修复，打印真实起止 step
13	p37 – 38 §4 实践一	跨步实验比较的是“读 1/64 元素”与“读全部元素”，预期“跨步更慢”；本机复现结果相反（0.20 ms 对 1.88 ms）	阻断	4.4 节改为读取相同数量的元素，实测按列读慢 11.4 倍
14	p37 §4 实践一	纯 Python 循环的计时包含了 `tolist()` 转换	改进	转换移出计时
15	p38 – 39 §4 实践二	用约 10 万元素测“带宽”，数据能放进 L2，测到的主要是启动开销	改进	4.7 节改用 6400 万元素并打印有效带宽
16	p62 – 63 §7	QLoRA 未调用 `prepare_model_for_kbit_training`	改进	7.6 节补上

#	第 1 版位置	问题	严重度	第 2 版处理
17	p63 §7	对 4 bit 基座直接 `merge_and_unload()`	改进	重新加载 bf16 基座后合并
18	p20 §2 实践三	`__getattr__` 访问 `self._search_dir`，在对象未初始化（拷贝、反序列化）时会无限递归	改进	2.7 节加 `_` 前缀保护

A.2 事实与版本

#	第 1 版位置	第 1 版写法	实际情况（2026-10-22 核查）
19	p27 §3	pytest “当前 8.4.x 线”	当前为 9.1.1（9.0.0 于 2025-11 发布）
20	p58 – 59 §7	transformers “2026-04 首个 RC”	v5 首个 RC 为 2025-12-01，正式版 2026-01-26；当前 5.20.2
21	p58 §7	“官方建议升级到 ⩾ 5.3.0”（未说明是哪个漏洞）	对应 CVE-2026-4372（`config.json` 字段可绕过 `trust_remote_code`），修复于 5.3.0；Trainer 的 `torch.load` 漏洞 CVE-2026-1839 修复于 5.0.0rc3
22	p59 §7	PyPI 上 “mistralai 2.4.6 投毒”、“extra-huggingface 仿冒包”	无法从公开来源核实，第 2 版删除；改为给出可核实的 CVE
23	p62 §7	LoRA 可训练参数“约 0.3%”	实测 0.76%（4,587,520 / 600,637,440），7.4 节给出逐层算式
24	p53 §6	DeepSpeed “0.18.x 系列”	当前 0.19.8
25	p66 – 75 §8	`vllm:gpu_cache_usage_perc`、`vllm:prefix_cache_hit_rate`	V1 中分别为 `vllm:kv_cache_usage_perc`；没有命中率指标，用 `prefix_cache_hits / prefix_cache_queries` 计算
26	p79 §9	`vllm:time_per_output_token_seconds`	V1 中为 `vllm:request_time_per_output_token_seconds` 与 `vllm:inter_token_latency_seconds`
27	p69 §8	`gpu_memory_utilization` “默认 0.9，2026 年从 0.95 下调”	当前文档标注默认 0.92；以所用版本的 `--help` 为准
28	p79 §9	Ray “最新稳定版 2.58（2026 年 8 月）”	当前 2.59.0（2026-10-02）
29	p79、p84 §9	Ray 漏洞“需 ⩾ 2.56”	CVE-2025-62593 修复于 2.52.0；token 认证默认关闭，需手动开启
30	p79 §9	“Anyscale 于 2026 年 7 月被 Nscale 收购，约 16.5 亿美元”	2026-07-30 宣布收购协议，预计下半年完成；官方未披露金额，16.5 亿美元为媒体报道
31	p78 §9	“DeepSeek-V3 的生产架构是 3 个 prefill 节点 + 9 个 decode 节点”	DeepSeek 官方：预填充部署单元 4 节点（32 张 H800），解码 18 节点（144 张 H800）
32	p78 §9	PD 分离“约 6.4× 吞吐、延迟方差降约 20×、成本省 15 – 40%”	来源无法核实，删除
33	p16 §2	3.14 “最新补丁 3.14.7”	3.14.9（2026-10-21）
34	p4、p16 §1 – 2	未提示 3.15 的生态兼容性	Triton 3.8.0、vLLM 0.32.0 均要求 Python < 3.15
35	p24 §3	类型检查器的规范符合度百分比、“1500 万行 12.5 秒”等第三方数字	无法核实，删除；改为列出当前版本与定位

#	第 1 版位置	第 1 版写法	实际情况（2026-10-22 核查）
36	p25 § 3	pydantic “PyPI 累计下载约 176 亿次”	无法核实，删除
37	p15 § 2	free-threading “4 线程约 3 – 4x，16 线程 10 – 14x” 等数字	改为本机实测（8 线程 4.8×）与官方的单线程开销口径
38	p15 § 2	“matplotlib 设计上会重开 GIL”	无法核实，删除
39	p45 § 5	“2.13 起 Inductor 新增 CUTLASS 第二条 GPU 代码生成路径”	无法核实，删除
40	p69 – 70 § 8	TGI “最后一个版本 v3.3.7”；SGLang “部署在 40 万+ GPU”	无法核实，删除；保留可核实的“2025-12-11 进入维护模式”
41	p71 – 72 § 8	“2026 年起 vLLM 还兼容 Anthropic API”	无法核实，删除

A.3 表述与交叉引用

#	第 1 版位置	问题	第 2 版处理
42	p8 – 9 § 1	“async/await（第 5 章）”、“第 5 章会把它改写成 asyncio”	asyncio 在第 3 章
43	p9 § 1	示例 URL 列表与预期输出不一致（输出里出现了列表中没有的 go.dev）	统一
44	p17 § 2	数据描述符定义为“同时定义了 `__get__` 和 `__set__`”	只要定义了 `__set__` 或 `__delete__` 就是数据描述符
45	p42 § 5	用 `t.storage()` 判断共享内存	`TypedStorage` 已弃用，改用 `untyped_storage()`
46	p44 § 5	AMP 示例同时使用 bf16 与 `GradScaler`	说明 `GradScaler` 只有 fp16 需要
47	p45 § 5	“AMP 约 1.5 – 2.5×，compile 再叠加 1.2 – 1.5×”	给出实测：MPS 上 AMP 1.2×，compile 无收益；CPU 上 bf16 AMP 慢约 200 倍，并解释前提
48	p53 § 6	激活检查点“把激活从 O(L) 压到 O(√L)”	区分理论最优分段与常见的逐层检查点
49	p56 § 6	“跨节点 DP 在大模型下不可行”	改为按模型规模分别讨论
50	p70 – 71 § 8	第 8 章用 Qwen2.5，第 7 章用 Qwen3	统一为 Qwen3

附录 B 版本核查表

以下数据于 2026-10-22 从 PyPI、python.org 发布日程（endoflife.date 汇总）和各项目官方文档核查。AI 基础设施版本变化很快，使用时请以官方为准。

B.1 Python 解释器

版本	首发	最新补丁	维护状态	结束支持
3.15	2026-10-09	3.15.x	bugfix	2031-10
3.14	2025-10-07	3.14.9（2026-10-21）	bugfix（至 2027-10）	2030-10
3.13	2024-10-07	3.13.x	2026-10-01 起只修安全问题	2029-10
3.12	2023-10-02	3.12.x	只修安全问题	2028-10
3.10	2021-10-04	3.10.x	已结束	2026-10

B.2 主要库（PyPI 最新稳定版）

包	版本	发布日期	Python 要求	备注
numpy	2.5.4	2026-10-10	⩾ 3.12	
torch	2.14.1	2026-09-30	⩾ 3.10	
triton	3.8.0	2026-08-28	3.10 – 3.14	不支持 3.15
pydantic	2.14.1	2026-10-11	⩾ 3.10	
pytest	9.1.1	2026-06-19	⩾ 3.10	
mypy	2.4.0	2026-10-01	⩾ 3.10	2.0 于 2026-05 发布
pyright	1.1.414	2026-09-10		
pyrefly	1.4.0	2026-10-15		1.0 于 2026-05-12 发布
ty	0.0.90	2026-10-20		预发布阶段
ruff	0.17.1	2026-10-15		
uv	0.13.5	2026-10-22		
httpx	0.28.1	2024-12-06		1.0 处于预发布
transformers	5.20.2	2026-10-19	⩾ 3.10	安全基线 ⩾ 5.3.0
peft	0.21.2	2026-10-01	⩾ 3.10	
trl	1.16.0	2026-10-15	⩾ 3.11	
datasets	5.1.0	2026-10-05	⩾ 3.10	
accelerate	1.15.0	2026-09-09	⩾ 3.10	
tokenizers	0.23.3	2026-10-09	⩾ 3.10	1.0 处于 RC
deepspeed	0.19.8	2026-10-20		
megatron-core	0.19.2	2026-09-18	⩾ 3.12	

包	版本	发布日期	Python 要求	备注
torchtitan	0.3.0	2026-09-03	⩾ 3.11	
vllm	0.32.0	2026-10-18	3.11 – 3.14	不支持 3.15
sglang	0.5.22	2026-10-16	⩾ 3.11	
ray	2.59.0	2026-10-02	⩾ 3.10	安全基线 ⩾ 2.52.0
ai-dynamo	1.5.1	2026-10-07	⩾ 3.10	1.0 于 2026-03-13 发布
bitsandbytes	0.50.2	2026-08-27	⩾ 3.10	
llmcompressor	0.14.0	2026-09-22	⩾ 3.10	AutoAWQ 的接替者
gptqmodel	7.6.0	2026-10-12	⩾ 3.10	AutoGPTQ 的接替者
autoawq	0.2.9	2025-05-11		已停止维护
auto-gptq	0.7.1	2024-03-01		已归档

B.3 关键安全公告

编号	组件	影响	修复版本
CVE-2026-4372	transformers	恶意 `config.json` 绕过 `trust_remote_code`，`from_pretrained` 下载并执行任意代码	5.3.0
CVE-2026-1839	transformers	`Trainer` 恢复 RNG 状态时 `torch.load` 未加 `weights_only=True`	5.0.0rc3
CVE-2025-62593	Ray	Dashboard 的 User-Agent 检查可被绕过，配合 DNS rebinding 可远程执行命令；2026-08-17 列入 CISA KEV	2.52.0（需手动开启 token 认证）

附录 C 随书代码索引

全部代码位于随书目录 `code/`，实测输出位于 `out/`。

章	文件	内容	运行条件
1	`ch01/fetch_titles.py`	Go 直译版与 Pythonic 版并发抓取	CPU，联网
1	`ch01/pipeline.py`	生成器管道；`gen`/`run`/`eager` 三种模式	CPU
1	`ch01/pitfalls.py`	七个常见坑的复现	CPU
2	`ch02/refcount_demo.py`	引用计数与循环 GC	CPU
2	`ch02/gil_demo.py`	常规构建与 free-threaded 构建的多线程扩展性	CPU，3.14 与 3.14t
2	`ch02/descriptors.py`	描述符字段校验、`__getattr__` 懒加载	CPU
2	`ch02/py315_features.py`	3.15 新特性验证	Python 3.15
3	`ch03/typing_demo.py`、`typing_bad.py`	Protocol、PEP 695；检查器对比	CPU
3	`ch03/config.py`、`test_config.py`	pydantic v2 配置与 pytest	CPU
3	`ch03/async_pitfalls.py`	阻塞 loop、忘记 await	CPU
3	`ch03/bench.py`	asyncio 并发压测客户端	CPU
3	`ch03/concurrency_matrix.py`	线程池 / 进程池 / 子解释器池对比	CPU，3.14+
4	`ch04/bench_numpy.py`	向量化、视图、访存局部性、NumPy 2 变更	CPU
4	`ch04/vector_add.py`	Triton vector add	NVIDIA/AMD GPU
5	`ch05/tensor_basics.py`	张量内存模型与 autograd 规则	CPU
5	`ch05/micrograd.py`	手写 autograd 并与 PyTorch 核对	CPU
5	`ch05/train_bench.py`	eager / AMP / compile 对比	CPU、MPS 或 CUDA
6	`ch06/ddp_demo.py`	torchrun + DDP	CPU（gloo）或多 GPU
6	`ch06/memory_calc.py`	显存与通信估算	CPU
7	`ch07/sft_lora.py`	Qwen3-0.6B LoRA SFT 全链路	MPS、CUDA 或 CPU
8	`ch08/mock_llm_server.py`	模拟 OpenAI 兼容流式服务	CPU
8	`ch08/bench_stream.py`	流式压测（TTFT/TPOT）	CPU
9	`ch09/kv_capacity.py`	KV cache 容量规划	CPU